TUNING MODEL COMPLEXITY USING CROSS-VALIDATION FOR SUPERVISED LEARNING

by

Olcay Taner Yıldız

B.S, in CmpE., Boğaziçi University, 1997

M.S, in CmpE., Boğaziçi University, 2000

Submitted to the Institute for Graduate Studies in
Science and Engineering in partial fulfillment of
the requirements for the degree of
Doctor of Philosophy

Graduate Program in Computer Engineering

Boğaziçi University

2005

TUNING MODEL COMPLEXITY USING CROSS-VALIDATION FOR SUPERVISED LEARNING

APPROVED BY:

Prof. Ethem Alpaydın (Thesis Supervisor)	.....
Prof. H. Levent Akın	.....
Assoc. Prof. Taner Bilgiç	.....
Prof. Filiz Güneş	.....
Prof. Fikret Gürgen	.....

DATE OF APPROVAL: 11.03.2005

ACKNOWLEDGEMENTS

I would like to thank Prof. Dr. Ethem Alpaydın for his supervision in this Ph.D. study and for his support and advises. I am also grateful to all my friends, my family, my teachers and especially my love AYSEL (ISELL). I would also thank to all Matlab helpers including Oya Aran, Onur Dikmen, Berk Gökberk, Itır Karaç, Mehmet Aydın Ulaş. I also want to thank Arzucan Özgür for her support in printing and submitting this thesis. Without their support, this thesis would not been accomplished.

This thesis has been supported by Boğaziçi University Scientific Research Projects 02A104D and 03K120250.

Dünyaya hoş geldin sevgili oğlum Oğuz Kerem Yıldız.

ABSTRACT

TUNING MODEL COMPLEXITY USING CROSS-VALIDATION FOR SUPERVISED LEARNING

In this thesis, we review the use of cross-validation for model selection and propose the MultiTest method which solves the problem of choosing the best of multiple candidate supervised models. The MultiTest algorithm orders supervised learning algorithms (for classification and regression) taking into account both the result of pairwise statistical tests on expected error, and our prior preferences such as complexity of the algorithm. In order to validate the MultiTest method, we compared it with Anova, Newman-Keuls algorithms which check whether multiple methods have the same expected error. Though Anova and Newman-Keuls results can be extended to find a "best" algorithm, this does not always work. On the other hand, our proposed method is always able to find an algorithm as the "best" one.

By using MultiTest method, we try to solve the problem of optimizing model complexity. For doing this, either we compare all possible models using MultiTest and select the best model or if the model space is very large, we make an effective search on the model space via MultiTest. If all possible models can be searched, MultiTest-based model selection always selects the simplest model with expected error not significantly worse than any other model.

We also propose a hybrid, omnivariate architecture, for decision tree induction and rule induction. This is a hybrid architecture that contains different models at different places matching the complexity of the model to the complexity of the data reaching that model. We compare our proposed MultiTest-based omnivariate architecture with the well-known techniques for model selection on standard datasets.

ÖZET

GÖZETİMLİ ÖĞRENMEDE ÇAPRAZ GEÇERLEME İLE MODEL KARMAŞIKLIĞININ AYARLANMASI

Bu tezde, model seçiminde çapraz geçerleme kullanımını gözden geçirerek gözetimli modellerden en iyisini bulan MultiTest metodunu önerdik. MultiTest algoritması, gözetimli öğrenme algoritmalarını beklenen hata üzerindeki ikili istatistiksel testlerin sonuçlarına ve algoritmanın karmaşıklığı gibi önceliklere göre sıralar. MultiTest metodunu geçerlemek için ANOVA ve Newman-Keuls algoritmalarıyla karşılaştırdık. Bu algoritmalar metodların hata oranlarının aynı olup olmadığını kontrol eder. En iyi algoritmayı bulmak için kullanılabilseler bile, bu her zaman çalışmayabilir. Oysa, bizim önerdiğimiz metod her zaman en iyiyi bulabilir.

MultiTest metodunu model karmaşıklığını eniyilemede kullanmaya çalıştık. Bunun için ya tüm olası modelleri MultiTest'le karşılaştırdık ve en iyi modeli seçtik ya da (model uzayı genişse) MultiTest'i kullanarak model uzayında etkili bir arama yaptık. Tüm modeller aranabildiğinde, MultiTest diğerlerinden anlamlı bir şekilde kötü olmayan en basit modeli seçer.

Tezde, ayrıca karar ağacı ve kural çıkarımı için karma, tüm değişkenli bir yapı önerdik. Bu yapı, modelin karmaşıklığını oraya ulaşan verinin karmaşıklığına uyduran, farklı yerlerde farklı modellerin olabildiği karma bir yapıdır. Önerdiğimiz MultiTest'e dayalı, çok değişkenli yapıyı çok bilinen model seçme teknikleriyle standart veri kümeleri üzerinde karşılaştırdık.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF SYMBOLS/ABBREVIATIONS

C_i	Class i
d	Number of inputs
E_g	Generalization error
E_t	Training error
$\hat{f}(\mathbf{x})$	Estimator to the unknown function $g(\mathbf{x})$
$g(\mathbf{x})$	Unknown function
h	Number of free parameters
K	Number of classes
L	Loss function
M	Model
$\boldsymbol{m}$	Estimator to the mean vector
N	Number of training instances
N_i	Number of training instances of class i
r	Required output
$\mathbf{S}$	Estimator to the covariance matrix
V	Validation sample
X	Training sample
$\mathbf{x}$	Input vector
x	Input
x_i	Input variable i
y	Output
$\mathcal{L}$	Loglikelihood
μ	Mean
σ^2	Variance
θ	Model parameters
AIC	Akaike Information Criterion
BIC	Bayesian Information Criterion

CV	Cross-validation
DEE	Direct Eigenvalue Estimator
FPE	Final Prediction Error
KGV	Kernel Generalized Variance
LDA	Linear Discriminant Analysis
MAP	Maximum A Posteriori
MCMC	Markov Chain Monte Carlo
MDL	Minimum Description Length
MLP	Multilayer Perceptron
NFL	No Free Lunch
QDA	Quadratic Discriminant Analysis
PCA	Principal Component Analysis
RBF	Radial Basis Function
SEB	Smallest Empirical Bound
SIC	Subspace Information Criterion
SRM	Structural Risk Minimization
SVM	Support Vector Machine
VC	Vapnik-Chervonenkis
VM	Vapnik's Measure

1. INTRODUCTION

Nowadays, with increasing use of computers in the industry, enourmous amount of data is collected in databases. This stored data is useful only if it is analyzed by intelligent techniques or tools and turned into information. If we knew the exact relationships in the data, we would write the code to process it and we would not need any analysis. But since life is not exact and we do not know the exact relationships, causes or effects in the data, we can not write the code to make use of it.

Although life is not exact, it is not random either. Though we do not know exactly the underlying process which generates the data, we know that it is not random. So we hope that we can construct a good approximation and we may able to make further predictions based on our approximation. Therefore we collect data and use machine learning (ML) techniques to answer questions from the data.

Data mining is one of the possible application areas of ML. In data mining, the data in large databases is processed to generate simple, explonatory models. For example, banks analyze past data of their customers to predict their future behavior, telecommunication companies analyze the call patterns of their customers improve the quality of service, supermarkets extract associations between products bought by customers for improving their selling strategies.

ML is also a very important part of artificial intelligence. Using ML, one can produce intelligent agents, which learn, that is, adapt to the changing environment. These agents will not be hard coded for each environment, they will change their behavior according to their environment.

Supervised learning, unsupervised learning and reinforcement learning are three main learning categories of ML. In supervised learning, the task is to learn the mapping function $\hat{f}(\mathbf{x})$ from the input data $\mathbf{x}$ to the output data (given by the supervisor) $\mathbf{y}$. In unsupervised learning, the goal is to directly estimate the properties or structure

of the input data without the help of a supervisor. In reinforcement learning, the aim is to find the best *policy*, that is, the sequence of correct actions to the goal. In this thesis, we will focus on supervised learning algorithms.

1.1. Supervised Learning

Supervised learning from examples is described by three major components. First, there is a data generator which produces input vectors $\mathbf{x}$ independently from a fixed but an unknown distribution $p(\mathbf{x})$. The input vectors consist of d input variables which can be continuous ($x_j \in \Re$) or discrete ($x_j \in \{v_1, v_2, \ldots, v_L\}$), $j = 1, \ldots, d$. Second, a supervisor assigns an output vector $\mathbf{r}$ for every input vector $\mathbf{x}$ according to a fixed but unknown conditional distribution function $p(\mathbf{r}|\mathbf{x})$. Third, a learner which is able to produce an estimator $\hat{f}(\mathbf{x}|M, \theta)$ where M represents the model of the estimator and θ represents the parameters of the model M.

The problem of supervised learning is then choosing the function from the given set $\hat{f}(\mathbf{x}|M, \theta)$, which predicts the supervisor's response in the best way. The selection is based on a training set of N identically and independently distributed vectors (x_1, r_1), (x_2, r_2), ..., (x_N, r_N) drawn from the probability distribution $p(\mathbf{x}, \mathbf{r}) = p(\mathbf{x})p(\mathbf{r}|\mathbf{x})$.

Supervised learning problems can be divided into two categories, classification and regression.

1.1.1. Classification

In classification problems, the required output r takes one of K different values. Each of these values corresponds to a different class. Learning is finding the best discriminant function(s), $\hat{f}_i(\mathbf{x})$, that separates these K classes. The instance is assigned to the class having the largest discriminant value:

$$y = \arg\max_i \hat{f}_i(\mathbf{x}) \tag{1.1}$$

The 0-1 loss function L is defined as

$$L(r, y) = \begin{cases} 0, & \text{if } r = y \\ 1, & \text{if } r \neq y \end{cases} \tag{1.2}$$

The training error, E_t also called, the classification error rate, is defined as the average loss over all training sample

$$E_t = \frac{1}{N} \sum_{t=1}^{N} L(r^t, y^t) \tag{1.3}$$

where r^t and y^t denote the required and the estimated output values of the instance t respectively.

In some classification algorithms, one can estimate the posterior probabilities of the instance t as $P(r^t|\mathbf{x^t})$. Based on these probabilities, one can estimate the log-likelihood

$$\mathcal{L} = -2 \sum_{t=1}^{N} \log P(r^t|\mathbf{x^t}) \tag{1.4}$$

1.1.2. Regression

In regression problems, the aim is to find a real-valued function $\hat{f}(\mathbf{x})$ to approximate the output value from d input dimensions. The required output is assumed to be the sum of an unknown function $g(\mathbf{x})$ and random noise ϵ with zero mean and an unknown variance σ^2.

$$r = g(\mathbf{x}) + \epsilon \tag{1.5}$$

Several loss functions to measure the difference between the estimated output value and the required output value are defined in the literature. Two well-known loss functions

are absolute error and squared error

$$L(r, \hat{f}(\mathbf{x})) = \begin{cases} (r - \hat{f}(\mathbf{x}))^2, & \text{squared error} \\ |r - \hat{f}(\mathbf{x})|, & \text{absolute error} \end{cases} \tag{1.6}$$

The training error E_t, also called mean square error, is defined as the average loss over all training sample

$$E_t = \frac{1}{N} \sum_{t=1}^{N} L(r^t, \hat{f}(\mathbf{x^t})) \tag{1.7}$$

1.2. Bias-Variance Dilemma

Learning methods can be classified into two: Parametric learning procedures assume a prior model for the data at hand (Gaussian model, polynomial model, etc.) and nonparametric methods do not make such an assumption. Since parametric methods assume a prior model for data, this may result in large error, called the bias error, when the data does not come from the assumed prior model. In the case of nonparametric methods, we need a large amount of data.

Whether we use a parametric or nonparametric learning approach, we have the model complexity problem. Choosing the optimal model complexity is an unsolved problem. If we assume a complex model we can easily learn the training data but we risk overfitting, that is, the model does not learn a general rule but the particular data we have and gives large error on unseen test data. If we assume a simple model, even the training data may not be learned well and error will be large on both training and test data.

Let us assume we are trying to solve a regression problem, and as we said in Section 1.1.2, the output is assumed to be the sum of an unknown function $g(\mathbf{x})$ and random noise ϵ with zero mean and an unknown variance σ^2. The expected error at an input point x_0 which uses a square-error loss function is $E[(r - \hat{f}(\mathbf{x_0}))^2 | x_0]$ where

$\hat{f}(\mathbf{x_0})$ is our estimate at x_0 and could be decomposed into three parts:

$$E[(r - \hat{f}(\mathbf{x_0}))^2|x_0] = \underbrace{\sigma^2}_{noise} + \underbrace{(E[\hat{f}(\mathbf{x_0})] - f(\mathbf{x_0}))^2}_{bias} + \underbrace{E[\hat{f}(\mathbf{x_0}) - E[\hat{f}(\mathbf{x_0})]]^2}_{variance} \quad (1.8)$$

The first part is the irreducible error that comes from the noise. The second part is the *bias*, how much the estimated output value (averaged over training sets) varies around the unknown true value. The third part is the *variance* which measures how much $\hat{f}(\mathbf{x_0})$ fluctuates around its expected value.

We can reduce the bias part of the error by using more and more complex functions but this has the effect of increasing variance. Or, we can reduce the variance by using simpler functions, but this risks increasing bias. The bias-variance dilemma says that there is always a trade-off between the bias and the variance of an estimator. If you decrease the bias, you will increase the variance and overfit the data; if you decrease the variance, you will probably increase the bias and underfit the data.

An example is given in Figure 1.2 where we have seven data points in two dimensions. Our prior information for this data gives us a polynomial model. If we fit a sixth degree polynomial to these data, we will have zero training error but large test error and poor approximation (large variance). If we fit a first degree polynomial namely a line to these data, we will have large training and test errors and poor approximation (large bias) (Figure 1.2).

Model selection is choosing the model with the right complexity. In this example, the complexity depends on the order of the polynomial. Given a certain order, regression allows us to find the parameters of the model, e.g., coefficients of the polynomial. But before this, the correct model complexity, e.g., the order of the polynomial, should be chosen. Machine learning algorithms are generally for optimizing the parameters of an assumed model (from a space of possible models); model selection is for finding the right model in the space of possible models, e.g., polynomials. This thesis is on this second problem.

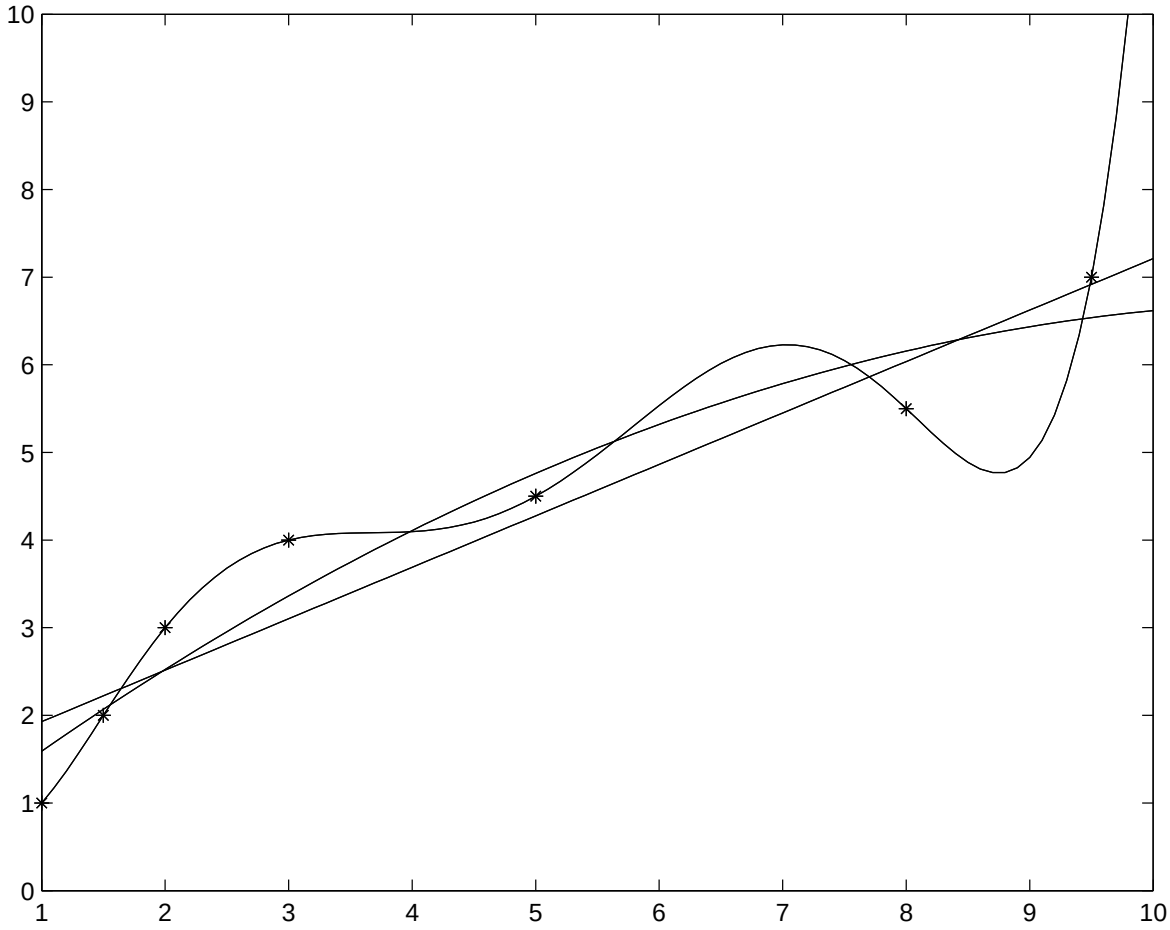


Figure 1.1. Polynomial regression: Polynomial of order 1, 2, 6 fitted to an example dataset of seven points

1.3. Outline of the Thesis

This thesis is organized as follows: In Chapter 2, we define the model selection problem and survey different solutions. These include penalization techniques, early stopping criteria, Bayesian technique, structural risk minimization, and cross-validation. In Chapter 3, we propose the MultiTest algorithm using cross-validation which orders and finds the best of multiple supervised learning algorithms. We also propose the combined 5×2 t test which could be used as a pairwise statistical test to compare the expected errors of supervised learning algorithms. We also show how it is possible to use the MultiTest algorithm as a tool in model selection and apply it to

the didactic example of polynomial regression.

Chapter 4 reviews and extends different learning algorithms such as decision trees and rule induction. In Chapter 5, we show how we can incorporate our model selection algorithm to these learning algorithms for tuning the model complexity. Chapter 6 gives the simulation results on several standard classification and regression datasets. First we compare our cross-validation-based model selection with other model selection techniques on the didactic examples of polynomial regression and classification. We give experimental results on the MultiTest algorithm using five classification algorithms on thirty datasets and five regression algorithms on eleven datasets. Then we give the comparison results of cross-validation-based model selection with other model selection techniques on the learning algorithms we have given in Chapter 5. We conclude and propose future work in Chapter 7.

2. MODEL SELECTION TECHNIQUES

As we have mentioned in Chapter 1, the training error

$$E_t = \frac{1}{N}\sum_{t=1}^{N} L(r^t, \hat{f}(\mathbf{x^t})) \tag{2.1}$$

is not the correct estimate of the generalization error (true error) because the same data (training data) is used to find the function estimate and to evaluate its error. The learning algorithms are designed to fit the training data, and their errors on this same data will be an optimistic estimate of the generalization error. To get the generalization error E_g, one can add an estimate of the difference to the training error which is called the *optimism* by Hastie *et al.* [1]:

$$E_g = E_t + \text{optimism} \tag{2.2}$$

For squared error loss function, the expected training error is,

$$\begin{aligned}
E_t &= \frac{1}{N}\sum_{t=1}^{N} E_r[(r^t - \hat{f}(\mathbf{x^t}))^2] \\
&= \frac{1}{N}\sum_{t=1}^{N} E_r[(r^t - g(\mathbf{x^t}) + g(\mathbf{x^t}) - E[\hat{f}(\mathbf{x^t})] + E[\hat{f}(\mathbf{x^t})] - \hat{f}(\mathbf{x^t}))^2] \\
&= \frac{1}{N}\sum_{t=1}^{N} E_r[(r^t - g(\mathbf{x^t}))^2 + (g(\mathbf{x^t}) - E[\hat{f}(\mathbf{x^t})])^2 + (E[\hat{f}(\mathbf{x^t})] - \hat{f}(\mathbf{x^t}))^2] \\
&\quad +2E_r[(r^t - g(\mathbf{x^t}))(g(\mathbf{x^t}) - E[\hat{f}(\mathbf{x^t})])] \\
&\quad +2E_r[(r^t - g(\mathbf{x^t}))(E[\hat{f}(\mathbf{x^t})] - \hat{f}(\mathbf{x^t}))] \\
&\quad +2E_r[(g(\mathbf{x^t}) - E[\hat{f}(\mathbf{x^t})])(E[\hat{f}(\mathbf{x^t})] - \hat{f}(\mathbf{x^t}))] \\
&= \frac{1}{N}\sum_{t=1}^{N} \sigma^2 + (g(\mathbf{x^t}) - E[\hat{f}(\mathbf{x^t})])^2 + E[(E[\hat{f}(\mathbf{x^t})] - \hat{f}(\mathbf{x^t}))^2] \\
&\quad -\frac{2}{N}\,\mathrm{cov}(r^t, \hat{f}(\mathbf{x^t}))
\end{aligned} \tag{2.3}$$

where cov shows the covariance.

The generalization error is extra-sample error since the test instances do not coincide with the training instances. Without loss of generality, we can assume that the input values of the test instances are the same as the input values of the training instances but the corresponding output values are different. If we represent the new output values by r^t_{new}, the expected generalization error is:

$$\begin{aligned}
E_g &= \frac{1}{N}\sum_{t=1}^{N} E_{rnew}E_r[(r^t_{new} - \hat{f}(\mathbf{x^t}))^2] \\
&= \frac{1}{N}\sum_{t=1}^{N} E_{rnew}E_r[(r^t_{new} - g(\mathbf{x^t}) + g(\mathbf{x^t}) - E[\hat{f}(\mathbf{x^t})] + E[\hat{f}(\mathbf{x^t})] - \hat{f}(\mathbf{x^t}))^2] \\
&= \frac{1}{N}\sum_{t=1}^{N} E_{rnew}[(r^t_{new} - g(\mathbf{x^t}))^2 + (g(\mathbf{x^t}) - E[\hat{f}(\mathbf{x^t})])^2 + E_r[(E[\hat{f}(\mathbf{x^t})] - \hat{f}(\mathbf{x^t}))^2] \\
&\quad +2((r^t_{new} - g(\mathbf{x^t}))(g(\mathbf{x^t}) - E[\hat{f}(\mathbf{x^t})])) \\
&\quad +2((r^t_{new} - g(\mathbf{x^t}))E_r[E[\hat{f}(\mathbf{x^t})] - \hat{f}(\mathbf{x^t})]) \\
&\quad +2((g(\mathbf{x^t}) - E[\hat{f}(\mathbf{x^t})])E_r[E[\hat{f}(\mathbf{x^t})] - \hat{f}(\mathbf{x^t})])] \\
&= \frac{1}{N}\sum_{t=1}^{N} E_{rnew}[(r^t_{new} - g(\mathbf{x^t}))^2] + (g(\mathbf{x^t}) - E[\hat{f}(\mathbf{x^t})])^2 + (E[\hat{f}(\mathbf{x^t})] - \hat{f}(\mathbf{x^t}))^2 \\
&= \frac{1}{N}\sum_{t=1}^{N} \sigma^2 + (g(\mathbf{x^t}) - E[\hat{f}(\mathbf{x^t})])^2 + (E[\hat{f}(\mathbf{x^t})] - \hat{f}(\mathbf{x^t}))^2 \qquad (2.4)
\end{aligned}$$

Therefore, the amount of difference between the true error and the training error depends on the amount of interaction between r^t and its estimate $\hat{f}(\mathbf{x^t})$:

$$\begin{aligned}
\text{optimism} &= E_g - E_t \\
&= \frac{2}{N}\sum_{t=1}^{N} \text{cov}(r^t, \hat{f}(\mathbf{x^t})) \qquad (2.5)
\end{aligned}$$

2.1. Penalization

In this approach, we assume that there are large number of candidate models and to restrict the solution space, we introduce a penalization term. Here the prior infor-

mation about the model is introduced in the form of a penalty term. The penalization term is proportional to the complexity of the model. The performance of the model is the sum of the error of the model on the data and the complexity of the model. We want to minimize

$$P_{model} = E_{model} + \lambda C_{model} \tag{2.6}$$

where parameter λ controls the strength of the penalty term. If it is too large, the complexity of the model mainly determines the resulting function and simple models will be selected. If λ is too small, error will be the main cause in determining the resulting function, and complex models with small errors will be selected. Equations 2.2 and 2.6 are identical in the sense that

- We want to minimize the generalization error of the model
- Both equations include the training error of the model
- The complexity of the model and the optimism are proportional to each other. If the model is complex, the harder we fit the data and the covariance will be higher.

Different penalization techniques are introduced in the literature. Here we will briefly survey three of them, namely Akaike Information Criterion (AIC), Bayesian Information Criterion (BIC) and Minimum Description Length (MDL). Other penalization techniques can be found in [2] and [3].

2.2. Akaike Information Criterion

If the estimate $\hat{f}(\mathbf{x^t})$ is obtained by a linear fit with h inputs, the covariance in Equation 2.5 is:

$$\sum_{t=1}^{N} \text{cov}(r^t, \hat{f}(\mathbf{x^t})) = h\sigma^2 \tag{2.7}$$

The generalization error is given as:

$$E_g = E_t + 2\frac{h}{N}\sigma^2 \tag{2.8}$$

This sum is also called AIC or C_p. The optimal model is obtained by selecting the model which has the smallest generalization error according to this equation. Although AIC type model selection is simple to implement and to use, there are two major points that must be underlined:

- AIC is obtained under the linearity assumptions, but it is also used for nonlinear or more complex models as well. For those cases, we need to replace h by a measure of the complexity of the model. Although the number of free parameters in linear estimators can be used, it is not clear what to use for other type of estimators such as k-nearest neighbor, subset selection, neural networks etc.
- How will σ^2 be estimated? Currently, there is no consensus in the literature, but two different approaches are proposed [4]. In the first approach, σ^2 is estimated by a high-variance/low-bias estimator only once and the same noise estimate is used for all different models. In the second approach, σ^2 is estimated for each model separately, and therefore different noise levels are used for different models. In both cases, we estimate σ^2 by

$$\hat{\sigma}^2 = \frac{N}{N-h}\frac{1}{N}\sum_{t=1}^{N}(r^t - \hat{f}(\mathbf{x^t}))^2 \tag{2.9}$$

  If $\hat{\sigma}^2$ in Equation 2.9 is plugged into the Equation 2.8, we get the final prediction error (FPE) [5]:

$$E_g = E_t\frac{1+h/N}{1-h/N} \tag{2.10}$$

Akaike Information Criterion is also applicable when a log-likelihood-based loss function is used. Asympotically, when $N \to \infty$,

$$AIC = -\mathcal{L} + h \tag{2.11}$$

and we choose the model with the smallest AIC over the set of models we have for the solution of the problem.

2.3. Bayesian Technique and Bayesian Information Criterion

Bayesian technique uses an additional prior information for choosing the model of approximating functions. This is a subjective type of model selection because if the unknown function is not in our list, its prior probability will be zero and we can not attain that function. On the other hand, Bayesian type of prior information coding is very powerful when used by experts of the domain. Prior information is introduced in the model with the given formula:

$$P(model|data) = \frac{P(data|model)P(model)}{P(data)} \tag{2.12}$$

where $P(model)$ is the prior probability, $P(data)$ is the probability of observing the training data, $P(data|model)$ is the probability that the data are generated by a model also known as likelihood, and $P(model|data)$ is the posterior probability of a model given the data.

BIC [6] like AIC, is derived using asymptotic analysis and the generalization error is calculated as:

$$E_g = E_t + \log N \frac{h}{N} \sigma^2 \tag{2.13}$$

As in AIC, if we estimate the noise variance σ^2 using Equation 2.9 and we get

$$E_g = E_t \frac{N + h(\log N - 1)}{N - h} \tag{2.14}$$

For log-likelihood-based loss functions we can get the BIC estimate as follows [7]: Suppose that we have a set of candidate models M_i with their model parameters θ_i. Assuming that we have a prior distribution $P(\theta_i|M_i)$ for the parameters of each model, the posterior probability of the model M_i given the training dataset X is:

$$\begin{aligned} P(M_i|X) &= \frac{P(M_i)\int P(X|\theta_i, M_i)P(\theta_i|M_i)d\theta_i}{P(X)} \\ &= \frac{P(M_i)P(X|M_i)}{P(X)} \end{aligned} \tag{2.15}$$

To compare two models M_i and M_j, we compare their posterior probabilities as:

$$s = \frac{P(M_i|X)}{P(M_j|X)} = \frac{P(M_i)P(X|M_i)}{P(M_j)P(X|M_j)} \tag{2.16}$$

If the ratio s is larger than 1, we choose model i as the best model. If s is smaller than 1, we choose model j as the best model. Typically, we assume that the prior distributions of the models are uniform and s reduces to

$$s = \frac{P(X|M_i)}{P(X|M_j)} \tag{2.17}$$

which is called the *Bayes Factor*. $P(X|M_i)$ could be approximated by a Laplace transform and it reduces to

$$\log P(X|M_i) = \log(X|\theta_i, M_i) - \frac{h}{2}\log N \tag{2.18}$$

If the log-likelihood is used, the BIC estimate is calculated as

$$BIC = -\mathcal{L} + \frac{h}{2}\log N \tag{2.19}$$

and we choose the model with the smallest BIC value.

In the Bayesian approach, usually for choosing the best model, maximum a posteriori (MAP) estimator or Bayesian model averaging is used [8]. In MAP method, we choose the model which has the maximum posterior probability among the other possible models. However, when training data is scarce, there might be a large number of models which have non-negligible posterior. Friedman and Koller [9] propose a way for efficiently computing the posterior probability of a feature over all models that contain it. They first calculate the posterior probability of a feature on a fixed ordering on the network variables over all possible network models that are consistent with that ordering. Then by Markov Chain Monte Carlo (MCMC) methods, they approximate the Bayesian posterior probability of a feature over all possible orderings.

Clarke [10] compares Bayesian model averaging with stacking, when the best model is not in the list of all models considered. In stacking, the posterior probabilities of the models are calculated by cross-validation. Both type of model averaging techniques are evaluated in different settings and it is concluded that stacking is more robust compared to Bayesian model averaging.

Chickering [11] uses Bayesian model selection to find the best Bayesian network structure. Bayesian network learning algorithms usually make search on the individual Bayesian graphs, whereas Chickering [11] makes a search on Bayesian equivalence graphs. Two Bayesian network structures are said equivalent if the set of distributions represented with the first one is equal to the set of distributions represented with the second one. Each Bayesian equivalence graph is represented as a state and one can go from one state to another (from one equivalence graph to another) by one of the six operators (add or remove undirected adges, add, remove or reverse a directed edge and insert a 'v' structure in the network).

Andrieu *et al.* [12] propose a hierarchical full Bayesian model for radial basis function (RBF) networks. All hyperparameters of the RBF (number of hidden nodes, regularization parameters, basis function parameters and noise parameters) are taken

as unknown random variables. Bayesian model computation is done by reversible-jump MCMC.

Vila *et al.* [13] propose a general Bayesian conjugate prior approach to determine the structure of a neural network. The use of such a conjugate prior allows the analytic calculation of the parameter posterior and the predictive posterior densities in an empirical-Bayes-like approach.

Bayesian type model selection is also applied to support vector machines (SVM) and for other Gaussian procedures [14]. The algorithm needs no user interaction and calculates the evidence in MAP using a variational approach.

2.4. Minimum Description Length

Another example of complexity penalization is the minimum description length (MDL) principle [15] where we want to describe the data at hand by using as minimum memory as possible. In the MDL approach, there are two sides, namely the sender side and the receiver side. The sender side encodes and sends the data to the receiver side. In doing this, the shortest coding for transmission is aimed.

Let say we want to transmit the data $x^1, x^2, \ldots, x^n$ where each one has a probability of P_i to occur in the message. The theorem due to Shannon says that we should use the code lengths $l_i = -\log_2(P_i)$. The average message length satisfies the inequality

$$E[\text{message length}] \geq \sum_i P_i \log_2 P_i \tag{2.20}$$

From this theorem, we can deduce that to transmit a random variable z having the probability density function $P(z)$, we need $-\log_2 P(z)$ bits of information. We can use this result in the model selection problem: Let us say that for a supervised learning problem, both the sender and receiver sides know the input data and the sender side wants to transmit the outputs to the receiver side. In order to transmit the data using the shortest code possible, the sender side will first find (learn) some model $\hat{f}(\mathbf{x^t})$ to

describe the largest portion of the data and will transmit only the parameters of this model $\hat{f}(\mathbf{x^t})$. The other part of the data, namely the exceptions for the model, will be transmitted normally requiring more bits of information per data point. If the conditional probability of the outputs is $P(\mathbf{r}|\theta, M, \mathbf{X})$, then the total message length of the data will be

$$\text{total message length} = -\log_2 P(\mathbf{r}|\theta, M, \mathbf{X}) - \log_2 P(\theta|M) \tag{2.21}$$

where θ represents the parameters of the model M. The first term to the right is the code length of the difference between the model and the actual values and the second term is the code length of the model parameters.

Equation 2.21 is similar to Equation 2.6. The memory required to encode the model parameters can be thought of as the model complexity and the memory required to encode the exceptions to the model can be thought as the error of the model. Therefore again, if we use a simple model, the description length of the model (the second term) is small but we must encode exceptional instances with a large description (the first term). If the model is complex, the exceptions will take less memory but the description length of the model will require large memory. So, with an appropriate constant λ, the generalization error for MDL principle is calculated as

$$E_g = E_t + \lambda h \tag{2.22}$$

where h denotes the number of parameters of the model M. This equation can be derived from the general equation

$$E_g = E_t + \lambda \sum_{i=0}^{d} w_i^q \tag{2.23}$$

where w_i's are the weights of the input features. The value $q = 0$ corresponds to the Equation 2.22 and $q = 2$ corresponds to the ridge regression.

MDL principle is used for model selection in SVM in a recent work by Luxburg *et al.* [16]. Five different encoding schemes are proposed to describe SVMs. The main idea is to relate the encoding to geometrical concepts such as the width of the margin, the shape of the data in the feature space and the number of support vectors. The encoding schemes which use only a part of the information yield worse results than the encoding schemes that combine several parts of the information. In their work, they also compare the best two encoding schemes with the well-known bound-based model selection methods and get promising results.

Schuurmans and Southey [17](previous work [18]) use unlabeled data to adaptively control hypothesis complexity in supervised learning tasks. The idea is to find a metric whose hyperparameters can be estimated by using unlabeled data. One drawback of this method is that sufficient unlabeled data must be available. Hsu *et al.* [19] use metric-based model selection for multi-strategy learning in a time series application. The method consists of two steps. In the first step, the problem is decomposed into subproblems by means of feature subset selection and clustering. In the second step, the selection of learning components is done via metric-based model selection to produce a mixture model. Bengio and Chapados [20] apply metric-based model selection to feature selection. They combine metric-based model selection with cross-validation to produce a meta model selection technique. Since cross-validation is unbiased with higher variance and metric-based model selection has lower variance but significant bias, they expect that the combination of the two will have better performance. According to the experiments, the combined technique is rarely beaten by any of the two methods.

Bach and Jordan [21] present a class of algorithms for learning the structure of the graphical models from data. Using a measure called kernel generalized variance (KGV), they are able to learn hybrid graphs those include both discrete and continuous variables. They also use the kernel trick to effectively calculate the relevant statistics in linear time.

Chapelle *et al.* [22] offer two new penalization methods for regression called small-

est empirical bound (SEB) and direct eigenvalue estimator (DEE) which are appropriate even with small sample sizes. They compare their methods with six penalty-based and two state-of-the-art techniques one of which is cross-validation.

Recently, a new model selection criterion called subspace information criterion (SIC) was proposed by Sugiyama and Ogawa [23]. SIC is a generalization of Mallow's C_L [24] and assumes the availability of an unbiased estimate of the target function and the noise covariance matrix. In their second paper [25], they evaluate the criterion theoretically and experimentally and show that SIC performs well even with small sample sizes. They also compare the method with the existing model selection techniques including Cross-validation, Mallow's C_P, AIC, BIC, MDL and Vapnik's Measure (VM).

2.5. Early Stopping Rules

Another model selection method is early stopping, used when the learning is iterative. In this method, we stop training of the model before it reaches minimum error on the training set and starts overfitting. We stop training by penalizing the number of steps it takes to come to that point. We can also use the error reduction percentage in training and stop training, for example, if the error drop percentage is below a level. This type of model selection is difficult to control according to Friedman [26].

2.6. Structural Risk Minimization

As we have mentioned in Section 2.2, one problem in estimating the generalization error is to specify the number of free parameters h when the estimator is not linear. In the statistical learning theory, Vapnik-Chervonenkis (VC) dimension is a measure of complexity defined for any type of estimator.

VC dimension for a class of functions $f(x, \alpha)$ where α denotes the parameter vector is defined to be the largest number of points that can be shattered by members of $f(x, \alpha)$. A set of data points is *shattered* by a class of functions $f(x, \alpha)$ if for each

possible class labeling of the points, one can find a member of $f(x, \alpha)$ which perfectly separates them. For example, in two dimensions, we can separate three points with a line, but we can not separate four points (if their class labelings are done like in the famous XOR problem). Therefore, the VC dimension of the linear estimator class in two dimensions is 3. In general, the VC dimension of the linear estimator class in d dimensions is $d + 1$ which is also the number of free parameters.

Structural risk minimization (SRM) [27] uses the VC dimension of the estimators to select the best model by choosing the model with the smallest upper bound for the generalization error. In SRM, the possible models are ordered according to their complexity

$$M_0 \subset M_1 \subset \ M_2 \subset \ldots \tag{2.24}$$

For example, if the problem is selecting the best degree of a polynomial function, M_0 will be the polynomial with degree 0, M_1 will be the polynomial with degree 1, etc. For each model, the upper bound for its generalization error is calculated.

- For binary classification, the upper bound for the generalization error is [28]

$$E_g = E_t + \frac{\epsilon}{2}\left(1 + \sqrt{1 + \frac{4E_t}{\epsilon}}\right) \tag{2.25}$$

- For regression, the upper bound for the generalization error is [29]

$$E_g = \frac{E_t}{1 - c\sqrt{\epsilon}} \tag{2.26}$$

and ϵ is given by the formula

$$\epsilon = a_1 \frac{h[\log(a_2 N/h) + 1] - \log(\nu)}{N} \tag{2.27}$$

where h represents the VC dimension of the model and ν represents the confidence

level. It is recommended to use $\nu = \frac{1}{\sqrt{N}}$ for large sample sizes. For regression it is recommended to use $a_1 = 1$ and $a_2 = 1$, and for classification $a_1 = 4$ and $a_2 = 2$ corresponds to the worst-case scenarios.

Most recent research on classification nowadays focuses on finding upper bounds on the probability of misclassification of a classifier calculated by the same data that is used in the training of the classifier [30, 31].

Obtaining the VC-dimension of a classifier is necessary for complexity control in SRM. Unfortunately, it is not possible to obtain an accurate estimate of the VC-dimension in most cases. To avoid this problem, a set of experiments on artificial sets are done and based on the frequency of the errors on these sets, a best fit for theoretical formula is calculated. Shao *et al.* [32] used optimized experimental design to improve the VC-estimates. The algorithm starts with the uniform experiment design defined in Vapnik *et al.* [33] and by making pairwise exchanges between design points, the optimized design is obtained. The mean square error is used as a criterion to identify good and bad design points.

2.7. Comparison of Model Selection Techniques

Kearns *et al.* [34] compare three model selection methods, SRM, MDL and Cross-validation theoretically and experimentally. Their results coincide with the no free lunch (NFL) theorems. That is, none of the three methods dominates the others for all data sizes. The two penalty-based methods SRM and MDL have bias either towards or against the coding of the model. The bias can not be overcome because of the increased generalization error. Although cross-validation is not the best of the methods, it manages to closely track the best penalty-based method.

Cherkassky and Ma [4] present empirical comparisons between the penalization methods AIC, BIC and SRM for regression problems. They compare these model selection methods on various datasets and using three types of estimators, namely, linear estimator, subset selection, and k-nearest neighbor regression. According to the

results of the experiments, SRM and BIC methods show similar predictive performances and they consistently outperform AIC. They also point out the methodological issues important for meaningful comparisons and the correct application of SRM.

Another survey on model selection techniques is given by Bartlett *et al.* [35]. The survey studies model selection strategies based on penalized empirical loss minimization. They have shown that the generalization error obtained by max discrepancy penalization and Rademacher penalization methods are favorable compared to cross-validation error estimates. They also exhibit less variance.

3. CROSS-VALIDATION

Cross-validation (CV) techniques do not make any assumptions about model complexity or the prior model probabilities. The idea is to estimate the model using part of the training data and then validate that model with the remaining part of the training data. The first part of data is called *learning set* and second part of the data is called *validation set*.

When the dataset is small, to reduce the variance, this training and validation is done K times and averaged. In order to get K train and validation set pairs we can divide the data into K equal parts, K is typically 10 or 30. Each part then can be divided into two, where the first part is used for training and the second part is used for validation. The assumption of this approach is that the validation set and the training set come from the same unknown distribution. This method works well for large data sets, but in the case of small data sets, dividing a small dataset into ten or thirty parts will not give us enough data for neither training nor validation. With small datasets, we need resampling methods to be able to use a single data instance many times. For example, in K-fold cross-validation, we divide the data into K equal parts. To generate a train set and validation set pair, we keep one of the K parts out as the validation set and combine the remaining $K-1$ parts to produce the training set. If we do this for each of the K different parts, we get K training and validation set pairs. If we leave each time one instance for the validation set, we get the *leave-one-out* technique. Since there is a need of training K times, this technique can usually be applied only on small datasets. There is also evidence that K-fold cross-validation can give better results than leave-one-out [36].

One of the cross-validation techniques is 5×2 cross-validation [37]. In this type of cross-validation, we divide the data randomly into two equal parts. We get two sets $\mathcal{X}^1$ and $\mathcal{X}^2$. $\mathcal{X}^1$ is the training set and $\mathcal{X}^2$ is the validation set. Then we swap the roles of the parts, so $\mathcal{X}^2$ is the training set and $\mathcal{X}^1$ is the validation set. These two divisions give us the first fold. We repeat the same procedure five times, where at each time we

randomly divide data into two parts again to get first half of the fold and swap the roles of the two parts to get the second half of the fold. At the end of this procedure, we have 10 train and validation set pairs for which we will train and test our model.

Another cross-validation technique is *bootstrap* [38]. The basic idea is to randomly draw datasets with replacement from the training set where each sample contains the same number of instances as the training set. This is done K times producing K bootstrap datasets. After getting the bootstrap datasets, we can fit our model to each bootstrap sample and validate on the (full) training set to estimate the generalization error of our model. One problem with this type of validation is the overlapping nature of the bootstrap datasets. We use the bootstrap samples as training sets while using the original training set as the validation sample. These samples may have many instances in common which make predictions (generalization error) look unrealistically good. To solve this problem, for each instance in the training set, we only use the models generated from the bootstrap samples which do not include that instance [39]. This type of bootstrap technique is called *leave-one-out bootstrap*.

There are many variants to this basic strategy having to do with repeating training and validation splitting many times and averaging the results to choose the final hypothesis class [40].

Rivals and Personnaz [41] compare leave-one-out cross-validation based model selection with statistical tests-based model selection for the selection of linear models. They conclude that although leave-one-out cross-validation is not the subject of the no-free-lunch criticisms, for linear models, even for large data sizes, it does not perform well compared to statistical tests.

Although cross-validation is an unbiased technique for model selection, large scale cross-validation, where there are many candidate models, could be computationally expensive. Corresponding to this problem, Moore and Lee [42] propose efficient algorithms for calculating cross-validation errors. They apply leave-one-out cross-validation type model selection on instance based learning. They introduced two algorithms: One

is RACE where they start to calculate cross-validation errors on all models, but on the way they eliminate the unpromising models, which they are confident, are worse than some other model. The second is BLOCKING where the purpose is to eliminate the models which produce nearly identical predictions.

3.1. Statistical Tests

The MultiTest algorithm, which we propose in the next section uses a one-sided test to compare the expected errors of the learners, and towards this aim, we first propose a new test, which is a one-sided variant of the 5×2 cv t test proposed by Dietterich [37]. Let us use r_{ij}^t to denote the desired output (on instance i of fold j) and $f(x_{ij}^t)$ to denote the predicted output by the model. In classification, r_{ij}^t, $f(x_{ij}^t) \in \{0, 1\}$ and in regression r_{ij}^t, $f(x_{ij}^t) \in \Re$.

- In classification, X_{ij}^t is the Bernoulli random variable representing the outcome of classifier $i = 1, \ldots, K$ on instance $t = 1, \ldots, N$ of validation fold $j = 1, \ldots, L$. $X_{ij}^t = 1$ if the outcome is error and $X_{ij}^t = 0$ if the outcome is correct: $X_{ij}^t = 1$ if $r_{ij}^t = f(x_{ij}^t)$ and 0 otherwise.
- In regression, X_{ij}^t is the square of the residual error of the regressor $i = 1, \ldots, K$ on instance $t = 1, \ldots, N$ of validation fold $j = 1, \ldots, L$: $X_{ij}^t = (r_{ij}^t - f(x_{ij}^t))^2$.

The average error of learner i on fold j is

$$Y_{ij} = \frac{\sum_{t=1}^{N} X_{ij}^t}{N} \tag{3.1}$$

This corresponds to the *misclassification error rate* in classification and *mean square error* for regression. Y_{ij} are the sum of independent and identically distributed random variables (X_{ij}^t) and by the central limit theorem are approximately normal distributed with mean μ_i. For each learning algorithm i, the expected error is the sample average

m_i (estimator to μ_i), which is calculated as

$$m_i = \frac{\sum_{j=1}^{L} Y_{ij}}{L} \tag{3.2}$$

Let us first review the 5×2 cv t test which is a two-sided test with null hypothesis $H_0 : \mu_1 = \mu_2$ [37]. This test uses five replications of two–fold cross–validation. $p_i^{(j)}$ is the difference between the error values of the two learners on replication i of j fold; so $p_i^{(1)} = Y_{1(2i-1)} - Y_{2(2i-1)}$ and $p_i^{(2)} = Y_{1(2i)} - Y_{2(2i)}$. s_i^2 is the estimated variance of the replication i: $s_i^2 = (p_i^{(1)} - \overline{p_i})^2 + (p_i^{(2)} - \overline{p_i})^2$ where $\overline{p_i}$ is the average of the differences on replication i: $\overline{p_i} = (p_i^{(1)} + p_i^{(2)})/2$. Under the null hypothesis that the two means are equal, $p_i^{(j)}/\sigma$ is unit normal ($\mathcal{Z}$), and assuming that $p_i^{(1)}$ and $p_i^{(2)}$ are independent normals, s_i^2/σ^2 is chi-square distributed with one degree of freedom ($\mathcal{X}_1^2$). Assuming also that the s_i^2 are independent

$$M = \frac{\sum_{i=1}^{5} s_i^2}{\sigma^2} \tag{3.3}$$

is chi-square distributed with 5 degrees of freedom. We know that, if $Z \sim \mathcal{Z}$ and $X \sim \mathcal{X}_n^2$ and if Z and X are independent, then $Z/\sqrt{X/n}$ is t distributed with n degrees of freedom. Then

$$t' = \frac{p_1^{(1)}/\sigma}{\sqrt{M/5}} = \frac{p_1^{(1)}}{\sqrt{\sum_{i=1}^{5} s_i^2/5}} \tag{3.4}$$

is t-distributed with 5 degrees of freedom. Dietterich [37] proposed the two-sided test where the null hypothesis $H_0 : \mu_1 = \mu_2$ is accepted with α level of confidence if $t' \in (-t_{\alpha/2,5}, t_{\alpha/2,5})$. Similarly, we can derive a one-sided test and accept the null hypothesis $H_0 : \mu_1 \leq \mu_2$ if $t' \in (-\infty, t_{\alpha,5})$.

Though we can use this latter as the one-sided test in MultiTest, we choose not to. This is because the 5×2 t test uses $p_1^{(1)}$ in the numerator. This is arbitrary; actually there are ten possible $p_i^{(j)}$ that can be placed there and we notice that depending on which one we use, the test sometimes accepts and sometimes rejects. This is disturbing

because this depends on the order of folds and replications which should not effect the test. Alpaydın has previously used this to improve the two-sided 5×2 cv t test and proposes the 5×2 cv F test which uses all ten $p_i^{(j)}$ [43]. The 5×2 cv F test uses square of the differences of error values and cannot be used for a one-sided test. Similarly, if $p_i^{(j)}/\sigma \sim \mathcal{Z}$, then

$$\overline{p} = \frac{\sum_{i=1}^{5}\sum_{j=1}^{2} p_i^{(j)}}{10} \tag{3.5}$$

is $\mathcal{N}(0, 1/10)$ and therefore $\sqrt{10}\overline{p}/\sigma \sim \mathcal{Z}$. If we place $\overline{p}$ instead of $p_1^{(1)}$ in Equation (3.4), we get

$$t' = \frac{\sqrt{10}\overline{p}}{\sqrt{M/5}} = \frac{\sqrt{10}\overline{p}}{\sqrt{\sum_{i=1}^{5} s_i^2/5}} \tag{3.6}$$

which is also t distributed with 5 degrees of freedom. We expect this test to be more robust because it uses $\overline{p}$ (the average of ten $p_i^{(j)}$ values) instead of one $p_1^{(1)}$. The one-sided null hypothesis $H_0 : \mu_1 \leq \mu_2$ is accepted with α level of confidence if this value is smaller than $t_{\alpha,5}$. We call this the *combined* 5×2 cv t test. As an aside, though we do not use it here, the two-sided null hypothesis $H_0 : \mu_1 = \mu_2$ is accepted if $t' \in (-t_{\alpha/2,5}, t_{\alpha/2,5})$.

3.2. The MultiTest Algorithm

In many applications, we can use one of several supervised learning algorithms to induce the final learner and we would like to choose the one with the least expected error. To check for a statistically significant difference in expected error, the algorithms are trained and tested on several folds of training, validation set pairs and a sample of error values on the validation folds are obtained. Statistical tests are used to compare the means of populations which these validation error values are sampled from, i.e., their expected error.

Tests in the literature, e.g., k-fold paired t test, 5×2 cv t test [37], 5×2 cv F

test [43] are two-sided and test whether the means of two populations are equal [44]. That is, these tests are pairwise and in our case of expected error comparison, check whether two learning algorithms yield the same expected error, i.e., the null hypothesis is that the two samples of validation errors have the same mean, versus the alternative hypothesis that the means are significantly different:

$$H_0 : \mu_1 = \mu_2 \text{ vs. } H_1 : \mu_1 \neq \mu_2 \tag{3.7}$$

If the test accepts, we conclude that they have the same expected error, i.e., there is no statistically significant difference between them. But if the test rejects and we realize that they are statistically significantly different, we do not know which one is smaller; that requires a one-sided test, as we see below.

Typically, we have more than two candidate learning algorithms that we can use in an application. Analysis of variance (Anova) is a well-known and widely used method to test whether K samples are drawn from populations with the same mean, and can be used to test whether $K > 2$ learning algorithms induce learners with the same expected error. The null hypothesis is

$$H_0 : \mu_1 = \mu_2 = \cdots = \mu_K \tag{3.8}$$

If Anova accepts then all learning algorithms induce learners with the same expected error and we can choose any of them. If Anova rejects, we know that there is an inequality somewhere but we do not know where and we cannot use this to pinpoint the algorithm with the smallest expected error.

Another possibility is to use a multiple range test which has the same null hypothesis as Anova (Equation 3.8). A multiple range test checks for the equality of the means of subsets of populations and one such test is the Newman-Keuls test. There are also multiple range tests due to Duncan and Tukey but the Newman-Keuls test is favored over them ([45]; p. 87). In our case of comparing expected error, the multiple range test is used to find subsets of algorithms with the same expected error. For

example, given algorithms 1, 2, 3, 4, 5, a range test can conclude as

$$\underline{5\ 2}\ \underline{\underline{4}\ 3}\ 1$$

The algorithms are sorted in ascending average error. An underline implies that there is no statistically significant difference between the means of the underlined populations. In the example above, the test concludes that there is no statistically significant difference between the expected error of 5, 2, and 4, and also that there is no difference between 4 and 3. We see that for example there is difference between 2 and 3, and also between 3 and 1. Note that a range test checks for equality and a rejection, that is, the absence of an underline, does not imply an ordering. For example, we know that 3 and 1 have significantly different expected error and that the average of errors of 3 over the validation folds is less than the average of errors of 1 but this does not imply that 3 has *significantly* less error than 1; it might, or it might not; a range test does not check for this.

Comparing statistics from multiple populations is called *multiple comparison procedures* [46, 47] and additional to the parametric tests above, there are also nonparametric tests [48]; for example, Kruskal-Wallis test is the nonparametric version of Anova.

To the best of our knowledge, there exists no method in the statistical literature, which given $K > 2$ samples, finds the population with the statistically significantly smallest mean, or in our case, finds the learning algorithm with the smallest error. Finding the smallest mean is a particular case of the more general problem of ordering $K > 2$ populations in terms of increasing mean (expected error), which can be solved by iteratively applying the same method and removing the smallest at each iteration.

3.2.1. Ordering Two Learning Algorithms

First, we see that to be able to get an ordering, we cannot use a two-sided test checking for equality; we need a one-sided test to check whether a learning algorithm has less expected error or not because we want to find the one with the smallest expected error. In a one-sided test, we test if the mean of the first population is less than or equal to the mean of the second population. The null hypothesis is

$$H_0 : \mu_1 \leq \mu_2 \text{ vs. } H_1 : \mu_1 > \mu_2 \tag{3.9}$$

In our case of comparing expected errors, if the null hypothesis is accepted, the expected error of 1 is less than or equal to that of the expected error of 2. If the test rejects, this means that 2 has statistically significantly less expected error than 1. That is, keep in mind here that we cannot say which one is better if the test accepts, but only when it rejects.

At this point, we also need to remember that though expected error is the most important criterion in favoring one algorithm over another, it is by no means the only one. Various measures of cost, e.g., space/time complexity of training and test, or costs of features (if learners use different input representations), interpretability, or easy programmability of the final learner affect our choosing an algorithm over another one; various types of cost are discussed in detail in [49]. So for example, if we have two or more algorithms with the same expected error, then it is logical that we choose the simplest one based on the cost measure we consider important in the particular application we are working on.

In this work, we are going to use the fact that given a number of learning algorithms for a particular application, we can always order them in terms of such a preference, e.g., based on their complexities. That is given any two algorithms, if they have the same expected error, we favor one over another based on this preference. In our method, this information of our *prior* preferences (before looking at the data) is combined with what the data tells us through the statistical tests, to give us a final or-

dering of the algorithms. But the expected error remains the most important criterion and it overrides our prior preference.

In using the one-sided test, we use this prior ordering based on preference in choosing how to apply the test. We only test for

$$H_0 : \mu_1 \leq \mu_2 \tag{3.10}$$

where we have a prior preference of 1 over 2, e.g., because it is simpler. If the test accepts, we choose 1 over 2, either because: (i) $\mu_1 < \mu_2$, i.e., 1 has less expected error than 2; in such a case, 1 is chosen over 2 both because it has less expected error and it is simpler, or (ii) $\mu_1 = \mu_2$, i.e., they have the same expected error; we can choose either one and we choose the preferred one. Only when the test rejects, we know that $\mu_1 > \mu_2$, and in this case, we choose 2 over 1, the test overriding our prior preference.

Therefore, we choose an algorithm A over another one B, and say that A is "better than" B by taking into account both our prior preferences of A and B and the statistical test comparing their expected error. A is better than B either because (i) A has less expected error than B, or (ii) they have the same expected error and A is preferred to B, e.g., because it is simpler.

In the next subsection, we discuss how to combine the results of such pairwise orders to find the best of $K > 2$ algorithms, or order them in terms of their "goodness" in the general case.

3.2.2. Combining Pairwise Orders

Before applying the tests, we assume that we are given a full, linear ordering of learning algorithms in terms of our prior preferences. Depending on the particular requirements and constraints of the particular application at hand, such a linear ordering can always be defined. We denote learning algorithms by their indices in this ordering as $1, 2, \ldots, K$, such that 1 is the most preferred, 2 is preferred over $3, 4, \ldots, K$, and K

is the least preferred. We then apply the one-sided test of Equation 3.10 as

$$H_0 : \mu_i \leq \mu_j \tag{3.11}$$

where $i < j$ with $i, j = 1, \ldots, K$. There are $K(K-1)/2$ tests to be applied. Note that the real cost is the training and validating the K algorithms L times, where L is the number of folds, e.g., ten if we are using 10-fold cross-validation. Once the K algorithms are trained and validated L times each and these $K \cdot L$ validation errors are recorded, applying the tests are simple in comparison. And these $K \cdot L$ validation errors need to be calculated for us to be able to do *any* statistical comparison, e.g., Anova, range test, etc., and is not a particular requirement of our proposed method.

The confidence level of the one-sided statistical test of Equation 3.11 is set to $\frac{\alpha}{K(K-1)/2}$. This is because we are making $K(K-1)/2$ multiple tests to get the resulting final ordering and to have a confidence level of α, we need this Bonferroni correction for each test [46]. When K is large, Bonferroni correction may be too conservative and we can use Holm correction [50] instead to have higher confidence levels for the one-sided statistical tests.

We use graph theory to represent the result of the statistical tests and manipulate the graph to find the best. The algorithm is given in Figure 3.1. The graph has K vertices corresponding to the K algorithms. For all $i < j$ where $i, j = 1, \ldots, K$, we test $H_0 : \mu_i \leq \mu_j$, and if the test *rejects*, we place a directed edge from i to j to indicate that we are overriding the prior order. This corresponds to a binary relation R defined on the set of learning algorithms where jRi implies that $j > i$ and j has significantly less expected error than i ($\mu_i \leq \mu_j$ is rejected). If we have $jR\!\!\!/i$, this means that the hypothesis test is accepted and our choice of i over j stands. The resulting directed graph has thus directed edges where the test is rejected for its incident vertices. The number of incoming edges to a node j is the number of algorithms that are preferred over j but have higher expected error. The number of outgoing edges from a node i is the number of algorithms that are less preferred than i but have less expected error. The resulting graph need not be connected. For example when all algorithms have the

same expected error, there are no edges.

Once the directed graph is formed by the use of the statistical tests, we choose the "best" node. For this:

(i) We find the nodes with no outgoing edges to produce the set L_k. If there is no outgoing edge from a node, this means that there is no other learning algorithm that has less expected error.

(ii) From the elements of L_k, we select the node with the lowest index and report it. This selected algorithm is the one that is the most preferred among all with the least expected error.

This calculates the "best" node; if we want to find an ordering in terms of "goodness", we iterate steps (i) and (ii) above removing the best node and the edges incident to it at each iteration to get a *topological sort* [51].

```
1  Best MultiTest(1, ..., K; T; α)
2      E ← ∅ /* Edges of the graph */
3      for i = 1 to K
4          for j = i + 1 to K
5              Test H_0: μ_i ≤ μ_j using T
6              if H_0 rejected E ← E ∪ e[i, j] (α/(K(K − 1)/2)) /* Bonferroni */
7      L_k = {x : ∀e[j, k] ∈ E, j ≠ x}; /* Find the nodes with no outgoing edges */
8      l = ∀j ∈ L_k, l ≤ j; /* Select node l with the lowest index */
9      return l;
```

Figure 3.1. Pseudocode of MultiTest: $1, \ldots, K$: Learning algorithms in decreasing order of prior preference, T: The one-sided test used for pairwise comparison, α: Required confidence level. Lines 3 - 6 form the directed graph, lines 7 - 9 find the "best". If we iterate lines 7 - 9, removing l and edges incident to it after each iteration, we get an ordering in terms of "goodness"

Figure 3.2. Sample execution of the MultiTest algorithm on four algorithms 1, 2, 3, 4 in decreasing order of preference (increasing order of complexity). Nodes with thick lines indicate candidates at each step and among them, the one with the lowest index (the most preferred) is taken (shown shaded). The best one is 3 and if we continue iterating the ordering found is $\mathbf{3} < 2 < 4 < 1$

Figure 3.2 shows a sample execution of the MultiTest algorithm on four ($K = 4$) learning algorithms numbered 1 to 4. They are sorted in decreasing order of prior preference 1, 2, 3, and 4. After applying $4(4-1)/2$ tests, let us say that we place edges from 1 to 2, 1 to 3, 1 to 4 and 2 to 3 (shown in Figure 3.2.a). Then, we start generating the full ordering: There are two nodes 3 and 4 having no outgoing edges (shown with a thicker line), this means that there is not any other algorithm having less expected error than these two. In this case, we choose the simpler of them, 3, as the best learning algorithm (shown shaded). Assuming that we do not only want to find the best one but order all algorithms, we continue. We remove node 3 and all edges incident to 3 and have the graph shown in Figure 3.2.b. Here, nodes 2 and 4 have no outgoing edges, so we choose the simpler 2 as the second best learning algorithm. After removing node 2 and the edges arriving to 2, we have the graph shown in Figure 3.2.c. Now 4 has no outgoing edge and we select it as the third and 1 as the last (Figure 3.2.d).

4. LEARNING ALGORITHMS

4.1. Decision Trees

A decision tree is made up of internal decision nodes and terminal leaves. The input vector is composed of d attributes, $\boldsymbol{x} = [x_1, \ldots, x_d]^T$, and the aim in classification is to assign $\boldsymbol{x}$ to one of K mutually exclusive and exhaustive classes, $\mathcal{C}_1, \ldots, \mathcal{C}_K$. Each internal node m implements a decision function, $f_m(\boldsymbol{x})$, where each branch of the node corresponds to one outcome of the decision. Each leaf of the tree carries a class label. We use binary decision nodes even when K (the number of classes) > 2, but the approach we advocate holds also for trees with K-way splitting nodes.

Starting from the root, at each internal node, $f_m(\boldsymbol{x})$ is calculated and depending on the outcome, the corresponding branch is taken and the process is continued recursively until a leaf node is met, at which point the label of the leaf defines the class.

Geometrically, each $f_m(\boldsymbol{x})$ defines a discriminant in the d-dimensional input space dividing it into as many subspaces as there are branches. As one takes a path from the root to a leaf, these subspaces are further subdivided until we end up with a part of the input space which contains the instances of one class only. Different decision tree methods assume different models for the discriminant f_m and the model class defines the shape of the discriminant.

In *univariate* decision trees, the decision at internal node m uses only one attribute, i.e., one dimension of $\boldsymbol{x}$, x_j. If that attribute is numeric, the decision is of the form

$$f_m(\boldsymbol{x}) : x_j + w_{m0} > 0 \tag{4.1}$$

where w_{m0} is some constant number. This test has two outcomes, true and false, labelling the two branches, left and right, and thus the node is binary. This defines a

discriminant which is orthogonal to axis x_j, intersects it at $x_j = -w_{m0}$ and divides the input space into two.

If the attribute is discrete valued with L possible values, $\{a_1, a_2, \ldots, a_L\}$, the decision is of the form

$$f_m(\boldsymbol{x}) : x_j = a_i, i = 1, \ldots, L \tag{4.2}$$

This has L outcomes, one for each possible value, and thus there are L branches and the node is L-ary, dividing the input space into L. In a univariate tree, successive decision nodes on a path from the root to a leaf further divide these into two, or L, with splits orthogonal to each other and the leaf nodes define hyperrectangles in the input space.

When the inputs are correlated, looking at one feature may be too restrictive. A *linear multivariate tree*, at each internal node, uses a linear combination of all attributes.

$$f_m(\boldsymbol{x}) : \boldsymbol{w}_m^T \boldsymbol{x} + w_{m0} = \sum_{j=1}^{d} w_{mj} x_j + w_{m0} > 0 \tag{4.3}$$

To be able to apply the weighted sum, all the attributes should be numeric and discrete values need be represented numerically (usually by 1-of-L encoding) before. The weighted sum returns a number and the node is binary. Note that the univariate numeric node is a special case of the multivariate linear node, where all but one of w_{mj} is 0 and the other, 1. In this linear case, each decision node divides the input space into two with a hyperplane of arbitrary orientation and position where successive decision nodes on a path from the root to a leaf further divide these into two and the leaf nodes define polyhedra in the input space.

In the more general case, one can use a quadratic model as

$$\begin{aligned} f_m(\boldsymbol{x}) &: \boldsymbol{x}^T \mathbf{W}_m \boldsymbol{x} + \boldsymbol{w}_m^T \boldsymbol{x} + w_{m0} \\ &= \sum_i \sum_j W_{mij} x_i x_j + \sum_j w_{mj} x_j + w_{m0} > 0 \end{aligned} \tag{4.4}$$

The linear model is a special case where $W_{ij} = 0, \forall i, j = 1, \ldots, d$. Another possibility to get a nonlinear split at a node is to write the decision as a weighted sum of H nonlinear basis functions.

$$f_m(\boldsymbol{x}) : \sum_{h=0}^{H} w_{mh} g_{mh}(\boldsymbol{x}) + w_{m0} > 0 \tag{4.5}$$

where $g_{mh}(\boldsymbol{x})$ are the nonlinear basis functions. The multilayer perceptron (MLP) is such a model where the basis function is the soft-thresholded weighted sum.

$$g_{mh}(\boldsymbol{x}) = \frac{1}{1 + \exp[-(\boldsymbol{v}_{mh}^T \boldsymbol{x} + v_{mh0})]} \tag{4.6}$$

This model is called *nonlinear multivariate decision tree* and the difference between the univariate, linear multivariate and nonlinear multivariate splits is shown on an example in Figure 4.1.

4.1.1. Tuning Node Complexity

What any classifier, and in this case the decision tree, is doing is approximating the real (unknown) discriminant. With univariate nodes, we are limited to a piecewise approximation using axis-aligned hyperplanes. With multivariate linear nodes, we can use arbitrary hyperplanes and approximate the discriminant better.

The branching factor, i.e, the number of children of an internal node, has a similar effect in that it defines the number of discriminants that a node defines. A binary decision node, with a branching factor of two, defines one discriminant and

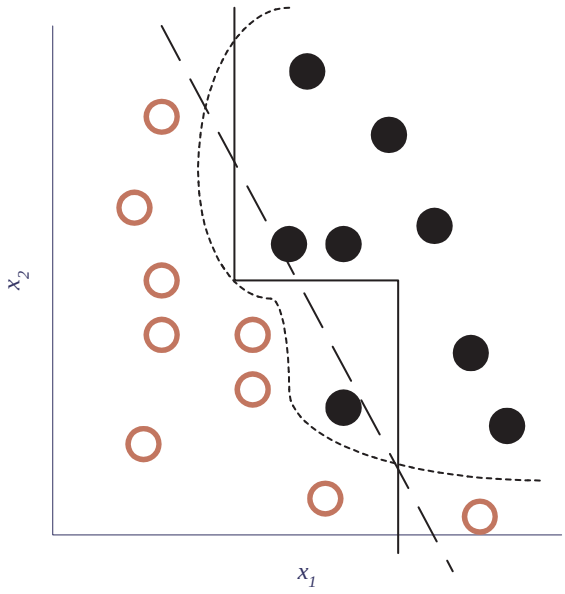


Figure 4.1. Example univariate (continuous line), linear multivariate (dashed line), and nonlinear multivariate (dotted line) splits that separate instances of two classes

separates the data into two. Thus with binary decision nodes, a K-class problem is converted into a sequence of 2-class problems. A K-way node separates the data into K parts at one time.

Thus there is a dependency between the complexity of a node, the branching factor, and the size of the tree. With complex nodes and large number of branches, the tree may be quite small but difficult to interpret. A tree with simple nodes and low branching factor may be large but interpretable (e.g., can be converted to simple IF-THEN rules); such a tree also better matches the underlying divide-and-conquer methodology of a tree.

A complex model with a larger number of parameters requires a larger training dataset and risks overfitting on a small amount of data. Hence one should be careful in tuning the complexity of a node with the properties of the data at hand. For example, using multivariate linear nodes, we are assuming that the input space can be divided using hyperplanes into localized regions (volumes) where classes, or groups of classes

are linearly separable.

4.1.2. Training Decision Trees

Training corresponds to constructing the tree given a training set. Finding the smallest decision tree that classifies a training set correctly is NP-hard [52]. For large training sets and input dimensions, even for the univariate case, one cannot exhaustively search through the complete space of possible decision trees. Decision tree algorithms are thus greedy in that at each step, we decide on one decision node. Assuming a model for f_m (univariate, or linear or nonlinear multivariate), we look for the parameters (w_m coefficients) that best split the data hitting node m, starting with the complete dataset in deciding on the root node. Once we decide on a split, tree construction continues recursively for each child with training instances taking that branch. Surveys about constructing and simplifying decision trees can be found in [53] and [54]. A recent survey comparing different decision tree methods with other classification algorithms is given in [55].

The best split is when all the instances from a class lie on the same side of the decision boundary, i.e., return the same truth value for f_m. There are various measures proposed for measuring the "impurity" of a split; examples are entropy [56] and the Gini index [57]. Murthy, Kasif & Salzberg [52] describe some other impurity indices. Our results and those of previous researchers indicate that there is no significant difference between these impurity measures.

For constructing univariate decision trees with discrete attributes, Quinlan proposed the *ID3 algorithm* [56] and later generalized it for numeric attributes with the *C4.5 algorithm* [58]. In this univariate case, at each decision node, one check for all possible splits for all attributes and choose the best as measured by the purity index. For a discrete attribute, there is only one possible split. For a numeric attribute, there are $N_m - 1$ possible splits, where N_m is the number of training instances reaching node m.

In the case of a linear multivariate tree, even the problem of finding the optimal split at a node when optimality is measured in terms of misclassification errors is NP-hard [52]. The problem of finding the best split is then an optimization problem to find the best coefficients, $w_{mj}, j = 0, \ldots, d$, that minimize impurity as defined by the entropy or Gini index. An iterative local search algorithm is used for optimization which does not guarantee optimality and may get stuck in local optima.

1. Linear Discriminant Analysis was first used in Friedman[59] for constructing decision trees. The algorithm has binary splits at each node, where a split is like in C4.5, i.e. $x_i < w_0$ but x_i can be an original variable, transgenerated, or adaptive. Linear discriminant analysis is applied to construct an adaptive variable. Kolmogorof-Smirnoff distance is used as the error measure. When there are more than $K > 2$ classes, it converts the problem into K different subproblems, where each subproblem separates one class from others.
2. In CART (Classification and Regression Trees)[57], parameter adaptation is through backfitting: At each step, all the coefficients w_{mj} except one is fixed and that coefficient is tuned for possible improvement in terms of impurity. One cycles through all j until there is no further improvement.
3. In FACT (Fast Algorithm for Classification Trees)[60], with K classes a node can have K branches. Each branch has its modified linear discriminant function calculated using Linear Discriminant Analysis (LDA) and an instance is channeled to the ith branch to minimize an estimated expected risk.
4. In Neural Trees[61], each decision node uses a multilayer perceptron which implements multivariate *nonlinear* decision trees. The nodes are binary and K classes are grouped into two using the supervised exchange heuristic. Backpropagation is used to learn parameters. To be able to compare with other multivariate linear methods, in our simulations, we replace the multilayer perceptron with a single layer, linear perceptron, and name such a tree ID-LP.
5. In OC1 (Oblique Classifier)[52], an extension to CART is made to get out of the local optima. A small random vector is added to $\boldsymbol{w}_m$ once there is convergence through backfitting. Adding a vector perturbs all coefficients together and makes a conjugate jump in the coefficient space. Another extension proposed is to

run the method several (20-50) times and choose the best solution in terms of impurity.

6. In LMDT (Linear Machine Decision Trees)[62], with K classes, as in FACT, a node is allowed to have K branches. For each class, i.e., branch, there is a vector of parameters, and the node implements a K-way split. There is an iterative algorithm that adjusts the parameters of classes to minimize the number of misclassifications, rather than an impurity measure as entropy or Gini.
7. QUEST (Quick Unbiased Efficient Statistical Tree)[63] is a revised version of FACT and uses binary splits at each decision node. It solves the problem of dividing K classes into two classes by using unsupervised 2-means clustering on the class means of the data. QUEST also differs from FACT in the way that it does not assume equal variances and uses Quadratic Discriminant Analysis (QDA) to find the two roots for the split point and uses the appropriate one.
8. LTREE (Linear Tree)[64] is a multivariate decision tree algorithm with binary splits. LTREE uses linear discriminant analysis to construct new features, which are linear combinations of the original features. For all constructed features, the best split is found using C4.5's exhaustive search technique. Best of these is selected to create the two children of the current node. These new constructed features can also be used down the tree in the children of that node. Extensions of this algorithm uses quadratic discriminant QTREE and logistic discriminant LGTREE for constructing new features.
9. CRUISE[65] (Classification Rule With Unbiased Interaction Selection and Estimation) is a multivariate algorithm with K-way nodes. Like FACT, CRUISE finds $K-1$ splits using linear discriminant analysis. The departure from FACT occurs when the split assigns the same class to all its K children. Because such a split is not useful, the best next class is chosen. Another departure occurs while assigning a class to a leaf: When there are two or more classes which have the same number of instances in that leaf, FACT selects randomly one of them but CRUISE selects the class which has not been assigned to any leaf node.

Because the univariate is a special case of the multivariate, most of these multivariate algorithms have their univariate versions, sometimes with slight modifications.

Table 4.1. Multivariate decision tree construction algorithms classified according to six dimensions, which are node type (*uni*variate, *lin*ear multivariate, *non*linear multivariate), branching factor, grouping algorithm for grouping $K > 2$ classes into two, error measure minimized, minimization method to find the direction vector $\boldsymbol{w}$, and minimization method to find the split point w_0

Algorithm	Node	Br	Group	Error	Search $\boldsymbol{w}$	Search w_0
C4.5	Uni	2	-	Impurity	-	Exhaustive
Friedman's	Uni/Lin	2	-	Kolm-Smir	Analytical	Analytical
CART	Lin	2	-	Impurity	Backfitting	Exhaustive
FACT	Uni/Lin	K	-	Fisher's	Analytical	Analytical
ID-LP/MLP	Lin/Non	2	Sup	MSE	Gradient	Gradient
OC1	Lin	2	-	Info Gain	Hill climb	Exhaustive
LMDT	Lin	K	-	Misclass	Thermal	Thermal
QUEST	Uni/Lin	2	Unsup	Fisher's	Analytical	Analytical
Ltree	Lin	2	-	Info Gain	Analytical	Exhaustive
Cruise	Uni/Lin	K	-	Fisher's	Analytical	Analytical
LDT	Uni/Lin	2	Sup	Fisher's	Analytical	Analytical

In Table 4.1, we compare the algorithms in terms of the six dimensions along which these algorithms differ. These are: Node type (univariate vs multivariate), branching factor (two or K, number of classes), grouping of classes into two if the tree is binary, error (impurity) measure, and the methods for minimization to find the best split vector and split point.

4.1.3. Pruning

A greedy algorithm is a local search method where at each step, one tries to make the best decision and proceeds to the next decision, never backtracking and reevaluating a decision after it has been made. Similarly in decision tree induction, once a decision node is fixed, it cannot be changed after its children have been created. This may cause suboptimal trees where for example subtrees are replicated. The only

exception is the *pruning* of the tree.

In pruning, we consider replacing a subtree with a leaf node labelled with the class most heavily represented among the instances that are covered by the subtree. If there is overfitting, we expect the more complex subtree to learn the noise and perform worse than the simple leaf. If this is indeed the case on a validation set different from the training set, then the subtree is replaced by the leaf. Otherwise it is kept. It makes sense to start with the smaller subtrees closer to leaves and proceed up towards the root.

This process is called *post-pruning* to differentiate it from *pre-pruning*. In post-pruning, the tree is constructed until there is no misclassification error and then pruned simpler. In pre-pruning, the tree is not fully constructed until zero training error but is kept simple by early termination. At any node, if the dataset reaching that node is small, even if it is not pure, it is not further split and a leaf node is created instead of growing a subtree. Pre-pruning is faster. Post-pruning may be more accurate but is slower and requires a separate validation set.

4.2. Rule Induction Algorithms

A *rule* contains a conjunction of propositions (terms) and a class code which is the label assigned to an instance that is covered by the rule. We say that a rule *covers* an instance if all of the propositions evaluate true for that instance. An example rule containing two propositions for class C_2 is:

$$\text{IF } (x_2 = \text{`red'}) \text{ AND } (x_1 < 1.5) \text{ THEN } C_2 \tag{4.7}$$

The propositions are of the form $x_i = v$, $x_i < \theta$ or $x_i \geq \theta$, depending on respectively whether the input feature x_i is discrete or continuous (Rules can also be extended from propositions to the first-order case, but this is beyond the scope of this thesis). A *rule set* is an ordered list of such rules.

To test an instance, there are two possibilities: Either the rules in the rule set are ordered which we check sequentially and use the class of the first rule that covers the example. Or, we check all the rules and take a vote over the rules that cover the instance. In this thesis our focus is on the first type, which is called *sequential covering*.

Rule induction algorithms learn a rule set from a training set. Such algorithms have a number of desirable properties: (i) For each class C_i there is a rule set in conjunctive normal form. Each rule set merges rules with OR's, and each rule consists of conditions merged via AND's. Such rule sets are easy to understand, allowing knowledge extraction and validation by application experts. (ii) These methods are nonparametric in that they assume no a priori form on the model or class densities and fit their complexity to that of data. (iii) They learn fast and can be used on very large data sets with a large number of instances. (iv) They do their own feature extraction/dimensionality reduction and can be used on data sets with a large number of features. Because of these reasons, rule induction algorithms are more and more frequently used in many data mining and pattern recognition applications and are preferred over other black-box methods like artificial neural networks.

4.2.1. Survey on Rule Induction Algorithms

Figure 4.2 shows a taxonomy of different rule inducers with a tree structure. There are three main groups. Separate-and-conquer algorithms (rule induction algorithm), divide-and-conquer algorithms (tree induction algorithms) and reconsider-and-conquer algorithms.

Separate-and-conquer strategy solves a problem by iteratively solving smaller subproblems. First a solution is found to the problem, which creates single subproblem and this subproblem creates another single subproblem. The loop ends when a subproblem does not create another subproblem. Separate-and-conquer algorithms first find the best rule that explains part of the training data. After *separating* the examples those are covered by this rule, the algorithms *conquer* remaining data by finding next best rules recursively. Since every learned rule separates its covered examples from

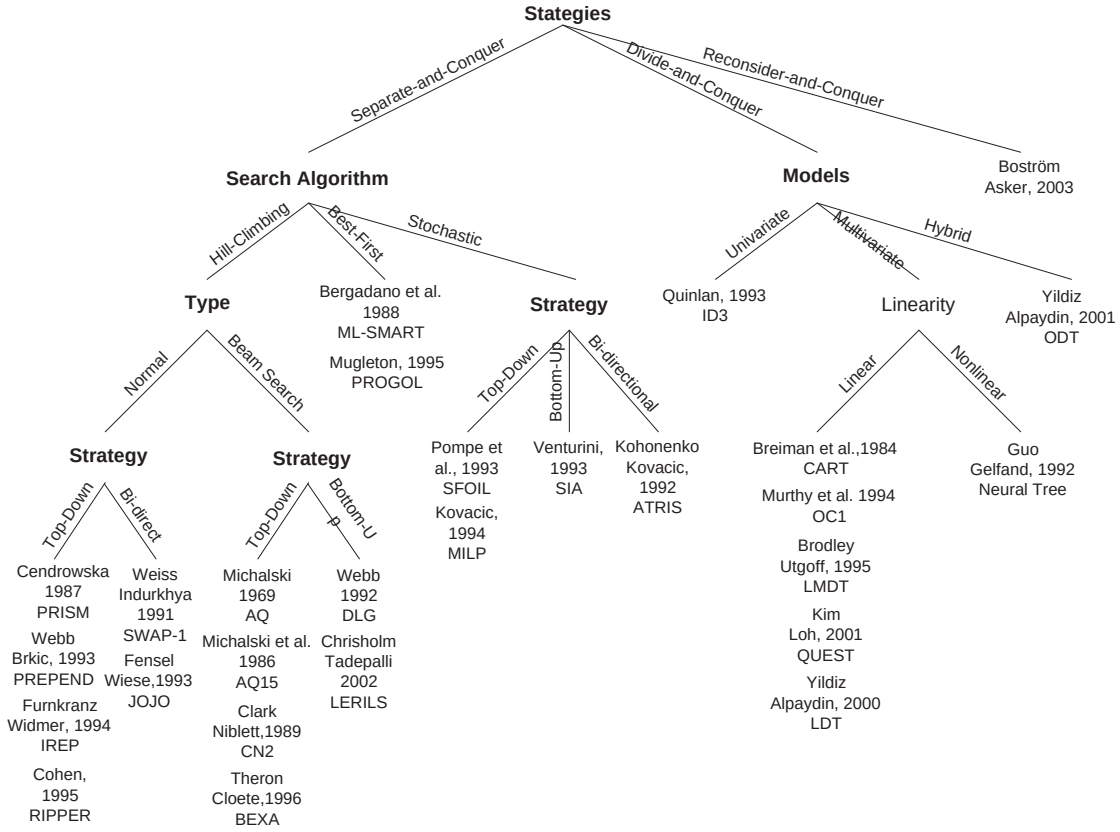


Figure 4.2. Rule induction algorithms

the training data, these algorithms are also called covering algorithms. Consequently, previously learned rules directly influence the data of the other rules. For an excellent review of separate-and-conquer algorithms see [66].

4.2.1.1. Hill-Climbing. Hill-climbing is a widely known and used technique in Artificial Intelligence. In Hill-climbing one starts from an initial solution (start state) and makes improvements (operators) iteratively over the initial solution. Since each of these improvements are local improvements, hill-climbing technique usually sticks into local minima while trying to discover global optima. In rule induction, one uses hill-climbing to find the best rule for a given training data. First, an initial solution is provided, depending on the search strategy. After that, refinements over the initial rule are applied. The search stops, when there is no further improvement for the rule is possible.

Normal Hill-Climbing. PRISM [67] is one of the oldest rule induction techniques, which uses hill-climbing in learning rules. It starts with an empty rule and adds conditions as long as the rule covers negative examples. The algorithm compares possible refinements with the purity measure $-log\frac{p}{p+n}$.

PREPEND [68] puts the new learned rule before the previously learned rules. The logic behind is that, general rules will be learned first and if we put these rules at the end of the list, they will also cover exceptions.

The two previous algorithms do not have any pruning step. Since hill-climbing is a greedy search method, pruning the rule set may improve the performance. For this reason several hill-climbing algorithms do use pruning. REP [69] prunes the rule set by deleting the last condition in a rule, I-REP [70] deletes each condition in a rule, GROW [71] deletes a final sequence of conditions, SWAP-1 [72] finds the best replacement for a condition. Like the post-pruning in the decision tree induction, these algorithms first finds the rule set that covers all positive examples in the trainset, which overfits the training data. Then they prune the rule set using another set, prune set.

Although post-pruning seems to improve the performance of the rule induction algorithms, it has drawbacks. One important drawback is that pruning a rule affects all subsequent rules. This is due to the nature of the separate-and-conquer. If one rule covers one example, subsequent rules can not cover that example. By pruning a previous rule, we increase the set of examples that are covered by that rule, which decrease the sets of examples those are covered by the subsequent rules. To overcome this problem, I-REP [70] and RIPPER [73] immediately prune a rule after they learn it. To do this, for each rule, they separate the examples into two sets grow set and prune set. They use grow set to *grow* a rule and use the prune set to immediately prune that rule.

PN-RULE [74] claim that existing sequential covering algorithms try to achieve maximum precision for each condition, which causes the rule set to be unsuccessful for the rare classes. This is due to the fact that, usually rare classes are considered

as false positives to the general rules. They use a two-phase design to overcome this problem, where in the first phase (P-phase) they predict presence of the target class, and in the second phase (N-phase) they predict absence of the target class. N-phase uses the true and false positives covered by the low accuracy P-rules. Afterwards for testing purposes, two-phases are combined to a single scoring mechanism.

JOJO [75] and SWAP-1 [72] are bi-directional hill-climbing rule induction algorithms, where they not only allow adding conditions but also allow removing conditions from a rule. SWAP-1 checks if a new learned condition can be dropped or can be replaced by a new condition to improve the rule.

Beam Search. To alleviate local minima problem of hill-climbing, several solutions are proposed. One of the solution is the beam search. In beam search, at each step of the hill-climbing a group of alternative solutions additionally to the best rule are stored. These alternative solutions are called beam and the number of these solutions are called beam size. The beam search reduces the efficiency of the hill-climbing by the size of the beam. Beam search in rule induction is applied by storing alternative rules additionally to the best rule. Then the search for the best refinement is done over all these rules.

AQ [76] is the predecessor of all rule induction systems. AQ and its antecedents AQ15 [77], AQ17 [78], and some other algorithms as CN2 [79], POSEIDON [80], BEXA [81] use beam-search as top-down to induce rules from most general to most specific manner. On the other hand, DLG [82] and LERILS [83] use beam-search as bottom-up to induce rules from most specific to most general manner.

4.2.1.2. Best-First. The search for the best rule over the training data can be thought as a search in the rule space. Each rule is a state and we want to find the optimal state (optimal rule) in this space. Refinements over a rule transports us to another state in the state space. Best first search specifically A^* [84] is one of the earliest strategies to find the optimal state in the state space. A^* stores all candidate states those are

reachable and it also removes unpromising states in order not to waste time to search whole state space.

Best-First search can be seen as a Beam-Search technique with an infinite beam size. ML-SMART [85] used that fact to generate a set of candidate rules. Since the number of alternative rules can grow exponentially, ML-SMART uses pruning heuristics to remove unpromising rules.

PROGOL [86] is also a best-first rule induction algorithm working top-down manner. PROGOL makes its best first search according to the A^* technique. According to the A^* technique each rule have its score as the sum of f(n) + g(n). f(n) is the performance of the rule, which is calculated as the number of positive examples covered minus the number of negative examples covered minus the length of the rule. g(n) is the heuristic that how the rule will behave if it was completed. PROGOL implements g(n) by estimating the number of conditions required to complete the rule.

Although Best-First search guarantees the optimal solution if the given heuristic is admissible, [68] showed that beam search techniques such as CN2 have better performance over the best-first techniques.

4.2.1.3. Stochastic. Another technique used to escape from local optima is stochastic search. Since following simpler steps to a best rule directs us to local optima, the idea in stochastic is to make bigger and random jumps in rule space to avoid local optima. In rule induction, bigger jumps correspond to making more than one refinements to a rule at a time, random jumps correspond to making random refinements over the rules. The probability of the refinements are directly proportional to the goodness of the rule applied. Promising rules have higher chance then unpromising rules to be refined.

Top-Down. Top-down approach is most commonly used search algorithm in separate-and-conquer learning. Search for the best rule starts from the most-general rule 'true' which covers all training data. Each refinement specializes the rule by adding

conditions. After each refinement, the new rule covers a subset of the previously covered data.

SFOIL [87] and MILP [88] used stochastic search to remove local minima problem of hill-climbing. When the hill-climbing search reaches an optima (local optima), they restart the search for best rule from other unexamined rule.

Bottom-Up. Bottom-up approach is direct inverse of the top-down approach. Search for the best rule starts from the most-specific rule which usually covers single example. Each refinement generalizes the rule by removing conditions. After each refinement, the new rule covers a superset of the previously covered data.

SIA [89] effectively uses genetic algorithm to generate best rule set. SIA maintains a list of candidate rules as in beam search called a generation of the genetic algorithm. This generation produces children (new rules), by cross-over operations such as exchanging the conditions of the two rules, by mutation as generating new rules for a parent rule.

Bi-directional. Top-down and bottom-up approaches are the extreme cases of a search technique like Divide-and-conquer and separate-and-conquer techniques. As a hybrid approach exists for two strategies, there exists also a hybrid of top-down and bottom-up approaches bi-directional approach. Bi-directional approaches such as ATRIS [90], can use both specialization and generalization operators. Since bi-directional approach can go in both directions in the rule space, the starting point, the initial rule may be (i) the most-general rule, (ii) the most-specific rule or (iii) a random rule, which is neither specific nor general.

4.2.2. C4.5Rules

C4.5Rules is based on the C4.5 tree induction algorithm, which is a greedy algorithm that searches for the best split and the best feature at each node in terms of information gain [56]. This corresponds to searching through $N_m - 1$ possible split points for a numeric feature (where N_m is the number of data instances reaching node m) and one split for a discrete feature. If the best feature x_i is numeric, it creates two children ($x_i < \theta$ and $x_i \geq \theta$) and divides the instance space of the parent node into two parts. If the best feature is discrete, it creates k children ($x_i = v_j, j = 1, \ldots, L$), where L is the number of possible values of the feature and divides the instance space of the parent node into L parts. Tree growing continues recursively with the newly generated child nodes until each node has instances from a single class, at which point it is marked as a leaf node and is labeled by the code of that class.

C4.5Rules starts with the C4.5 tree and converts it to a rule set by writing each path from the root to a leaf as a rule. The initial rule set therefore has as many rules as there are leaves in the tree [58]. These generated rules may contain superfluous conditions and may cause overfitting. C4.5Rules prunes rules by removing those conditions whose removal do not increase the error rate on the training set. After pruning the conditions in the rules, the best subset of the rules is searched according to MDL [91]. The search is exhaustive if the number of rules is small (less than or equal to ten), otherwise it is done by best first search starting from ten different subsets of rules.

4.2.3. Ripper

RIPPER learns rules from scratch starting from an empty rule set. It has two phases: In the first phase, it builds an initial set of rules, one at a time, and in the second phase, it optimizes the rule set m times, typically twice [73].

In the first phase, RIPPER learns rules one by one. First, propositional conditions are added one at a time to a rule, at each step choosing the condition that maximizes

the information gain. Choosing the best is done exhaustively by searching all possible split points ($x_i < \theta$ or $x_i \geq \theta$ for numeric features and $x_i = v$ for discrete features). When we insert a new condition, the coverage of the rule (number of instances that a rule covers) decreases, since some of the examples that are covered by the rule do not obey the new condition. We stop adding conditions further, when the current rule does not cover any negative examples.

The rule is then pruned to alleviate overfitting. Conditions are removed one by one, where each time the condition that mostly increases a rule value metric (based on the accuracy on the prune set) is selected for removal. We stop removing conditions when the rule value metric can not be increased further.

RIPPER uses MDL and in RIPPER, rules are added to a rule set to minimize the total description length. The total description length of a rule set is the number of bits to represent the rules plus the number of bits needed to identify the exceptions to the rules in the training set. We stop adding rules when the total description length of the rule set is larger than 64 bits from the minimum description length obtained so far or if the error rate of the new rule is larger than 0.5.

In the second phase, rules in the rule set are optimized. Two alternatives are grown for each rule. The first candidate, the replacement rule, is grown starting with an empty rule, whereas the second candidate, the revision rule, is grown starting with the current rule. These two rules and the original rule are compared and the one with the smallest description length is selected and put in place of the original rule.

RIPPER learns rules to separate a positive class from a negative class. A ($K > 2$)-class problem is converted into a sequence of $K - 1$ two-class problems. This is done first by sorting classes in the order of increasing prior probabilities. In order to solve the ith two-class problem, $i = 1, \ldots, K - 1$, C_i is taken as the positive class and the union of classes C_{i+1}, C_{i+2}, ..., C_K is taken as the negative class. For each two-class problem, there are the two phases of learning a rule set and optimizing it, as we discussed above. We therefore generate a sequence of rules and the most probable class

is the final default rule without conditions and it matches any instance which is not covered by any previous rule.

4.2.4. Multivariate Rule Induction

MRIPPER is the multivariate version of RIPPER where each condition is a linear combination of the features. MRIPPER differs from original RIPPER in several respects:

Because we take a weighted sum, discrete attributes should be converted to a numeric form; this is done by 1-of-L encoding by defining L dummy 0/1 variables for a discrete attribute with L possible values.

In training a multivariate linear decision, Fisher's LDA is used to find the weight vector $\boldsymbol{w} = [w_1, w_2, \ldots, w_d]^T$ that best separates the two classes

$$\boldsymbol{w} = \mathbf{S}^{-1}(\boldsymbol{m}_1 - \boldsymbol{m}_2) \tag{4.8}$$

where $\mathbf{S}$ is the total within class scatter matrix and $\boldsymbol{m}_1, \boldsymbol{m}_2$ are the means of the two classes [92]. The threshold w_0 is then calculated analytically for best separating the classes after projection. This calculation is analytical and does not require an exhaustive search as done in the univariate case, but the vector and matrix operations (e.g., matrix inversion) can be time-consuming when the dimensionality d is high. When there is few data or if the features are correlated, $\mathbf{S}$ is singular and in such a case, we use principal component analysis and map the data to the new space defined by the eigenvectors of $\mathbf{S}$ with nonzero eigenvalues, and do LDA.

MRIPPER has a learning time complexity of $\mathcal{O}(Nd^2)$ for constructing the covariance matrix (which is the costliest part), $\mathcal{O}(d^4)$ for finding the eigenvectors of the covariance matrix, $\mathcal{O}(d^2k)$ for converting the covariance matrix to the new k-dimensional covariance matrix and $\mathcal{O}(k^3)$ for taking the inverse of this new covariance matrix.

RIPPER uses MDL to check model complexity and avoid overfitting. This is done in two places: to stop adding conditions to a rule, and to stop adding rules to the rule set. The MDL calculation used in RIPPER cannot be generalized to the multivariate case because of the combinatorial number of possible hyperplanes that can be drawn [52]. We have therefore decided to use cross-validation as the model selection method in MRIPPER, instead of MDL. For this, at each stage, we leave out one-third of the dataset as validation set and use the remaining two-thirds for actual training. We stop adding a condition to a rule, or adding a rule to the rule set, if the error on the validation set stops decreasing. Similarly, in the optimization phase of MRIPPER, the two, revision and replacement rules, and the original rule, are compared according to the error rate on the validation set and the one with the smallest validation error rate is selected and put in place of the original rule.

5. APPLICATIONS

5.1. Decision Trees

In approximating the real (unknown) discriminant, with univariate nodes we are limited to a piecewise approximation using axis-aligned hyperplanes. With multivariate linear nodes, we can use arbitrary hyperplanes and thus approximate the discriminant better. It is clear that if the underlying discriminant is curved, a nonlinear approximation through a nonlinear multivariate node allows a better approximation using a smaller number of nodes and leaves. Thus there is a dependency between the complexity of a node and the size of the tree. With complex nodes the tree may be quite small; with simple nodes one may grow large trees.

However we should keep in mind that a quadratic model has $\mathcal{O}(d^2)$ parameters, compared to linear's $\mathcal{O}(d)$ and univariate's $\mathcal{O}(1)$. A complex model with a larger number of parameters requires a larger training dataset and risks overfitting on small amount of data. For example in Figure 4.1, the nonlinear split has less error than the linear split but is too wiggly. Thus one should be careful in tuning the complexity of a node with the properties of the data reaching that node.

Each node type has a certain bias; using multivariate linear nodes for example, we are assuming that the input space can be divided using hyperplanes into localized regions (volumes) where classes, or groups of classes are linearly separable. Using a decision tree with the same type of nodes everywhere, we assume that the same bias is appropriate at all levels.

This assumption is not always correct and at each node of the tree, which corresponds to a different subproblem defined by the subset of the training data reaching that node, a different model may be appropriate, and we should find and use the right model. For example we expect that though closer to the root a quadratic model may be used, as we get closer to the leaves, we have easier problems in effectively smaller

dimensional subspaces and at the same time, we have smaller training data and simple, e.g., univariate, splits may suffice and generalize better [93].

Therefore, the model selection problem in decision trees can be defined as choosing the best model at each node of the tree. In our experiments, we use three candidate models, namely, univariate, linear multivariate, and quadratic multivariate. At each node of the tree, we train these three models to separate two class groups from each other and choose the best.

```
1  Split FindBestSplit(n, t)
2      Create initial partition P
3      model = ModelSelectionDecisionTree(P, t)
4      bestgain = CalculateGain(model)
5      while improved
6          improved = FALSE
7          for i = 1 to K
8              Change class group of class i
9              model = ModelSelectionDecisionTree(P, t)
10             gain = CalculateGain(model)
11             if gain < bestgain
12                 bestsplit = split of model
13                 bestgain = gain
14                 improved = TRUE
15     return bestsplit
```

Figure 5.1. Pseudocode for finding the best split by considering splits with different complexities

Figure 5.1 shows the pseudocode for finding the best split at a decision node by considering splits with different complexities. Since LDA separates two classes from each other, if we have $K > 2$ classes, these classes must be divided into two class groups (partition) and LDA then finds the best split to separate these two class groups. The algorithm starts with creating the initial partition P and fits the best model (using one

of the model selection techniques) to that partition (Lines 2–4). After the initialization phase, we iteratively change the class group of each class i to get other partitions (Line 7), that is we move it from the left class group to the right class group or vice versa. We again fit the best model to those partitions and select the partition (and the model) which most decreases the information gain (Lines 9–14). We continue the loop until there is no further decrease in the information gain (Lines 5–14) and return the best split found (This heuristic of splitting $K > 2$ classes into two groups is originally proposed by Guo and Gelfand [61]).

```
1  BestModel ModelSelectionDecisionTree(P, t)
2      for s = Uni, Linear, Quadratic
3          switch (s)
4              Uni: Use univariate LDA to find best split, h = 2
5              Linear: Use LDA for linear kernel (x_1 + x_2 + ... + x_d + 1)
6                      Use SVD with ε = 0.99 to find k components, h = k + 1
7              Quadratic: Use LDA for quadratic kernel (x_1 + x_2 + ... + x_d + 1)^2
8                      Use SVD with ε = 0.99 to find m components, h = m + 1
9          L_s = Loglikelihood(s)
10         E_s = CalculateError(s)
11     switch (t)
12         AIC: Gen_s = -L_s + h
13         BIC: Gen_s = -L_s + h/2 log N
14         MDL: Gen_s = Exceptionlength(s) + λh
15         SRM: Gen_s = E_s + ε/2 (1 + sqrt(1 + 4E_s/ε)), where ε = a_1 (h[log(a_2 N/h) + 1] - log(ν))/N
16     l = ∀s, Gen_l ≤ Gen_s
17     return l;
```

Figure 5.2. Pseudocode of AIC, BIC, MDL and SRM model selection techniques for decision tree problem

Figure 5.2 shows the pseudocode to find the best model for a given partition of classes for penalization techniques and SRM technique. First, we train all three candidate models, univariate, linear and quadratic (Lines 3–8). For the univariate

model, we use univariate LDA and the model complexity is two, one for the index of the used attribute and one for the threshold (Line 4). For the multivariate linear model, we use multivariate LDA and to avoid a singular covariance matrix, we use singular value decomposition with $\epsilon = 0.99$ to get k new dimensions and the model complexity is $k + 1$ (Lines 5–6). For the multivariate quadratic model, we choose polynomial kernel of degree 2 ($(x_1 + x_2 + \ldots + x_d + 1)^2$) and use multivariate LDA to find the weights. Again to avoid a singular covariance matrix, we use singular value decomposition with $\epsilon = 0.99$ to get m new dimensions and the model complexity is $m + 1$ (Lines 7–8). Then, we calculate the generalization error of each candidate model with the corresponding model complexity and the data loglikelihood error (Lines 11–15). In the last step, we choose the optimal model having the least generalization error (Line 16).

```
1  real LogLikelihood(Split)
2      L = Σ_{i=1}^{K} N_i^L log2(N_i^L / N_L) + Σ_{i=1}^{K} N_i^R log2(N_i^R / N_R)
3      return L
```

Figure 5.3. Pseudocode for calculating the loglikelihood at a decision tree node

At a decision tree node, the loglikelihood of a single instance of class i is $log_2(\frac{N_i}{N})$, where N_i is the number of instances of class i and N is the total number of instances at that node. Then the loglikelihood of class i will be $N_i log_2(\frac{N_i}{N})$ and the total loglikelihood of the node will be $\sum_{i=1}^{K} N_i log_2(\frac{N_i}{N})$. Figure 5.3 shows the pseudocode for calculation of the loglikelihood at a decision tree node. Since we have two child nodes for a binary split, the loglikelihood of both children must be calculated and summed up. N_L and N_R show the number of instances in the left and right nodes respectively.

To calculate the error at a decision node, we must first assign classes to the left and right child nodes. Assume that C_L and C_R are classes assigned to the left and right nodes respectively. Assume also that N_{C_L} and N_{C_R} are the number of instances of the classes C_L and C_R respectively. Then the error at the decision node will be calculated by subtracting the number of instances of these two classes from the total number of instances (Figure 5.4).

```
1  real CalculateError(Split)
2      e = N − N_{C_L} − N_{C_R}
3      error = e / N
4      return error
```

Figure 5.4. Pseudocode for calculating the training error at a decision tree node

```
1  MDL Exceptionlength(Split)
2      C = Σ_{i=1}^{K} N_i^L
3      U = Σ_{i=1}^{K} N_i^R
4      fp = C − N_{C_L}
5      fn = U − N_{C_R}
6      e = fp + fn
7      length = log2(|D| + 1) + fp (− log2(e/2C)) + (C − fp)(− log2(1 − e/2C))
8          + fn(− log2(fn/2U)) + (U − fn)(− log2(1 − fn/U))
9      return length
```

Figure 5.5. Pseudocode for calculating the description length of the exceptions at a decision tree node

The pseudocode for calculating the description length of the exceptions at a decision tree node is given in Figure 5.5. C and U are the number of instances in the left and right child nodes respectively. False positives (fp) and false negatives (fn) are misclassified examples in the left and right child nodes respectively, therefore the total error e is defined as their sum. According to these values, the description length of the parent node is calculated in lines 7–8.

Figure 5.6 shows the pseudocode to find the best model for a given partition of classes for MultiTest-based model selection technique. First, we generate ten training and validation sets using 5×2 cross-validation (Line 2). Second, for each training and validation set pair, we train all three candidate models, univariate, linear and quadratic on the train set and get their performances on the validation set (Lines 3–11). The

```
1  BestModel MultiTestBasedModelSelectionDecisionTree(P)
2      Generate k training and validation sets
3      for j = 1 to k
4          for s= Uni, Linear, Quadratic
5              switch (s)
6                  Uni: Use univariate LDA to find best split
7                  Linear: Use LDA for linear kernel (x_1 + x_2+...+x_d + 1)
8                          Use SVD with ε = 0.99 to find k components
9                  Quadratic: Use LDA for quadratic kernel (x_1 + x_2+...+x_d + 1)^2
10                         Use SVD with ε = 0.99 to find m components
11             Calculate E_s^j, error of split s on validation set j
12     t = Combined 5×2 t test
13     l = MultiTest(Uni–Linear–Quadratic, t, α)
14     return l;
```

Figure 5.6. Pseudocode of MultiTest-based model selection for decision tree induction

training phase is the same as in other model selection techniques but the training set size is smaller. In the last step, we choose the optimal model using MultiTest algorithm (Line 13) where we use the combined 5×2 paired t test to compare the expected error rates of the models (Line 12).

5.2. Rule Induction

The model selection problem in rule induction can be defined as choosing the best model at each condition in a rule. In our experiments, we use three candidate models, namely, univariate, linear multivariate, and quadratic multivariate. At each condition of a rule, we train these three models to separate the positive class from the negative class and choose the best.

The pseudocode for learning ruleset from examples using model selection techniques AIC, BIC, SRM, MDL, and MultiTest-based model selection technique are given

```
1  Ruleset LearnRulesetModelSelection(X, t)
2      RS = {}
3      Classes C_i ordered in increasing prior probability
4      for p = 1 to K − 1
5          Pos = C_p, Neg = C_{p+1}, ..., C_K
6          RS_p = {}
7          while X contains positive samples
8              r = GrowRuleModelSelection(X, t)
9              PruneRuleModelSelection(r, t)
10             if CalculateError(r)> 0.5
11                 break
12             else
13                 RS_p = RS_p + r
14                 Remove examples covered by r from X
15         for i = 1 to 2
16             OptimizeRulesetModelSelection(RS_p, X, t)
17             SimplifyRulesetModelSelection(RS_p, X, t)
18         RS = RS + RS_p
19     return RS
```

Figure 5.7. Pseudocode for learning ruleset using model selection t on dataset X

in Figures 5.7 and 5.8 respectively. We start learning from an empty ruleset (Line 2), and learn ruleset for each class C_i one at a time. To do this, we sort the classes increasingly according to prior probabilities (Line 3), and we try to separate each class C_p (positives) from the remaining classes C_{p+1}, ..., C_K (negatives) (Line 5). Rules are grown (Line 8), pruned (Line 9) and added (Line 13) one by one to the ruleset. We stop adding rules when (i) the training error of the rule is larger than 0.5 (Line 10) or (ii) if there are no remaining positive examples (Line 7). After learning a ruleset, it is optimized twice (Line 16) and pruned (Line 17). The difference of MultiTest-based model selection technique occurs in two places: First, the rule is grown and pruned with separate datasets called growset G and pruningset P in the MultiTest-based model se-

```
1   Ruleset LearnRulesetMultiTestbasedModelSelection(X, V)
2       RS = {}
3       Classes C_i ordered in increasing prior probability
4       for p = 1 to K − 1
5           Pos = C_p, Neg = C_{p+1}, ..., C_K
6           RS_p = {}
7           while X contains positive samples
8               Divide X into Growset G and Pruneset P
9               r = GrowRuleMultiTestbasedModelSelection(G)
10              PruneRuleMultiTestbasedModelSelection(r, P)
11              if CalculateError(r)> 0.5
12                  break
13              else
14                  RS_p = RS_p + r
15                  Remove examples covered by r from X
16          for i = 1 to 2
17              OptimizeRulesetModelSelection(RS_p, X, V)
18              SimplifyRulesetModelSelection(RS_p, V)
19          RS = RS + RS_p
20      return RS
```

Figure 5.8. Pseudocode for learning a ruleset using MultiTest-based model selection technique

lection technique, whereas in the other model selection techniques the growing is done on the whole dataset and the pruning is done according to the estimated generalization error of the rule (which is based on complexity and not on validation error). Similarly, the optimization and simplification are done using a validation dataset V in MultiTest-based model selection technique, whereas in the other model selection techniques it is done according to the generalization error of the ruleset on the whole dataset.

The pseudocode for growing a rule from examples using model selection tech-

```
1 Rule GrowRuleModelSelection(X, t)
2   r = {}
3   while X covers negative examples
4       for c = Uni, Linear, Quadratic condition
5           switch (c)
6                   Uni: Use exhaustive search to find best univariate condition, h = 2
7                   Linear: Use LDA with linear kernel (x_1 + x_2 + ... + x_d + 1)
8                       Use SVD with ε = 0.99 to find k components, h = k + 1
9                   Quadratic: Use LDA with quadratic kernel (x_1 + x_2 + ... + x_d + 1)^2
10                      Use SVD with ε = 0.99 to find m components, h = m + 1
11          Gen_c = GeneralizationError(c, X, t)
12      l = ∀c, Gen_l ≤ Gen_c
13      r = r ∪ l
14      Remove examples from X that are covered by l
15  return r
```

Figure 5.9. Pseudocode for growing a rule using dataset X with model selection technique t

niques AIC, BIC, SRM, MDL, and MultiTest-based model selection technique are given in Figures 5.9 and 5.10 respectively. We start learning from an empty rule (Line 2), and add conditions one by one. To add a condition, we train three different candidate models, univariate model (Line 6), multivariate linear model (Lines 7–8) and multivariate quadratic model (Lines 9–10). For MultiTest-based model selection technique, we first generate ten training and validation sets (Line 4), find the best model using MultiTest algorithm on the validation errors of the models and the combined 5×2 t test (Lines 13–14). For other model selection techniques, the best model has the least generalization error on the dataset X (Lines 11–12). When we find the best condition (model), we add the condition to the rule and remove examples covered by that condition from the dataset (Lines 13–14). We stop adding conditions to a rule when there are no negative examples in the dataset (Line 3).

```
1 Rule GrowRuleMultiTestbasedModelSelection(X)
2   r = {}
3   while X covers negative examples
4       Generate k training and validation sets
5       for j = 1 to k
6           for c = Uni, Linear, Quadratic condition
7               switch (c)
8                   Uni: Use exhaustive search to find best univariate condition
9                   Linear: Use LDA with linear kernel (x_1 + x_2 + ... + x_d + 1)
10                      Use SVD with ε = 0.99 to find k components
11                  Quadratic: Use LDA with quadratic kernel (x_1 + x_2 + ... + x_d + 1)^2
12                      Use SVD with ε = 0.99 to find m components
13          E_c^j = CalculateError(c, V_j)
14      l = MultiTest(Uni–Linear–Quadratic, Combined 5×2 t test, α)
15      r = r ∪ l
16      Remove examples from X that are covered by l
17  return r
```

Figure 5.10. Pseudocode for growing a rule using dataset X with MultiTest-based model selection technique

The pseudocode for pruning a rule from examples is given in Figures 5.11 and 5.12 respectively. In pruning, each time in the MultiTest-based model selection technique, we find the condition whose removal increases the rule value metric the most, whereas with the other model selection techniques, we find the condition whose removal decreases the generalization error the most (Lines 9–12). If we find such a condition, we remove it (Lines 14–15). We stop pruning conditions when there is no such condition (Line 4).

The pseudocode for optimizing a ruleset using model selection techniques AIC, BIC, SRM, MDL, and MultiTest-based model selection technique are given in Figures 5.13 and 5.14 respectively. In the optimization phase, two alternatives are grown

```
1  Rule PruneRuleModelSelection(r, t)
2      improved = TRUE
3      bestgen = GeneralizationError(r, t)
4      while improved
5          improved = FALSE
6          for each condition c in r
7              r = r − c
8              gen = GeneralizationError(r)
9              if (gen < bestgen)
10                 improved = TRUE
11                 conditionremove = c
12                 bestgen = gen
13             r = r ∪ c
14         if improved
15             r = r− conditionremove
16     return r
```

Figure 5.11. Pseudocode for pruning rule r using model selection technique t

for each rule (Line 2). The first candidate, the replacement rule, is grown (Line 4) and pruned (Line 5) starting with an empty rule, whereas the second candidate, the revision rule, is grown (Line 6) and pruned (Line 7) starting with the current rule. The replacement rule is grown from the original sample in MultiTest-based model selection technique, whereas in other model selection techniques it is grown from a bootstrap sample generated from the original sample (Line 3). These two rules and the original rule are compared and the one with the smallest error (Lines 10–13) (validation error for MultiTest model selection technique, generalization error for other model selection techniques) is selected and put in place of the original rule (Line 14).

The pseudocode for simplifying a ruleset using model selection techniques AIC, BIC, SRM, MDL, and MultiTest-based model selection technique are given in Figures 5.15 and 5.16 respectively. In simplifying the ruleset, rules are pruned in reverse order

```
1  Rule PruneRuleMultiTestbasedModelSelection(r, X)
2      improved = TRUE
3      bestmetric = RuleValueMetric(r, X)
4      while improved
5          improved = FALSE
6          for each condition c in r
7              r = r − c
8              metric = RuleValueMetric(r, X)
9              if (metric > bestmetric)
10                 improved = TRUE
11                 conditionremove = c
12                 bestmetric = metric
13             r = r + c
14         if improved
15             r = r− conditionremove
16     return r
```

Figure 5.12. Pseudocode for pruning a rule r on dataset X using MultiTest-based model selection technique

(Line 3). We prune a rule from the ruleset if its removal decrease the error (validation error for MultiTest-based model selection technique, generalization error for other model selection techniques) (Lines 4–8).

The pseudocode for calculating the generalization error of a condition, rule or a ruleset using model selection techniques AIC, BIC, MDL and SRM is given in Figure 5.17. The loglikelihood of the dataset X is calculated using the pseudocode in Figure 5.18. The number of parameters of a condition is 2 if the univariate model is used, $k + 1$ if multivariate linear model is used and $m + 1$ if the quadratic model is used where k and m are the number of dimensions after SVD. The number of parameters of a rule is obtained by summing up the complexities of its conditions. The number of parameters of a ruleset is obtained by summing up the complexities of its rules.

```
1  Ruleset OptimizeRulesetModelSelection(RS, X, t)
2      for each rule r in RS
3          Create a bootstrap sample B from X
4          r_replace = GrowRuleModelSelection(B)
5          PruneRuleModelSelection(r_replace, X)
6          r_revise = GrowRuleModelSelection(X)
7          PruneRuleModelSelection(r_revise, X)
8          RS_replace = RS − r ∪ r_replace
9          RS_revise = RS − r ∪ r_revise
10         E = GeneralizationError(RS, t)
11         E_replace = GeneralizationError(RS_replace, t)
12         E_revise = GeneralizationError(RS_revise, t)
13         r_min = Select rule with min(E, E_replace, E_revise)
14         RS = RS − r ∪ r_min
15     return RS
```

Figure 5.13. Pseudocode for optimizing ruleset RS using model selection technique t on dataset X

```
1  Ruleset OptimizeRulesetMultiTestbasedModelSelection(RS, X, V)
2      for each rule r in RS
3          Divide X into Growset G and Pruneset P
4          r_replace = GrowRuleMultiTestbasedModelSelection(G)
5          PruneRuleMultiTestbasedModelSelection(r_replace, P)
6          r_revise = GrowRuleMultiTestbasedModelSelection(G)
7          PruneRuleMultiTestbasedModelSelection(r_revise, P)
8          RS_replace = RS − r ∪ r_replace
9          RS_revise = RS − r ∪ r_revise
10         E = CalculateError(RS, V)
11         E_replace = CalculateError(RS_replace, V)
12         E_revise = CalculateError(RS_revise, V)
13         r_min = Select rule with min(E, E_replace, E_revise)
14         RS = RS − r ∪ r_min
15     return RS
```

Figure 5.14. Pseudocode for optimizing ruleset RS using MultiTest-based model selection technique on training set X and validation set V

```
1  Ruleset SimplifyRulesetModelSelection(RS, X, t)
2      E = GeneralizationError(RS, t)
3      for each rule r in RS in reverse order
4          RS_removed = RS − r
5          E_removed = GeneralizationError(RS_removed, t)
6          if E_removed < E
7              RS = RS − r
8              E = E_removed
9      return RS
```

Figure 5.15. Pseudocode for simplifying ruleset RS using model selection technique t and validation dataset X

```
1  Ruleset SimplifyRulesetMultiTestbasedModelSelection(RS, X)
2      E = CalculateError(RS, X)
3      for each rule r in RS in reverse order
4          RS_removed = RS − r
5          E_removed = CalculateError(RS_removed, X)
6          if E_removed < E
7              RS = RS − r
8              E = E_removed
9      return RS
```

Figure 5.16. Pseudocode for simplifying ruleset RS using MultiTest-based model selection technique on dataset X

```
1  Real GeneralizationError(c, X, t)
2      L = Loglikelihood(c, X)
3      E_t = CalculateError(c, X)
4      h = Complexity(c)
5      switch (t)
6          AIC: E_g = −L + h
7          BIC: E_g = −L + h/2 log N
8          MDL: E_g = Exceptionlength + λh
9          SRM: E_g = E_t + ε/2 (1 + sqrt(1 + 4E_t/ε)), where ε = a_1 (h[log(a_2 N/h)+1] − log(ν))/N
10     return E_g
```

Figure 5.17. Pseudocode for calculating generalization error of a condition, rule or a ruleset using model selection technique t

```
1  real LogLikelihood(c, X)
2      cp = number of positive examples covered by c
3      C = number of examples covered by c
4      up = number of positive examples uncovered
5      U = number of examples uncovered
6      L = cp log2(cp/C) + up log2(up/U)
7      return L
```

Figure 5.18. Pseudocode for calculating the loglikelihood for a condition, rule or a ruleset c on dataset X

6. EXPERIMENTS

6.1. Toy Problem: Polynomial Regression

6.1.1. Regression

This section describes an empirical comparison of the classical model selection methods with our proposed MultiTest-based approach in the context of polynomial regression. First, we describe the experimental setup for comparisons, then we discuss the bias-variance dilemma for polynomial regression and finally we give the results of the comparison experiments.

6.1.1.1. Experimental Setup. In order to compare various model selection techniques on polynomial regression, we have generated a random dataset (x^t, r^t), $t = 1, \ldots, N$ where random x^t values are uniformly distributed in [0, 5] interval and r^t values are generated from the target function $f(x)$ with added gaussian noise ϵ having 0 mean and σ^2 variance

$$f(x) = 2\sin(1.5x) \tag{6.1}$$

The target function and one noisy sample generated from that function is shown in Figure 6.1. Three different sample sizes were used for training data: small ($N = 25$), medium ($N = 100$) and large ($N = 1000$). The test instances are generated in single size ($N = 1000$). The noise level is taken as $\sigma^2 = 0.5$.

The approximating functions are linear estimators with polynomial basis func-

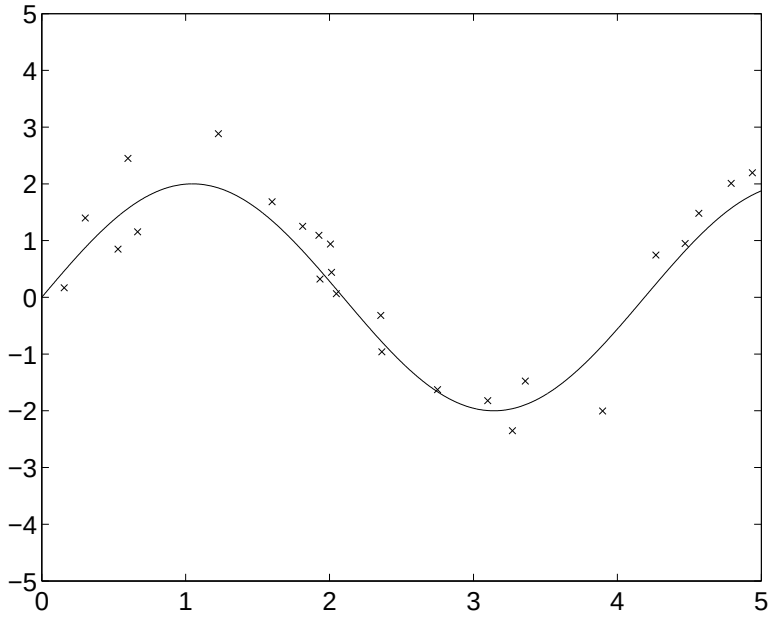


Figure 6.1. Target function $f(x) = 2\sin(1.5x)$, and one noisy dataset sampled from the target function

tions

$$g(x) = \sum_{i=1}^{m-1} w_i x^i + w_0 \tag{6.2}$$

where we consider polynomials of degree $m = 0$ - 15 as possible candidates for model selection. The parameters of the polynomial $(w_i, i = 0, \ldots, m-1)$ are estimated by linear least squares fit. We have used the squared error loss to calculate training and validation errors.

6.1.1.2. Bias-Variance Dilemma. In Chapter 2, we discussed the bias-variance dilemma where we saw that when the complexity of the estimator increases, it better fits the underlying distribution (smaller bias), whereas small changes in the training data will cause greater changes in the fitted estimators (higher variance). Tuning model complexity then corresponds to finding the best trade-off between the bias and variance.

To show the bias-variance dilemma in the polynomial regression, we have gen-

erated M training sets ($M = 100, N = 25$) from the target function $f(x)$. For each dataset i, we form the polynomial estimator $g_i(x)$ and the average of M estimators: $\overline{g}(x)$

$$\overline{g}(x) = \frac{1}{M}\sum_{i=1}^{M} g_i(x) \tag{6.3}$$

The estimated bias and variance are calculated on a separate test set having N instances ($N = 1000$):

$$\begin{aligned} Bias &= \frac{1}{N}\sum_{t=1}^{N}[\overline{g}(x^t) - f(x^t)]^2 \\ Variance &= \frac{1}{NM}\sum_{t=1}^{N}\sum_{i=1}^{M}[g_i(x^t) - \overline{g}(x^t)]^2 \end{aligned} \tag{6.4}$$

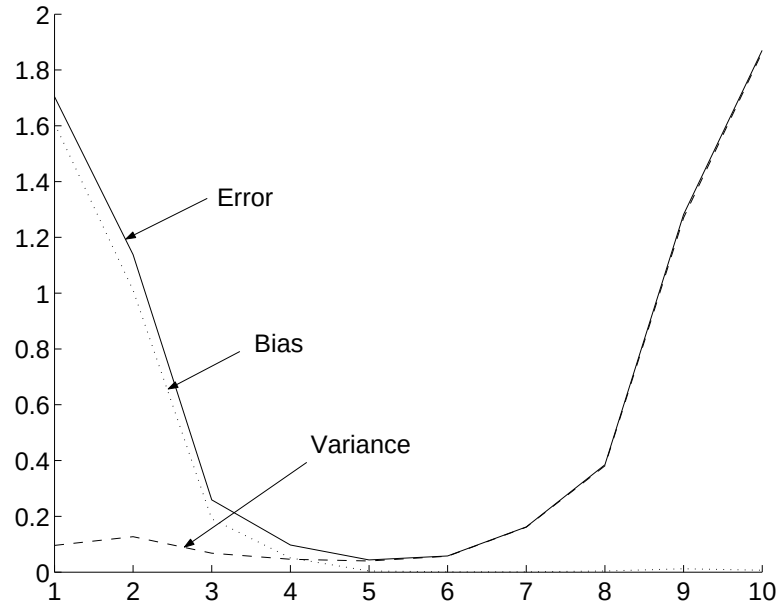
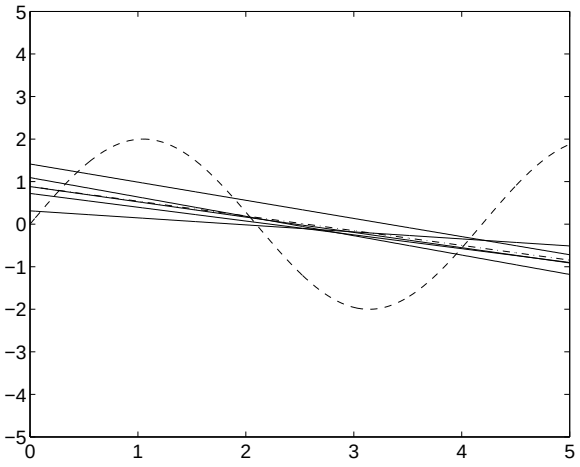


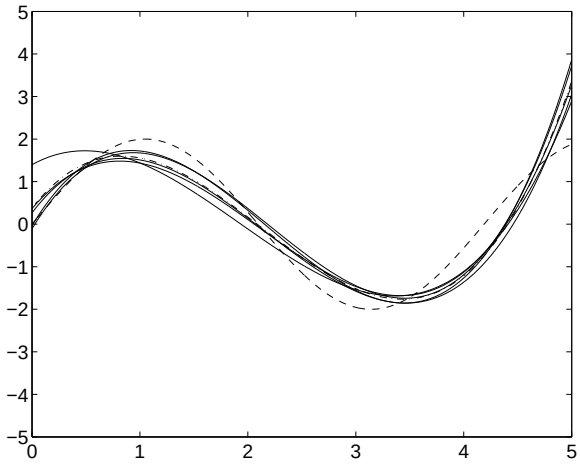
Figure 6.2. Bias, variance and error values for polynomials of order 1 to 10

Figure 6.2 shows bias, variance and error values of polynomials of order 1 to 10. We see that the bias decreases and the variance increases as the order of the polynomials increases. The polynomial of order 10 has the smallest bias, the one of order 1 has the smallest variance and the one of order 5 has the minimum error (optimal model

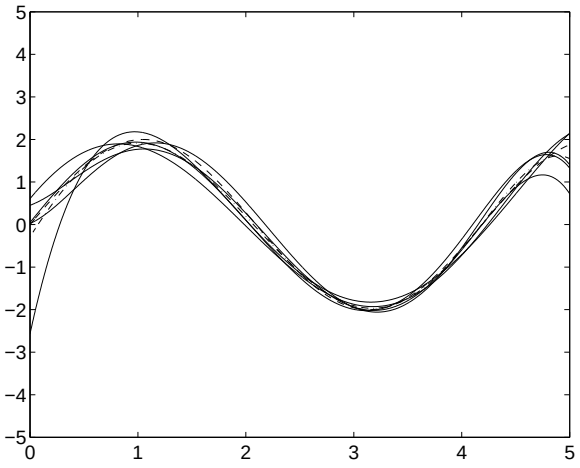
complexity).



(a) Order 1



(b) Order 3



(c) Order 5

Figure 6.3. Five polynomial fits are shown as solid lines, $\overline{g}(x)$ is shown in dash-dot style and original function $f(x)$ is shown as dashed line

For visualization purposes, we took five samples having $N = 25$ instances and plotted the corresponding five polynomial fits ($g_i(x)$, $i = 1, \ldots, 5$), the average of them ($\overline{g}(x)$) and the original function $f(x)$. (a), (b) and (c) parts of the Figure 6.3 show the polynomial fits and $\overline{g}(x)$ of orders 1, 3 and 5 respectively. Again we see that, as the order of polynomials increases, the average of the polynomials gets closer to the original function (bias decreases), whereas the polynomial fits around the average fluctuates more (variance increases).

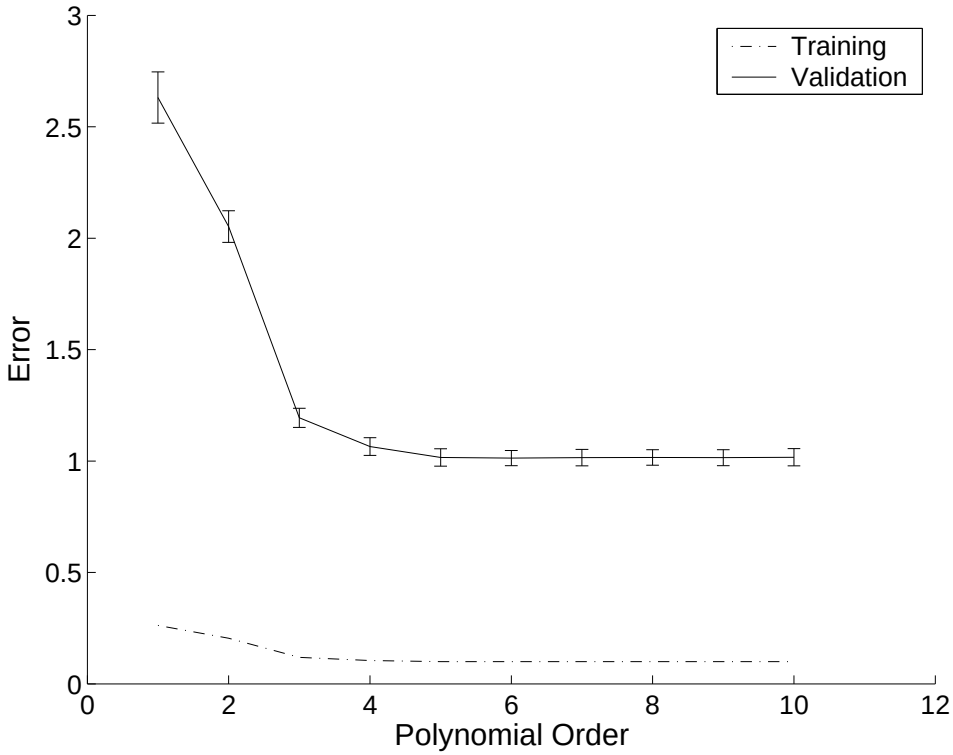


Figure 6.4. Training and validation errors of 100 random samples without noise. The average of training error, and the average and standard deviations of the validation error are shown

What is the optimal model complexity? In order to get the optimal model complexity, we generated $M = 100$ random samples from the original model without noise and fit polynomials of order 1 to 10. The performance of the polynomials are then tested on 100 validation datasets and the average and standard deviations of the validation errors are found (See Figure 6.4). According to the one standard deviation rule, we first choose the model with the smallest validation error, which is 5. After that, we look if there is a simpler model that has validation error no more than one standard deviation of the model with degree 5, which is 4. Therefore we can say that the fourth degree polynomial is the optimal model for this problem.

6.1.1.3. Comparison of Model Selection Techniques. In this section, we will give the empirical comparison of five different model selection techniques, namely AIC, BIC, MDL, SRM and our proposed MultiTest based cross-validation on polynomial regres-

sion problem. The comparison is done for three different sample sizes (small, medium and large). Two different criteria are used for comparison, namely the complexity of the optimal model and the error of the model on the test set.

```
1  BestDegree ModelSelectionTechniquePolyReg(maxd, t, N)
2      for i = 0 to maxd
3          Fit polynomial of order i using least squares fit
4          Calculate E_t^i /*training error*/
5          h = i + 1 /*complexity measure*/
6          switch (t)
7              AIC: E_g^i = E_t^i (1+h/N)/(1-h/N)
8              BIC: E_g^i = E_t^i (N+h(ln N-1))/(N-h)
9              MDL: E_g^i = E_t^i + λh
10             SRM: E_g^i = E_t^i/(1-c√ε) where ε = a_1 (h[log(a_2 N/h)+1]-log(ν))/N
11     l = ∀i, E_g^l ≤ E_g^i
12     return l;
```

Figure 6.5. Pseudocode of AIC, BIC, MDL and SRM model selection techniques for polynomial regression problem. $maxd$ is the maximum model complexity, t is the name of the model selection technique and N is the number of training instances

The penalized model selection techniques and SRM consist of four main steps (See Figure 6.5). In the first step, they fit polynomials with different model complexities to the training set (Line 3). In the second step, they calculate the training error of each model (Line 4). In the third step, they estimate the generalization error of each model by adding the optimism difference to the training error(Lines 5–10). In the last step, they choose the optimal model having the least generalization error E_g (Line 11).

The MultiTest-based model selection technique consists of three main steps (See Figure 6.6). In the first step, we generate k training and test sets for cross-validation (Line 3). For small sample sizes, we use leave-one-out type cross-validation ($k = N$), for medium and large sample sizes, we use 5×2 cross-validation ($k = 10$). In the second step, for each training set, we fit polynomials of degree i and calculate the test error

```
1  BestDegree MultiTestBasedModelSelectionPolyReg(maxd, N)
2      for i = 0 to maxd
3          Generate k training and test sets
4          for j = 1 to k
5              Fit polynomial of order i using least squares fit
6              Calculate E_i^j /*training error of model i on test set j*/
7      if small sample size
8          t = K-Fold Crossvalidated Paired t test
9      else
10         t = Combined 5×2 t test
11     l = MultiTest(0 ... maxd, t, α)
12     return l;
```

Figure 6.6. Pseudocode of MultiTest based model selection for polynomial regression problem. $maxd$ is the maximum model complexity and N is the number of training instances

(Lines 4–6). In the last step, we find the optimal model by the MultiTest algorithm using the appropriate statistical test (Lines 7–11). Since there are not enough instances to make 5×2 cross-validation for small sample sizes, for this setting leave-one-out test errors are compared with K-Fold cross-validated paired t test (Section B.3.2). Since there is not such a problem for medium and large sample sizes, we have used our proposed combined 5×2 cv t test (Section 3.1) in those settings.

The performances of different model selection techniques are affected by the training sample. Therefore, to create a valid comparison between the model selection techniques, the model selection experiment is repeated with 1,000 different training and validation samples. And at each time, the same training and validation sample pair is used for all model selection techniques. Figures 6.7, 6.8 and 6.9 show the frequency with which the polynomials of each degree is selected as the best model by the five different model selection techniques on small, medium and large samples respectively.

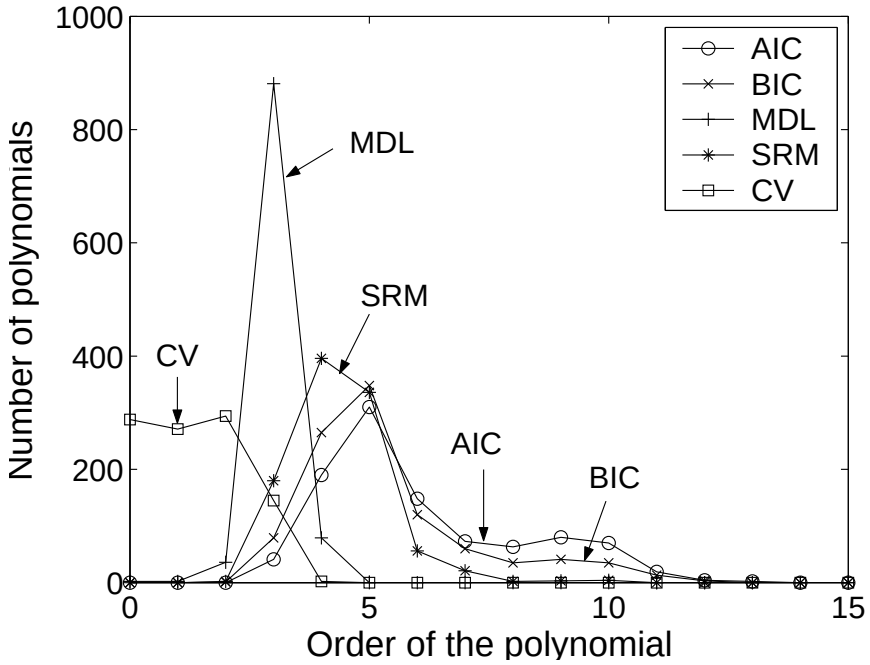


Figure 6.7. Comparison of model selection techniques for small sample size in terms of model complexities

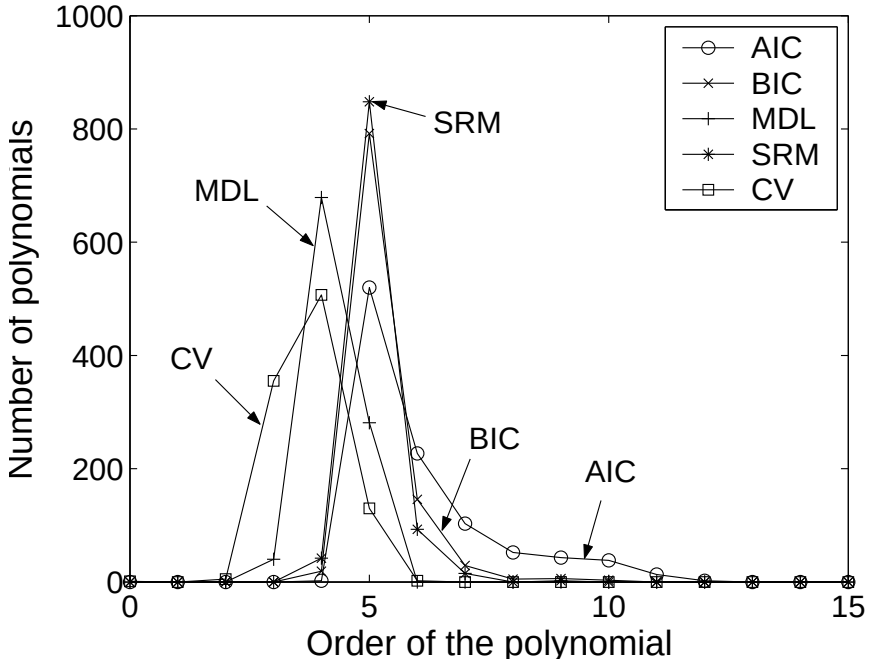


Figure 6.8. Comparison of model selection techniques for medium sample size in terms of model complexities

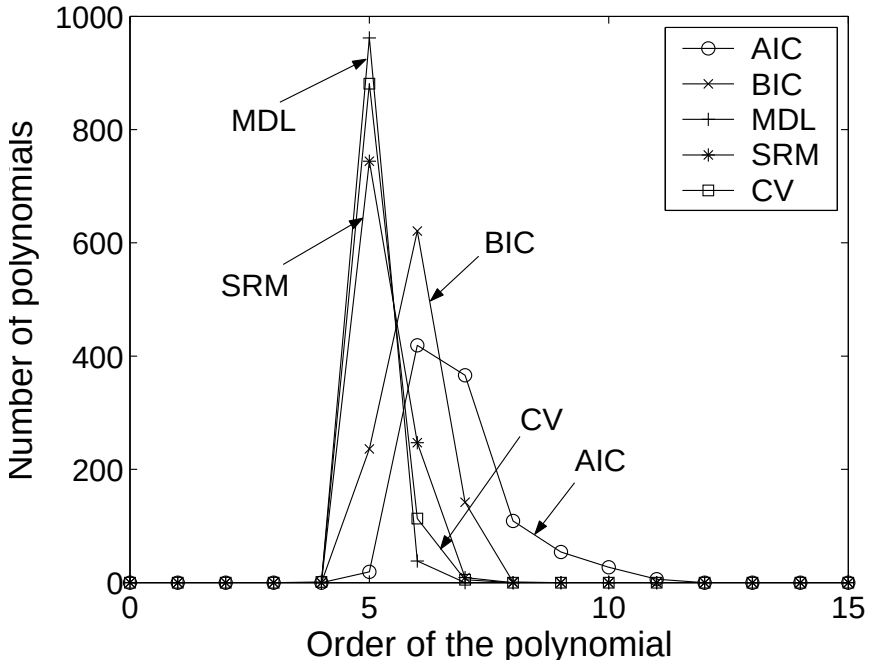


Figure 6.9. Comparison of model selection techniques for large sample size in terms of model complexities

For small sample sizes, AIC and BIC select mostly the polynomial with degree five as the optimal model, whereas SRM selects the fourth degree polynomial, MDL selects the third degree polynomial and MultiTest-based cross-validation approach selects the second degree polynomial. For medium sample sizes, AIC, BIC and SRM select mostly the fifth degree polynomial as the optimal model, whereas MDL and MultiTest-based cross-validation select the fourth degree polynomial. For large sample sizes, AIC, BIC select the sixth degree polynomial, whereas SRM, MDL and MultiTest-based cross-validation select the fifth degree polynomial.

Generally, AIC and BIC results are nearly the same, as expected from their formula. Due to the $\log N$ term in its formula, BIC punishes more heavily complex models than AIC does, especially when the sample size increases. Both criteria have larger variance than the other three model selection methodologies and usually select more complex models than the others.

SRM and MDL do not have a larger variance in their optimal model selection

and SRM chooses slightly more complex models than the MDL-based model selection.

Our proposed MultiTest-based CV model selection methodology usually selects simpler models than the other model selection methodologies. MultiTest algorithm uses statistical tests to compare the expected error rates of different models and it prefers a simple model if it is not statistically significantly worse than a complex model. For small sample sizes, the error rates may fluctuate and for that reason it is hard for a statistical test to show significant difference between the expected error rates of two models.

Table 6.1. Calculated $\hat{\sigma}^2$ values for different polynomial degrees and for different sample sizes

Degree	Sample Size		
	Small	Medium	Large
1	2.13 ±0.43	2.12±0.21	2.12 ±0.07
2	1.85 ±0.43	1.87±0.21	1.87 ±0.06
3	1.27 ±0.35	1.30±0.16	1.30 ±0.05
4	0.41 ±0.13	0.43±0.07	0.44 ±0.02
5	0.29 ±0.09	0.30±0.04	0.30 ±0.01
6	0.25 ±0.08	0.26±0.04	0.25 ±0.01
7	0.25 ±0.08	0.25±0.04	0.25 ±0.01
8	0.25 ±0.09	0.25±0.04	0.25 ±0.01
9	0.25 ±0.09	0.25±0.04	0.25 ±0.01
10	0.25 ±0.09	0.25±0.04	0.25 ±0.01
11	8.36 ±254.77	0.25±0.04	0.25 ±0.01
12	7.71 ±123.26	8.92±267.14	12.14±369.39
13	32.63 ±559.90	1.38±11.40	1.15 ±14.26
14	2649.92±82202.07	6.20±84.78	33.74±1020.01
15	517.33 ±14583.88	5.96±121.31	10.38±297.59

In all of the experiments, we use Equation 2.9 to estimate σ^2, the variance of the noise. Since we know the original noise level σ^2, we can calculate the noise estimate $\hat{\sigma}^2$

for each sample and we can compare them. Table 6.1 shows the averages and standard deviations of the calculated $\hat{\sigma}^2$ values for different polynomial degrees and for different sample sizes. We see from the table that

- For polynomial degrees 1, 2 and 3, since the calculation of the estimate includes bias, $\hat{\sigma}^2$ is larger than the original value.
- For polynomial degrees larger than 10, the calculation of the estimate includes large variance, therefore $\hat{\sigma}^2$ can be very large. Due to the same reasoning, for small sample sizes, the standard deviation of the estimate is also very large.
- For polynomial degrees 4–10, $2\hat{\sigma}^2$ is very close to the original value $\sigma^2 = 0.5$.

Table 6.2. Average and standard deviations of the test error rates of the optimal models selected by model selection techniques

Technique	Small Sample	Medium Sample	Large Sample
AIC	272.620±3971.748	0.281±0.100	0.250±0.011
BIC	55.055±694.705	0.274±0.090	0.251±0.011
MDL	0.678±0.643	0.312±0.047	0.252±0.011
SRM	0.963±8.268	0.274±0.088	0.252±0.011
CV	1.824±1.031	0.370±0.118	0.252±0.011

The second criterion for comparing the model selection methodologies is the validation error. Table 6.2 shows the average and standard deviations of the validation error rates of the optimal models selected by model selection techniques for three sample sizes. For small sample size, since AIC and BIC select more complex models, they have a large average error, BIC having smaller of the two. MDL has the lowest average validation error but the difference is not significant compared to SRM and MultiTest-based cross-validation. As the sample size increases, the techniques come closer. For the medium sample size, SRM and BIC have smallest average error rates, which is not significant compared to the other three model selection techniques. For the large sample size, there is not any difference between the five model selection techniques, as expected: Given enough training data, we do not need to rely on any a priori assumptions and therefore the assumptions we do make do not matter.

6.1.2. Classification

This section describes an empirical comparison of the classical model selection methods with our proposed MultiTest-based approach in the context of polynomial classification. First, we describe the experimental setup for comparisons, then we discuss the bias-variance dilemma for polynomial classification and finally we give the results of the comparison experiments.

6.1.2.1. Experimental Setup. In order to compare various model selection techniques on polynomial classification, we have generated random data (x^t, r^t), $(t = 1, \ldots, N)$ where random x^t values are uniformly distributed in [-1, 1] interval and r^t values are generated from the two-class Gaussian mixture model

$$\begin{aligned} p(x|C_0) &= 0.5\mathcal{N}(-0.58, 0.17) + 0.5\mathcal{N}(0.32, 0.13) \\ p(x|C_1) &= 0.6\mathcal{N}(-0.10, 0.15) + 0.4\mathcal{N}(0.65, 0.09) \end{aligned} \tag{6.5}$$

where $p(x|C_i)$ is class conditional density of class i. The posterior probabilities of each class is calculated as

$$\begin{aligned} P(C_0|x) &= \frac{p(C_0)p(x|C_0)}{P(C_0)p(x|C_0) + P(C_1)p(x|C_1)} \\ P(C_1|x) &= \frac{p(C_1)p(x|C_1)}{P(C_0)p(x|C_0) + P(C_1)p(x|C_1)} \end{aligned} \tag{6.6}$$

where $P(C_i)$ denotes the prior probability of class i.

The mixture model and the posterior probabilities of the classes are shown in Figure 6.10 ($P(C_0) = P(C_1)$). Example dataset generated from the mixture model is shown in Figure 6.11. Three different sample sizes were used for training data: small ($N = 25$), medium ($N = 100$) and large ($N = 1000$). The validation set contains $N = 1000$ instances.

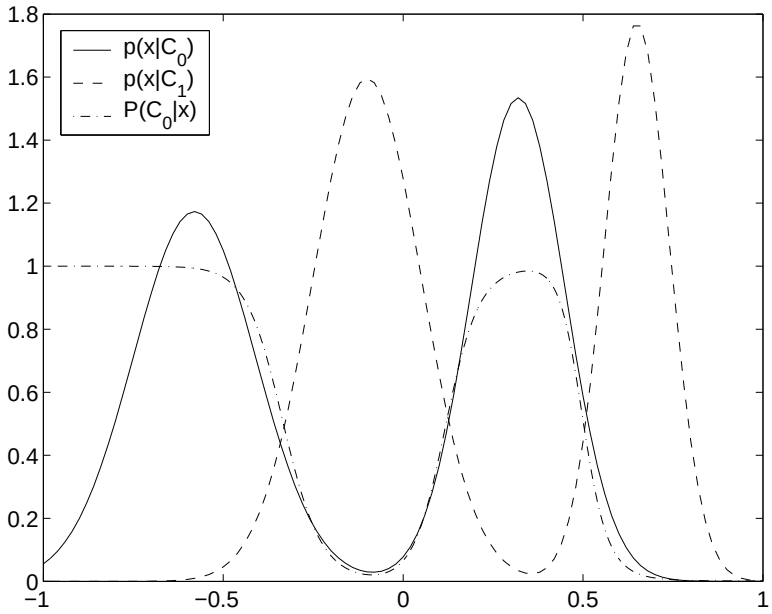


Figure 6.10. Mixture model $f(x)$ and the posterior probabilities of each class are shown

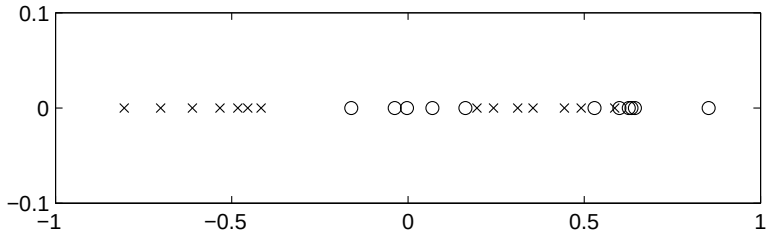


Figure 6.11. Example dataset generated from the mixture model

The discriminants are linear estimators with polynomial basis functions

$$g(x) = \sum_{i=1}^{m-1} w_i x^i + w_0 \tag{6.7}$$

where the class assigned to an instance x^t is

$$y^t = \begin{cases} 0, & \text{if } g(x) < 0 \\ 1, & \text{if } g(x) \geq 0 \end{cases} \tag{6.8}$$

We also estimate the posterior probability of class i by substituting $g(x)$ values into

the sigmoid function

$$\hat{P}(C_0|x) = \frac{1}{1+e^{-g(x)}}$$
$$\hat{P}(C_1|x) = 1 - \frac{1}{1+e^{-g(x)}} \tag{6.9}$$

We consider polynomials of degree $m = 0$ - 10 as possible candidates for model selection. The parameters of the polynomial $(w_i, i = 0, \ldots, m-1)$ are estimated by gradient-descent. We have used 0-1 loss function to calculate the training and validation errors.

6.1.2.2. Bias-Variance Dilemma. To show the bias variance dilemma in polynomial classification, we have generated M small size training sets ($M = 100$) from the mixture model $f(x)$. For each dataset i, we form the polynomial estimator $g_i(x)$ and the average of M estimators $\overline{g}(x)$

$$\overline{g}(x) = \frac{1}{M}\sum_{i=1}^{M} g_i(x) \tag{6.10}$$

Estimated bias and variance are calculated using posterior probabilities on a separate large size test set having N instances ($N = 1000$)

$$Bias = \frac{1}{N}\sum_{t=1}^{N}[\overline{\hat{P}}(C|x) - P(C|x)]^2$$
$$Variance = \frac{1}{NM}\sum_{t=1}^{N}\sum_{i=1}^{M}[\hat{P}_i(C|x) - \overline{\hat{P}}(C|x)]^2 \tag{6.11}$$

where $P(C|x)$ is the posterior probability calculated from the original mixture model, $\overline{\hat{P}}(C|x)$ is the posterior probability calculated from the average of the estimators $\overline{g}(x)$ and $\hat{P}_i(C|x)$ is the posterior probability calculated from the polynomial estimator $g_i(x)$ for the dataset i.

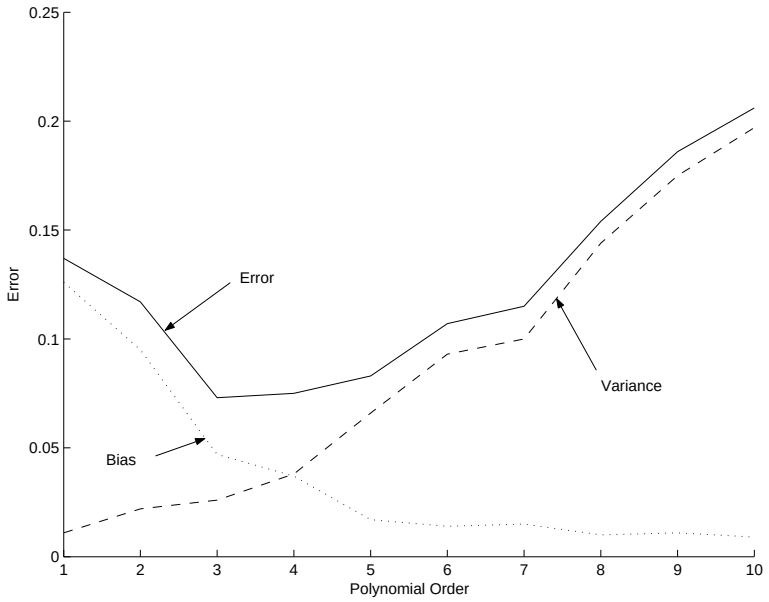
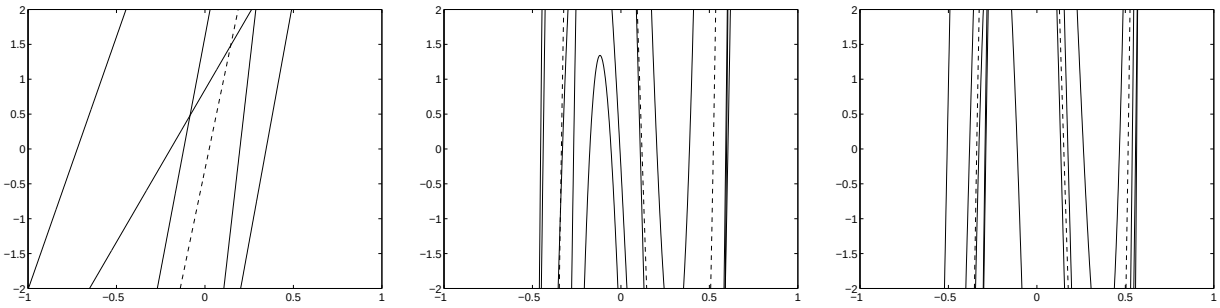


Figure 6.12. Bias, variance and error values for polynomials of order 1 to 10

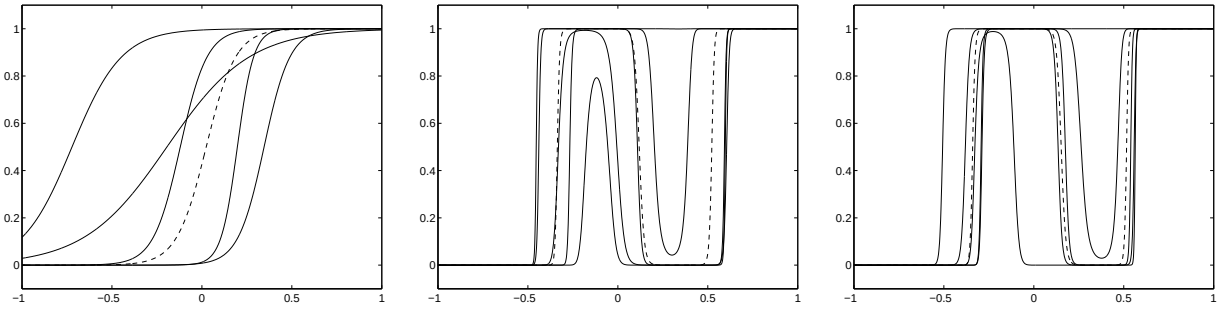
Figure 6.12 shows the bias, variance and error values for polynomials of order 1 to 10. According to the figure, the bias decreases and the variance increases as the order of the polynomial increases. Order 10 has the smallest bias, order 1 has the smallest variance and order 3 has the minimum error (optimal model complexity).



(a) Order 1 (b) Order 3 (c) Order 5

Figure 6.13. Five polynomial fits are shown as solid lines, $\overline{g}(x)$ is shown as dotted line

For visualization purposes, we took five samples having 25 instances and plotted the corresponding five polynomial fits ($g_i(x)$, $i = 1, \ldots, 5$), the average of them ($\overline{g}(x)$) (Figure 6.13), and their sigmoid counterparts (Figure 6.14) of orders 1, 3 and 5 respectively. Again we see that as the order of polynomials increases, the average of the polynomials gets closer to the original function (bias decreases), whereas the



(a) Order 1 (b) Order 3 (c) Order 5

Figure 6.14. Posterior probabilities of five polynomial fits are shown as solid lines, posterior probability of $\overline{g}(x)$ is shown as dotted line

polynomial fits around the average fluctuates more (variance increases).

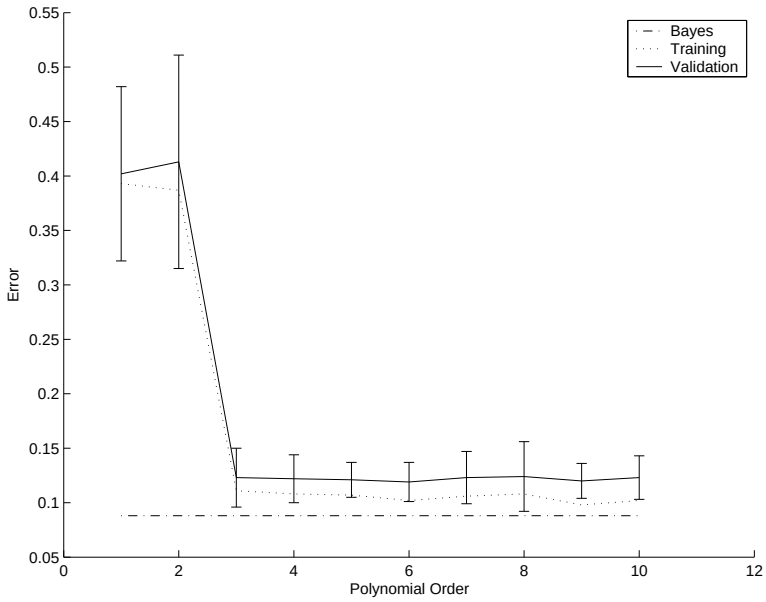


Figure 6.15. Training and validation errors of 100 random samples without noise. The average of training error, and the average and standard deviations of the validation error are shown

In order to get the optimal model complexity, we have generated 100 random samples from the original model and fit polynomials of order 1 to 10. The performance of the polynomials are then tested on 100 validation datasets and the average and standard deviations of the validation errors are found (See Figure 6.15). According to one standard deviation rule, we first choose the model with the smallest validation

error, which is 6. After that, we look if there is a simpler model that has validation error no more than one standard deviation of the model with degree 6, which is 3. Therefore the third degree polynomial is the optimal model for this problem.

6.1.2.3. Comparison of Model Selection Techniques. In this section, we give the empirical comparison of the five different model selection techniques, namely AIC, BIC, MDL, SRM and our proposed MultiTest-based cross-validation on polynomial classification problem. The comparison is done for three different sample sizes (small, medium and large), and two different criteria are used for comparison which are the complexity of the optimal model and the error of the model on the validation set.

```
1  BestDegree ModelSelectionTechniquePolyCls(maxd, t, N)
2      for i = 0 to maxd
3          Fit polynomial of order i using gradient-descent
4          Calculate E_t^i /*training error*/ or L_t^i /*loglikelihood*/
5          h = i + 1 /*complexity measure*/
6          switch (t)
7              AIC: E_g^i = -L_t^i + h
8              BIC: E_g^i = -L_t^i + h/2 log N
9              MDL: E_g^i = E_t^i + λh
10             SRM: E_g^i = E_t^i + ε/2 (1 + sqrt(1 + 4E_t^i/ε)) where ε = a_1 (h[log(a_2 N/h)+1] - log(ν))/N
11     l = ∀i, E_g^l ≤ E_g^i
12     return l;
```

Figure 6.16. Pseudocode of AIC, BIC, MDL and SRM model selection techniques for polynomial classification problem. $maxd$ is the maximum model complexity, t is the name of the model selection technique and N is the number of training instances

The penalized model selection techniques and SRM consist of four main steps (See Figure 6.16). In the first step, they fit polynomials with different model complexities using gradient-descent to the training set (Line 3). In the second step, they calculate the training error or likelihoods of each model (Line 4). In the third step, they calculate

the generalization error of each model (Lines 5–10). In the last step, they choose the optimal model having the least estimated generalization error E_g (Line 11).

```
1  BestDegree MultiTestBasedModelSelectionPolyCls(maxd, N)
2      for i = 0 to maxd
3          Generate k training and validation sets
4          for j = 1 to k
5              Fit polynomial of order i using gradient descent
6              Calculate E_i^j, error of model i on validation set j
7      if small sample size
8          t = Binomial Test
9      else
10         t = Combined 5×2 t test
11     l = MultiTest(0 ... maxd, t, α)
12     return l;
```

Figure 6.17. Pseudocode of MultiTest-based model selection for polynomial classification problem. $maxd$ is the maximum model complexity and N is the number of training instances

The MultiTest-based model selection technique consist of three main steps (See Figure 6.17). In the first step, we generate k training and validation sets for cross-validation (Line 3). For small sample sizes, we use leave-one-out type cross-validation ($k = N$), for medium and large sample sizes, we use 5×2 cross-validation ($k = 10$). In the second step, for each training set, we fit polynomials of degree i using gradient-descent and calculate the validation error (Lines 4–6). In the last step, we find the optimal model by the MultiTest algorithm using the appropriate statistical test (Lines 7–11). Since there are not enough samples to make 5×2 cross-validation for small sample sizes, for this setting leave-one-out validation errors are compared with Binomial test (Section B.3.6). Since there is not such a problem for the medium and large sample, we have used our proposed combined 5×2 cv t test in those settings.

The performances of different model selection techniques are affected by the train-

ing sample. Therefore, to create a valid comparison between the model selection techniques, the model selection experiment is repeated with 1,000 different training and validation samples. And at each iteration, the same training and validation sample pair is used for all model selection techniques. Figures 6.18, 6.19 and 6.20 show the frequency with which the polynomials of each degree is selected as the best model by the five different model selection techniques for small, medium and large sample sizes respectively.

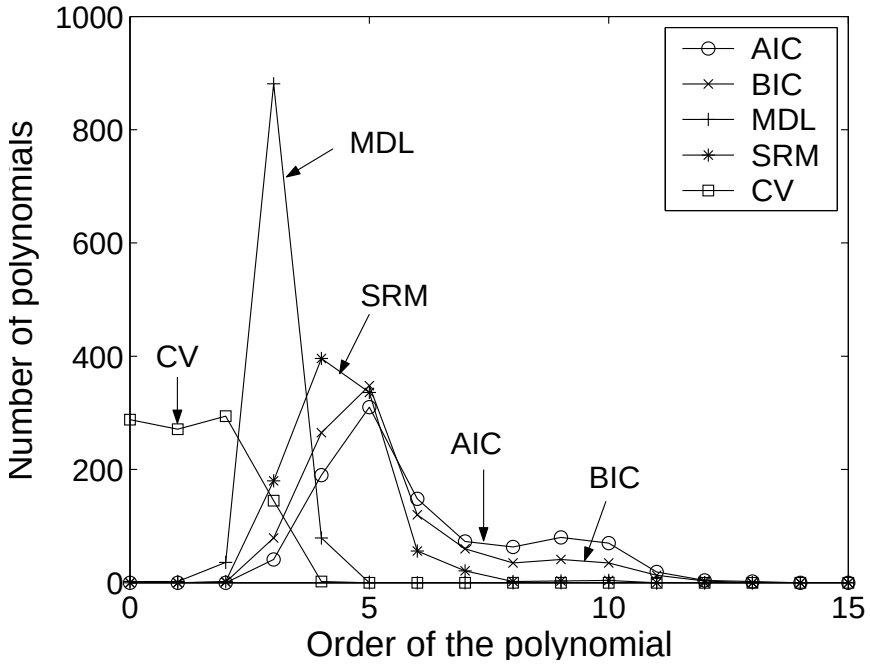


Figure 6.18. Comparison of model selection techniques for small sample size in terms of model complexities

For small and medium sample sizes, all model selection techniques select the third degree polynomial model as the optimal model. For large sample sizes, AIC and BIC select the sixth degree polynomial, whereas MDL, SRM and our proposed MultiTest-based cross-validation technique select the third degree polynomial as the optimal model mostly. Again due to the $\log N$ term in its formula, BIC punishes complex models more heavily than AIC does, and AIC usually selects more complex models than BIC. Both criteria have larger variance than the other three model selection methodologies.

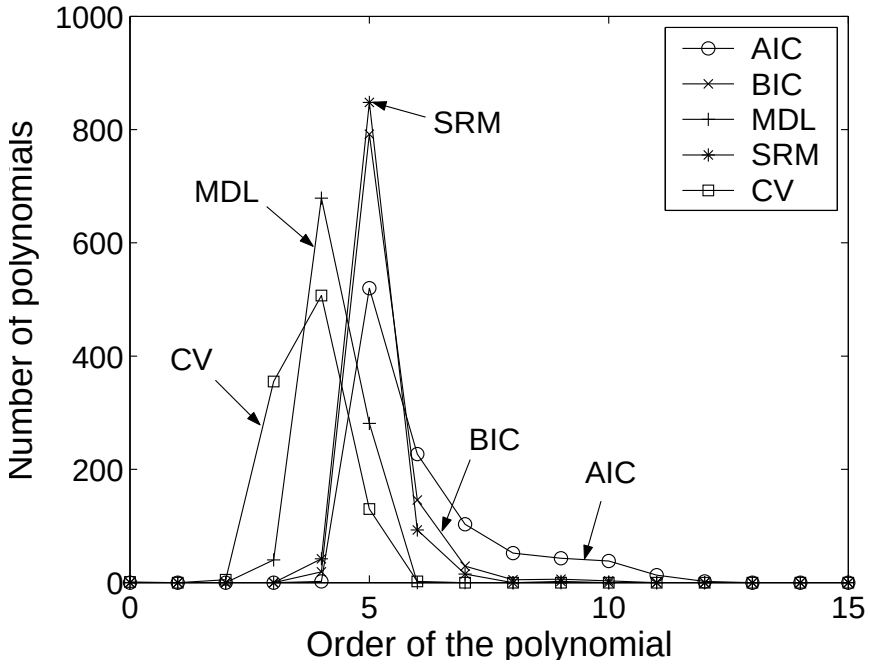


Figure 6.19. Comparison of model selection techniques for medium sample size in terms of model complexities

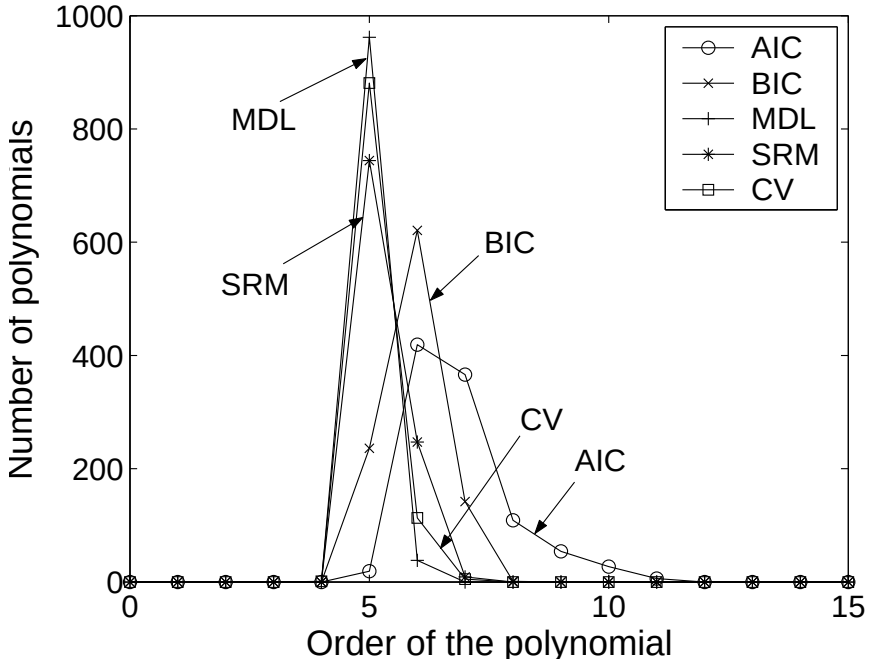


Figure 6.20. Comparison of model selection techniques for large sample size in terms of model complexities

MDL, SRM and our proposed MultiTest-based cross-validation technique results are nearly the same for all sample sizes except for large sample sizes. For large sample sizes SRM selects more complex models.

Table 6.3. Average and standard deviations of the validation error rates of the optimal models selected by model selection techniques

Technique	Small Sample	Medium Sample	Large Sample
AIC	0.153±0.052	0.127±0.028	0.120±0.022
BIC	0.155±0.062	0.126±0.027	0.121±0.022
MDL	0.189±0.107	0.138±0.065	0.125±0.037
SRM	0.182±0.105	0.127±0.029	0.123±0.027
CV	0.231±0.138	0.171±0.109	0.126±0.038

The second criterion for comparing model selection methodologies is the validation error rate. Table 6.3 shows the average and standard deviations of the validation error rates of the optimal models selected by model selection techniques for three sample sizes. For all sample sizes, models selected by AIC and BIC have nearly the same expected error rate. Like in polynomial regression, for large sample size, there is not any difference between five model selection techniques. Although our proposed MultiTest based cross-validation technique has larger average expected error rate than the other four techniques, the difference is not significant. Therefore we can conclude that, the Multitest-based cross-validation technique selects simpler models without sacrificing from expected error.

6.2. MultiTest Algorithm

6.2.1. Comparing MultiTest with Anova, Newman-Keuls, and TestFirst

In comparing with the results of MultiTest, we should stress that Anova and Newman-Keuls only check for equality of means and do not provide ordering whereas the pairwise test used by MultiTest is for ordering. Analysis of variance (Anova) tests whether K samples are drawn from populations with the same mean. Newman-Keuls

checks for the equality of the means of subsets of populations. These methods are discussed in more detail in sections B.3.1 and B.3.5. The result of Anova can be used to choose the best one only when Anova accepts and we choose the simplest one. The result of Newman-Keuls can be used to find the best learner if one of the following conditions hold:

- The first one, namely the algorithm with the smallest average is not underlined. For example, if Newman-Keuls result is 3 $\underline{5\ 4}$ $\underline{2\ 1}$, the best can be taken as 3.
- There is a line under the first one and this line does not overlap with any other line(s). For example, if Newman-Keuls result is $\underline{5\ 4\ 3}$ 2 1, the best is 3 because it is simpler than 4 and 5.
- There is a line under the first one, and this line overlaps with one or more lines, but the overlap does not include the first one. For example, if Newman-Keuls result is $\underline{2\ 4}\ \underline{5\ 3}$ 1, the best is 2.
- If we have the case above and the overlap does not contain a simpler algorithm, the most simple is selected as the best. For example, if Newman-Keuls result is $\underline{5\ 2}\ \underline{4\ 3}$ 1, the best is 2.

If neither of these four cases occurs, Newman-Keuls test result cannot yield the best. For example, if Newman-Keuls result is $\underline{5\ 4}\ \underline{2\ 1}$ 3, the first underline chooses 2, the second underline chooses 1 which is simpler than 2. But we cannot choose 1 as it has higher expected error than 5. These indicate that Anova and Newman-Keuls tests should be extended considerably to be able to generate orderings and in certain cases they may be unable to generate an ordering. This is expected because those two tests, unlike MultiTest, are not designed to generate orderings but to find subsets of equality.

For comparison, we also propose a simple algorithm which is what one would normally use to find the best. This *TestFirst* algorithm sorts the learners in increasing order of average error. Then TestFirst *tests* the *first* algorithm, the algorithm with the smallest average error, to be the best one by comparing it with the other $K-1$ algorithms using a one-sided pairwise test and accepts it if (i) it has less expected error, or (ii) it has equal expected error and is simpler. Otherwise, the TestFirst algorithm

can not find the best one. For example, let us say we have ordered the algorithms in terms of average error as $3 < 4 < 1 < 2 < 5$. TestFirst will try to select 3 as the best. Since 3 is simpler than 4 and 5 the first condition holds for those two. Since 3 is more complex than 1 and 2, we check if 3 has significantly smaller expected error than 1 and 2. If it has, 3 will be selected as the best, otherwise we say that TestFirst can not find the best one.

6.2.2. Finding the Best of $K > 2$ Classification Algorithms

The five classification algorithms we use in decreasing prior preference because of their increasing time/space complexity are:

1. MAX decides based on the prior class probability without looking at the input.
2. MEAN keeps the mean vector of the features for each class and chooses the class with the nearest mean using Euclidean distance.
3. LDA is the well-known linear classification algorithm.
4. C4.5 is the archetypal decision tree method.
5. NN is the nearest-neighbor classification algorithm.

These classification algorithms are tested on thirty datasets from the UCI machine learning repository (Table D.1) [94].

6.2.2.1. Pairwise Test Results. Before applying our MultiTest algorithm, we compare the two statistical tests, the 5×2 cv t test and our proposed combined 5×2 cv t test to find the most suitable one to use in MultiTest. Towards this aim, we look at the performance of the tests in comparing the expected error of two different algorithms. We have five different algorithms, therefore we can make ten different pairwise comparisons (MAX vs MEAN, MAX vs LDA, ..., C4.5 vs NN). For each pair, we test the one sided hypothesis $H_0 : \mu_1 \leq \mu_2$, where μ_1 and μ_2 are the expected error of the first and second algorithms respectively. This test is done on thirty datasets where for each dataset the rejection probability is calculated on 1,000 runs (of 5×2 folds). As in [43],

to compare the type I and II errors of the statistical tests, for each dataset and for each algorithm pair, we calculate the normalized distance between the expected error of the two algorithms:

$$z = \frac{m_1 - m_2}{s_2} \tag{6.12}$$

where m_1 and m_2 are the average expected error of the first and second algorithms and s_2 denotes the standard deviation of the error values of the complex algorithm on 1,000 runs. A small difference in expected error result in a smaller z measure, which implies we have similar algorithms and a rejection would be a type I error. On the other hand, a large z measure denotes large difference between expected error and we expect larger rejection rates for a test to have lower type II error (and higher power). Since we have thirty datasets and ten classifier pairs, we have 300 different z value and rejection probability pair for the two tests and we plot these in Figure 6.2.2. For visualization purposes, we have also fitted a function to the data of both tests. We see from the figure that for $z > 1.6$, the 5×2 cv t test rejects only 20 percent of the cases whereas the combined 5×2 cv t test rejects almost 99 percent of the cases, indicating that the combined test has higher power (lower type II error). Similarly when $z < 0$, the combined test has a lower probability of reject indicating that it has lower type I error.

6.2.2.2. Results on All Datasets. We report the average and standard deviations of validation errors and the frequency of best classifiers found by MultiTest over 1,000 runs. The overall α is taken as 0.05 and Bonferroni correction is used as K is small ($K = 5$). The full results on all thirty datasets are given in Tables 6.4 and 6.5. In Table 6.4, we report the average and standard deviations of the misclassification error rates of the five classifiers on thirty datasets in 1,000 runs. In Table 6.5, we give the frequencies of best algorithms found by TestFirst, Newman-Keuls and MultiTest over these runs.

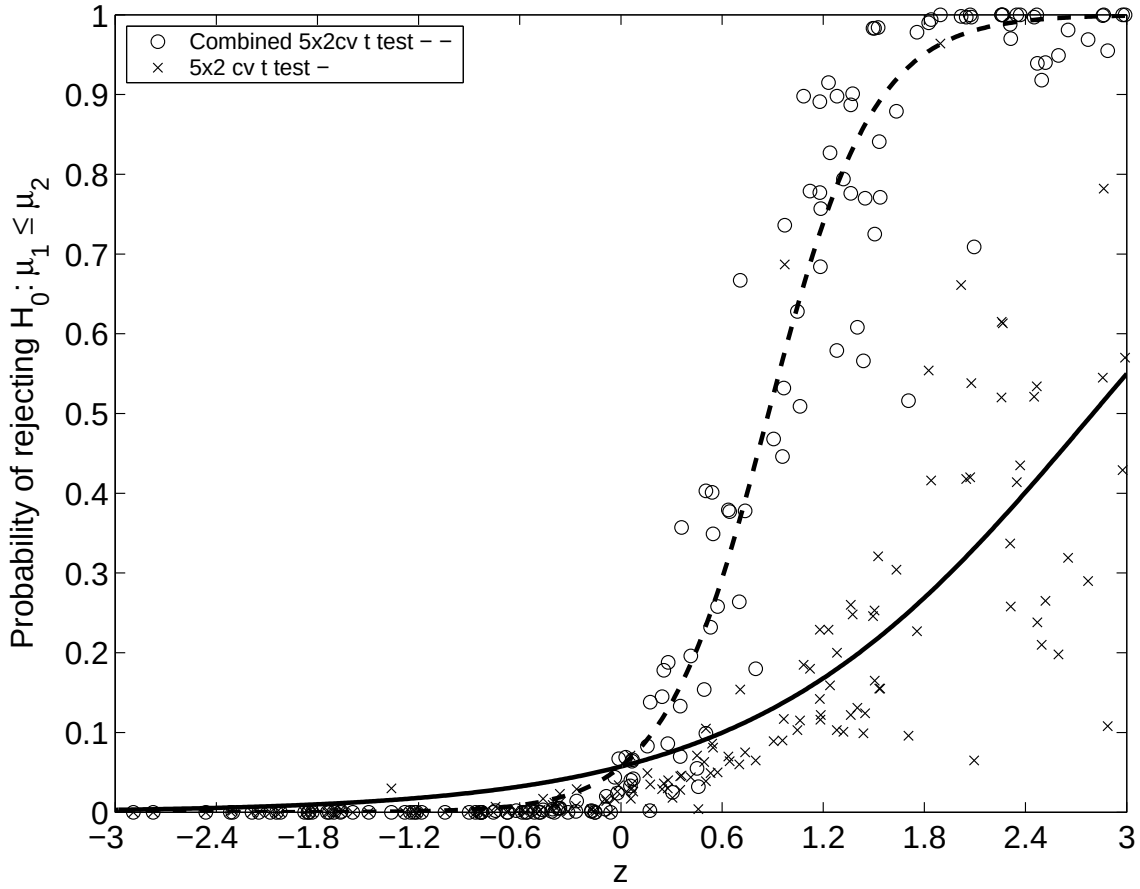


Figure 6.21. Comparison of type I and II errors of the combined 5×2 cv t and the 5×2 cv t tests on classification problems. x axis is $z = (m_1 - m_2)/s_2$, y axis is the rejection probability of $H_0 : \mu_1 \leq \mu_2$. For increasing z, the combined t test has a higher probability of rejecting indicating that it has higher power (lower type II error); similarly for $z < 0$, its probability of rejecting is lower indicating that is has lower type I error

Table 6.4. Average and standard deviations of the misclassification error rates of five classification algorithms on 5× 2 fold in 1,000 runs

Dataset	MAX (1)	MEAN (2)	LDA (3)	C4.5 (4)	NN (5)
balance	55.46±1.29	54.01±2.88	24.60±2.64	40.26±7.53	40.10±2.81
breast	34.48±1.81	3.80 ±0.86	5.75 ±0.97	6.88 ±1.69	4.72 ±0.88
bupa	42.09±2.84	42.83±4.23	44.66±4.64	40.15±4.66	39.63±3.34
car	29.98±1.12	47.77±2.25	31.64±1.29	15.55±1.95	21.82±1.36
cmc	57.30±1.27	57.65±2.02	55.11±1.66	53.22±3.46	55.72±1.51
credit	44.55±2.08	19.16±1.85	27.03±2.72	14.85±1.87	22.68±1.88
cylinder	42.22±2.13	38.75±2.93	36.40±3.32	33.32±4.84	26.68±2.40
dermatology	69.43±2.50	4.23 ±1.30	5.48 ±1.66	8.42 ±3.55	6.00 ±1.42
ecoli	57.44±2.67	21.28±4.07	20.78±8.46	23.65±4.61	20.01±2.53
flare	11.15±1.76	34.30±7.64	21.22±4.93	11.26±1.80	17.06±2.70
glass	67.51±3.14	54.35±5.83	42.29±4.92	42.41±8.77	32.45±3.99
haberman	26.47±2.54	28.28±3.80	25.54±2.58	26.90±2.80	33.77±2.96
hepatitis	20.65±3.22	19.03±3.51	20.01±3.67	21.24±3.91	18.78±3.51
horse	36.96±2.54	24.98±2.92	24.27±2.89	23.11±6.67	21.96±2.55
iris	70.68±2.25	13.59±3.53	2.55 ±1.52	6.29 ±4.47	6.51 ±2.38
ironosphere	35.90±2.56	20.16±5.26	13.72±2.23	13.59±4.53	14.77±2.57
monks	51.93±1.47	33.82±2.27	33.88±2.40	14.99±8.34	26.41±4.96
mushroom	48.21±0.57	12.64±0.44	10.51±0.43	0.11 ±0.13	0.00 ±0.01
nursery	67.02±0.44	62.90±2.71	61.36±22.17	6.54 ±0.48	23.86±0.55
optdigits	90.71±0.34	9.47 ±0.57	16.27±0.82	15.92±1.27	2.94 ±0.35
pendigits	90.05±0.25	15.70±0.49	27.58±0.55	7.45 ±0.73	0.74 ±0.13
pima	34.90±1.72	26.82±1.88	23.34±1.68	29.68±4.24	30.37±1.91
segment	86.77±0.39	15.66±0.97	14.63±0.95	8.00 ±1.22	5.07 ±0.66
spambase	39.40±0.71	10.56±0.61	11.18±0.72	9.89 ±1.04	10.35±0.61
tictactoe	34.66±1.54	32.87±3.31	30.22±1.89	23.51±3.27	31.57±1.63
vote	38.62±2.32	12.69±1.86	15.95±2.05	4.53 ±1.12	7.30 ±1.59
wave	66.91±0.82	18.94±0.61	14.38±0.60	25.62±1.39	23.84±0.70
wine	62.94±5.02	3.42 ±1.62	3.59 ±1.89	13.93±6.08	5.40 ±2.19
yeast	69.53±1.59	48.63±1.75	42.49±6.31	48.79±4.63	49.19±1.50
zoo	59.47±4.99	7.98 ±4.14	13.78±4.72	19.17±9.23	7.70 ±4.42

Table 6.5. The frequency of best classification algorithms found by TestFirst, Newman-Keuls and MultiTest over 1,000 runs

Dataset	Test	Not Found	MAX	MEAN	LDA	C4.5	NN
balance	TestFirst	0	0	0	1	0	0
	Newman-Keuls	0	0	0	1	0	0
	MultiTest		0	0	1	0	0
breast	TestFirst	0.001	0	0.999	0	0	0
	Newman-Keuls	0	0	1	0	0	0
	MultiTest		0	1	0	0	0
bupa	TestFirst	0.984	0.001	0	0	0.01	0.005
	Newman-Keuls	0.029	0.904	0	0	0.045	0.022
	MultiTest		0.917	0.066	0.003	0.013	0.001
car	TestFirst	0	0	0	0	1	0
	Newman-Keuls	0	0	0	0	1	0
	MultiTest		0	0	0	1	0
cmc	TestFirst	0.872	0	0	0.006	0.122	0
	Newman-Keuls	0.287	0.101	0	0.169	0.443	0
	MultiTest		0.499	0.188	0.261	0.052	0
credit	TestFirst	0.092	0	0	0	0.908	0
	Newman-Keuls	0	0	0.01	0	0.99	0
	MultiTest		0	0.282	0	0.718	0
cylinder	TestFirst	0.242	0	0	0	0	0.758
	Newman-Keuls	0	0	0	0	0.01	0.99
	MultiTest		0	0.001	0.022	0.487	0.49
dermatology	TestFirst	0.001	0	0.999	0	0	0
	Newman-Keuls	0	0	1	0	0	0
	MultiTest		0	1	0	0	0
ecoli	TestFirst	0.962	0	0.028	0.009	0	0.001
	Newman-Keuls	0	0	0.988	0.01	0	0.002
	MultiTest		0	0.987	0.013	0	0
flare	TestFirst	0.309	0.691	0	0	0	0
	Newman-Keuls	0	1	0	0	0	0
	MultiTest		1	0	0	0	0

Table 6.5. continued

Dataset	Test	Not Found	MAX	MEAN	LDA	C4.5	NN
glass	TestFirst	0.419	0	0	0	0	0.581
	Newman-Keuls	0.013	0	0	0.008	0.012	0.967
	MultiTest		0	0	0.097	0.586	0.317
haberman	TestFirst	0.978	0.006	0	0.016	0	0
	Newman-Keuls	0	1	0	0	0	0
	MultiTest		0.963	0.034	0.003	0	0
hepatitis	TestFirst	0.996	0	0.003	0	0	0.001
	Newman-Keuls	0.022	0.973	0	0	0	0.005
	MultiTest		0.974	0.026	0	0	0
horse	TestFirst	0.956	0	0	0	0.044	0
	Newman-Keuls	0.063	0	0.803	0	0.126	0.008
	MultiTest		0	0.827	0.115	0.058	0
iris	TestFirst	0	0	0	1	0	0
	Newman-Keuls	0	0	0.001	0.999	0	0
	MultiTest		0	0.004	0.996	0	0
ironosphere	TestFirst	0.802	0	0	0.182	0.016	0
	Newman-Keuls	0.003	0	0.023	0.958	0.016	0
	MultiTest		0	0.762	0.235	0.003	0
monks	TestFirst	0	0	0	0	1	0
	Newman-Keuls	0	0	0	0	1	0
	MultiTest		0	0.008	0.003	0.989	0
mushroom	TestFirst	0.69	0	0	0	0	0.31
	Newman-Keuls	0	0	0	0	1	0
	MultiTest		0	0	0	0.875	0.125
nursery	TestFirst	0	0	0	0	1	0
	Newman-Keuls	0	0	0	0	1	0
	MultiTest		0	0	0	1	0

Table 6.5. continued

Dataset	Test	Not Found	MAX	MEAN	LDA	C4.5	NN
pendigits	TestFirst	0	0	0	0	0	1
	Newman-Keuls	0	0	0	0	0	1
	MultiTest		0	0	0	0	1
pima	TestFirst	0.037	0	0	0.963	0	0
	Newman-Keuls	0	0	0.007	0.993	0	0
	MultiTest		0	0.183	0.817	0	0
segment	TestFirst	0.004	0	0	0	0	0.996
	Newman-Keuls	0	0	0	0	0	1
	MultiTest		0	0	0	0.05	0.95
spambase	TestFirst	0.874	0	0.005	0	0.121	0
	Newman-Keuls	0.001	0	0.637	0	0.362	0
	MultiTest		0	0.932	0.003	0.065	0
tictactoe	TestFirst	0.066	0	0	0	0.934	0
	Newman-Keuls	0	0	0	0	1	0
	MultiTest		0	0.075	0.144	0.781	0
vote	TestFirst	0	0	0	0	1	0
	Newman-Keuls	0	0	0	0	1	0
	MultiTest		0	0	0	1	0
wave	TestFirst	0	0	0	1	0	0
	Newman-Keuls	0	0	0	1	0	0
	MultiTest		0	0	1	0	0
wine	TestFirst	0.374	0	0.622	0.004	0	0
	Newman-Keuls	0	0	1	0	0	0
	MultiTest		0	0.999	0.001	0	0
yeast	TestFirst	0.26	0	0.004	0.736	0	0
	Newman-Keuls	0	0	0.264	0.736	0	0
	MultiTest		0	0.263	0.737	0	0
zoo	TestFirst	0.604	0	0.379	0	0	0.017
	Newman-Keuls	0	0	1	0	0	0
	MultiTest		0	0.989	0.003	0.002	0.006

6.2.2.3. Sample Datasets. Below, we discuss results on some example datasets in more detail:

Hepatitis Dataset. On *hepatitis*, all tests says that all five algorithms have the same expected error (Figure 6.22). Anova accepts; Newman-Keuls underlines all five. With MultiTest, all pairwise tests are accepted and in the absence of any other information, the classification algorithms are sorted in terms of prior preference. TestFirst can not find the best algorithm because 5 (NN) has the smallest average error but it is not the simplest. *Haberman, flare* are other similar datasets where all algorithms have the same expected error.

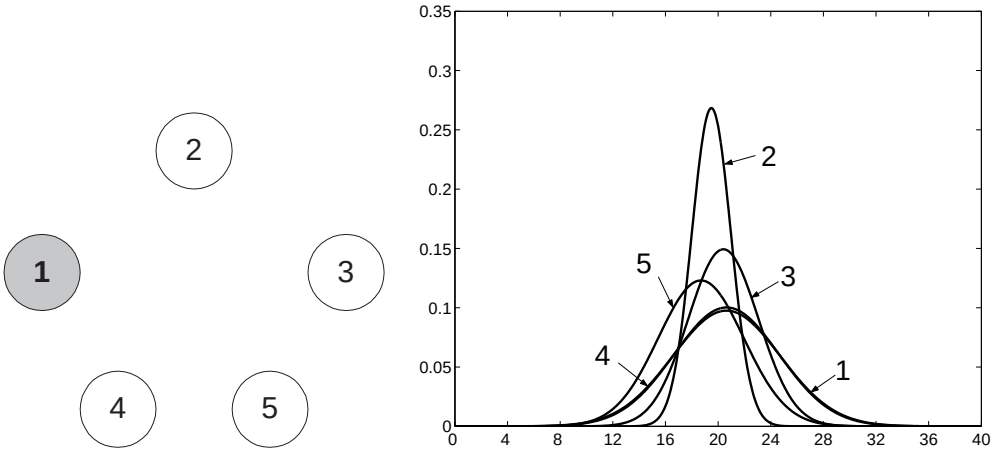


Figure 6.22. Results on *hepatitis*. The most frequently occurring graph and the corresponding error distributions of the classifiers are shown

Pendigits Dataset. On *pendigits*, as shown in Figure 6.23, there is an exact ordering of algorithms. Anova of course rejects. Newman-Keuls does not underline any algorithm: $5 < 4 < 2 < 3 < 1$. With MultiTest, all pairwise tests except $H_0 : \mu_2 \leq \mu_3$ are rejected. The ordering therefore is $5 < 4 < 2 < 3 < 1$ and 5 is chosen. TestFirst also selects 5 as the best algorithm. *Balance, breast, car, dermatology, iris, monks, nursery, optdigits, segment, vote, wave, wine, zoo* are other datasets where one algorithm is significantly better than the other four algorithms.

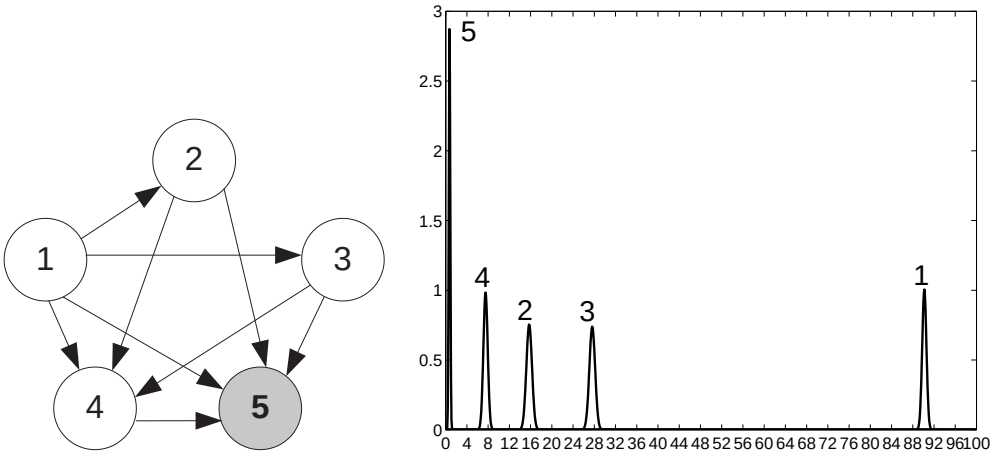


Figure 6.23. Results on *pendigits*

Mushroom Dataset. On *mushroom*, in all 1,000 cases, Newman-Keuls reports equality between algorithms 4 and 5, which are different from the others: 5 4 3 2 1 (Figure 6.24). Since there is only a single line under 4, it is chosen as the best by Newman-Keuls. Other algorithms are also reported different from each other. Because there is difference between their expected error, Anova never accepts. With MultiTest, in all 1,000 cases, all pairwise tests are rejected, except when comparing 4 with 5; therefore MultiTest places 4 in front of 5 because of prior preference and the other three algorithms are behind these two algorithms in the ordering because of the pairwise tests, in the opposite order in which they are preferred. TestFirst can assign 5 as the best classifier when the pairwise test between 4 and 5 rejects which only occurs 30 percent of the time. *Credit, ecoli, pima, spambase, tictactoe, yeast* are other datasets where two algorithms have nearly the same expected error.

Cmc Dataset. On *cmc*, during different runs because of slight differences in folds, the expected error and the decisions of tests vary in different runs and many outcomes occur in the 1,000 runs (Figure 6.25). Anova finds all algorithms equal in 3 percent of the cases. Newman-Keuls finds the best algorithm 1 in 10 percent of the cases when all algorithms are equal 4 3 5 1 2 (7 percent) or all algorithms are equal except 2 (4 3 5 1 2, 4 3 5 1 2, 4 3 5 1 2) (3 percent). Newman-Keuls chooses 3 as the best (17 percent) when 3, 4 and 5 are equal but others are different (4 3 5 1 2,

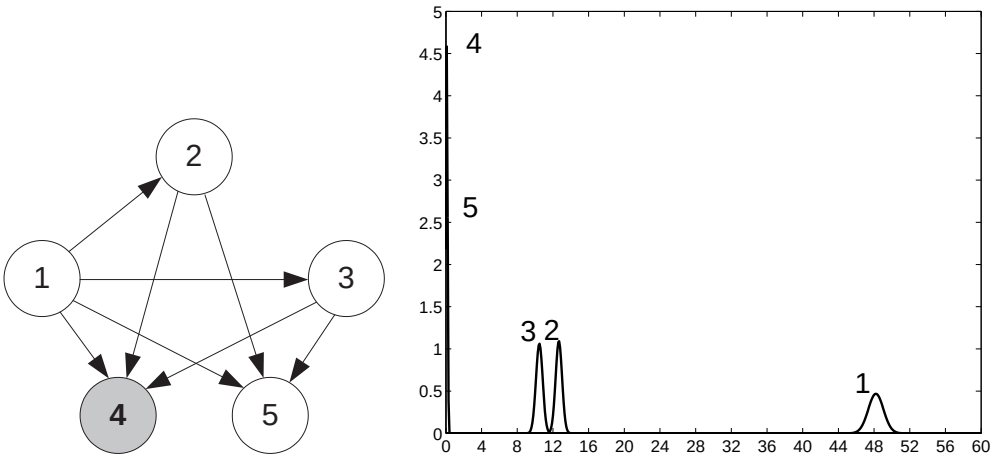


Figure 6.24. Results on *mushroom*

4 3 5 1 2) (10 percent) or when 3 is equal to 4 and 5 but not equal to others (4 3 5 1 2, 4 3 5 1 2, 4 3 5 1 2) (6 percent). Newman-Keuls chooses 4 as the best (44 percent) when 4 is significantly different than the other algorithms (4 3 5 1 2, 4 3 5 1 2, 4 3 5 1 2, 4 3 5 1 2). In 29 percent of the cases, Newman-Keuls can not find the best.

MultiTest does not have such a problem and it selects 1 as the best algorithm in 50 percent of the cases when all algorithms are equal (30 percent) or when 1 is not significantly worse than any other algorithm but there are significant differences between other algorithms. MultiTest finds 2 as the best in 19 percent of the cases when 1 is significantly worse than 3, 4, or 5, but 2 is not. MultiTest finds 3 as the best in 26 percent of the cases, when both of 1 and 2 are significantly worse than 3, 4, or 5, but 3 is not. MultiTest finds 4 as the best in five percent of the cases when there is no significant difference between 4 and 5 but when the other three algorithms are significantly worse than 4 and/or 5. *Bupa, cylinder, glass, horse, ironosphere* are other datasets where the expected error of the algorithms are not equal but near to each other. For this reason, the best algorithm could be one of these algorithms and Newman-Keuls sometimes can not find the best algorithm.

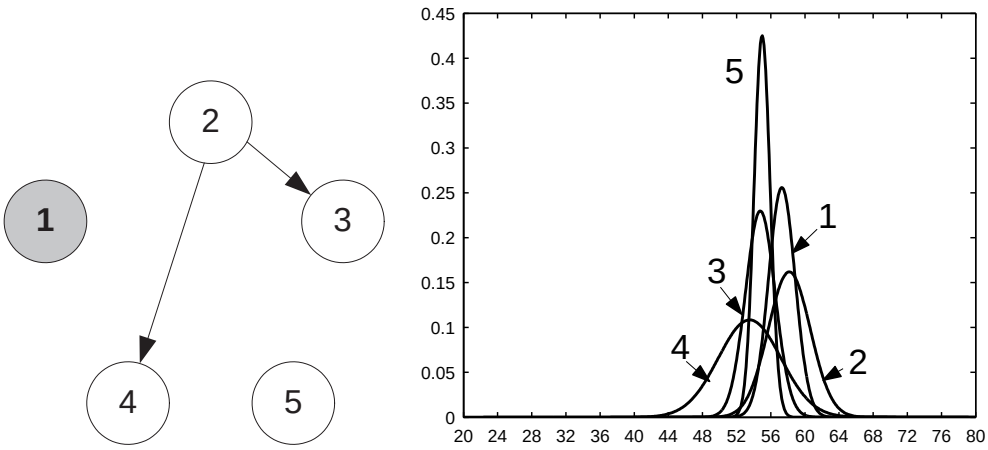


Figure 6.25. Results on *cmc*

6.2.3. Finding the Best of $K > 2$ Regression Algorithms

The five known regression algorithms we use in decreasing prior preference because of their increasing time/space complexity are:

1. AVE finds the average of the output values of the training data. It uses this value as the estimated output value for all test cases.
2. ONE finds the best axis-aligned split $x_i < c$. Using that split, the training instances are divided into two parts. For each part, the estimated output value is calculated by taking the average of the output values. This is a regression stump with only one node and two leaves.
3. LIN is the classic linear regression algorithm.
4. RTR is the well-known regression tree algorithm.
5. ROG divides each input feature into ten equal-sized bins to generate hyperrectangles in the instance space and calculates the average output in each bin. To combat the curse of dimensionality, Principal Component Analysis (PCA) is used to decrease dimensionality to two and regressogram is applied in this two dimensional space.

These regression algorithms are tested on eleven datasets from the Delve machine learning repository (Table D.2) [95].

6.2.3.1. Pairwise Test Results. First, we look at the behavior of the two statistical tests namely, the 5×2 cv t test and our proposed combined 5×2 cv t test, on regression problems. We have five different regression algorithms (AVE, ONE, LIN, RTR, ROG in increasing order of complexity) and eleven datasets, where for each dataset the rejection probability is calculated on 1,000 runs (of 5×2 folds). To compare the type I and II errors of the statistical tests, for each dataset and for each algorithm pair, we again use z, the normalized distance between the expected error values of the two algorithms.

We plot the rejection probability for the two tests in Figure 6.26. For visualization purposes, we have also fitted a function to the data of both tests. We see that for $z > 1.2$, the 5×2 cv t test rejects only 20 percent of the cases whereas the combined 5×2 cv t test rejects almost 99 percent of the cases indicating that the combined test has higher power (and lower type II error). Similarly when $z < 0$, the combined test has a lower probability of reject indicating that it has less type I error. We therefore see that the combined 5×2 cv t test is a suitable test also usable in regression problems.

6.2.3.2. Results on All Datasets. We report the average and standard deviations of validation errors and the frequency of algorithms found by MultiTest over 1,000 runs. The overall α is taken as 0.05 and Bonferroni correction is used as K is small ($K = 5$). The full results on all eleven datasets are given in the Tables 6.6 and 6.7. In Table 6.6, we report the average and standard deviations of the mean square errors of the five regression algorithms on eleven datasets in 1,000 runs. In Table 6.7, we give the frequencies of best algorithms found by TestFirst, Newman-Keuls and MultiTest over these runs.

6.2.3.3. Sample Datasets. Below, we discuss results on some example datasets in more detail:

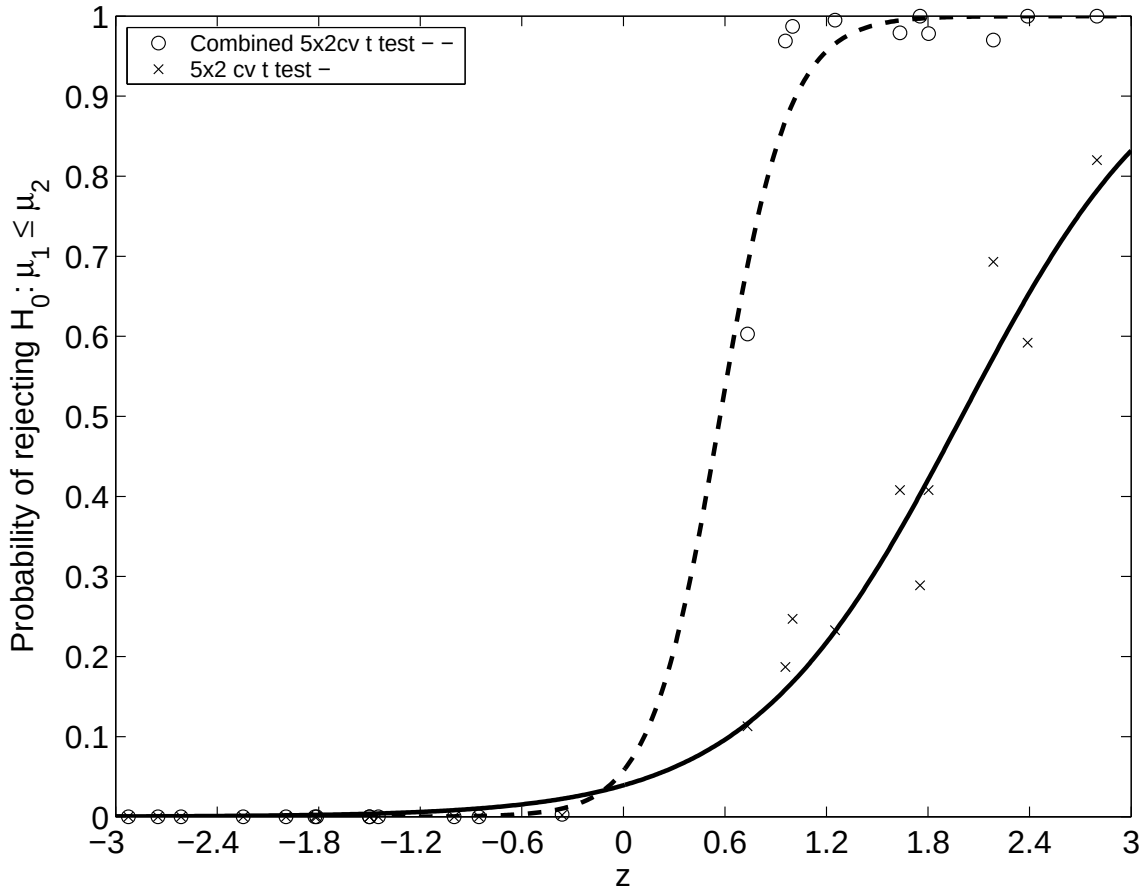


Figure 6.26. Comparison of type I and II errors of the combined 5×2 cv t and the 5×2 cv t tests on regression problems. x axis is $z = (m_1 - m_2)/s_2$, y axis is the rejection probability of $H_0 : \mu_1 \leq \mu_2$. As in Figure 6.2.2, we see that for $z > 1$, the combined test has a higher probability of rejecting indicating lower type II error and a lower probability of rejecting for $z < 0$, indicating lower type I error

Abalone Dataset. On *abalone*, as shown in Figure 6.27, there is an exact ordering of algorithms. Newman-Keuls does not underline any algorithms: 3 4 5 2 1. Anova of course rejects. With MultiTest, except $H_0 : \mu_3 \leq \mu_4$, $H_0 : \mu_3 \leq \mu_5$, and $H_0 : \mu_4 \leq \mu_5$ all pairwise tests are rejected. The ordering therefore is $3 < 4 < 5 < 2 < 1$ and 3 is chosen. TestFirst also selects 3 as the best algorithm. *Bank8fm, bank8nm, boston, kin8fm, kin8nm* are other datasets where 3 (LIN) has significantly less expected error than the other four algorithms. On *Comp, puma8nm, sine* the ordering is $4 < 3 < 5 < 2 < 1$ and 4 (RTR) is chosen. TestFirst also selects 4 as the best algorithm in those datasets.

Table 6.6. Average and standard deviations of the mean square errors of five regression algorithms on 5× 2 fold in 1,000 runs

Dataset	AVE (1)	ONE (2)	LIN (3)	RTR (4)	ROG (5)
abalone	10.4 ±0.34	7.54 ±0.27	5.04 ±0.20	5.99 ±0.36	6.60 ±0.34
add10	24.62 ±0.32	18.28 ±0.22	6.98 ±0.12	6.38 ±0.37	23.53 ±1.13
bank8fm	0.02 ±0	0.01 ±0	0 ±0	0 ±0	0.02 ±0
bank8nm	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0
boston	84.93 ±6.99	52.06 ±4.47	24.75 ±3.18	28.63 ±10.71	45.33 ±9.16
comp	338.71 ±14.52	77.08 ±1.99	94.89 ±6.09	12.57 ±1.52	247.92 ±12.46
kin8fm	0.01 ±0	0 ±0	0 ±0	0.00 ±0	0 ±0
kin8nm	0.07 ±0	0.05 ±0	0.04 ±0	0.04 ±0	0.07 ±0
puma8fm	21.50 ±0.21	10.68 ±0.14	1.66 ±0.03	1.55 ±0.06	14.76 ±2.82
puma8nm	29.88 ±0.31	19.05 ±0.25	14.31 ±0.24	2.30 ±0.68	22.59 ±4.16
sine	0.51 ±0.00	0.11 ±0.00	0.21 ±0.00	0.02 ±0	0.03 ±0

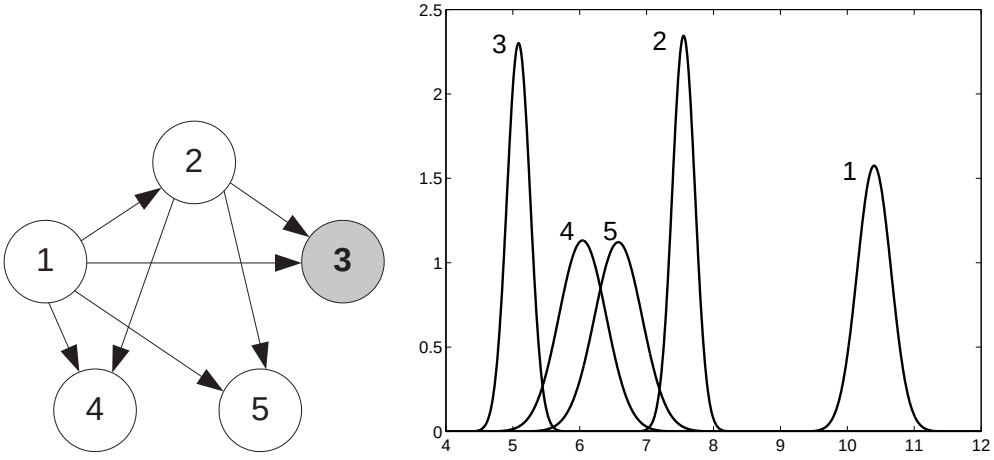


Figure 6.27. Results on *abalone*. The most occuring graph and the corresponding error distributions of the regressors are shown

Puma8fm Dataset. On *puma8fm*, Newman-Keuls reports equality between algorithms 3 and 4 which are different from the others: 3 4 2 5 1 (Figure 6.28). Since there is only a single line under 3, it is chosen as the best by Newman-Keuls. Other

Table 6.7. The frequency of best regression algorithms found by TestFirst, Newman-Keuls and MultiTest over 1,000 runs

Dataset	Test	Not Found	AVE	ONE	LIN	RTR	ROG
abalone	TestFirst	0	0	0	1	0	0
	Newman-Keuls	0	0	0	1	0	0
	MultiTest		0	0	1	0	0
add10	TestFirst	0.145	0	0	0	0.855	0
	Newman-Keuls	0	0	0	0.277	0.723	0
	MultiTest		0	0	0.294	0.706	0
bank8fm	TestFirst	0	0	0	1	0	0
	Newman-Keuls	0	0	0	1	0	0
	MultiTest		0	0	1	0	0
bank8nm	TestFirst	0.004	0	0	0.996	0	0
	Newman-Keuls	0	0	0	1	0	0
	MultiTest		0	0	1	0	0
boston	TestFirst	0.080	0	0	0.920	0	0
	Newman-Keuls	0	0	0	1	0	0
	MultiTest		0	0	1	0	0
comp	TestFirst	0	0	0	0	1	0
	Newman-Keuls	0	0	0	0	1	0
	MultiTest		0	0	0	1	0

algorithms are also reported different from each other. Since there are differences between the expected error values, Anova never accepts. With MultiTest, the pairwise test comparing 3 with 4 sometimes accepts and sometimes rejects. When the test rejects, which occurs 75 percent of the time, MultiTest selects 4 as the best algorithm. When the test accepts, MultiTest selects 3 as the best algorithm. TestFirst assigns 4 as the best regressor when the pairwise test between 3 and 4 rejects which occurs 88 percent of the time. *Add10* is another dataset where two algorithms have nearly the same expected error.

Table 6.7. continued

Dataset	Test	Not Found	AVE	ONE	LIN	RTR	ROG
kin8fm	TestFirst	0.005	0	0	0.995	0	0
	Newman-Keuls	0	0	0	1	0	0
	MultiTest		0	0	1	0	0
kin8nm	TestFirst	0	0	0	1	0	0
	Newman-Keuls	0	0	0	1	0	0
	MultiTest		0	0	1	0	0
puma8fm	TestFirst	0.133	0	0	0	0.877	0
	Newman-Keuls	0	0	0	1	0	0
	MultiTest		0	0	0.248	0.752	0
puma8nm	TestFirst	0	0	0	0	1	0
	Newman-Keuls	0	0	0	0	1	0
	MultiTest		0	0	0	1	0
sine	TestFirst	0	0	0	0	1	0
	Newman-Keuls	0	0	0	0	1	0
	MultiTest		0	0	0	1	0

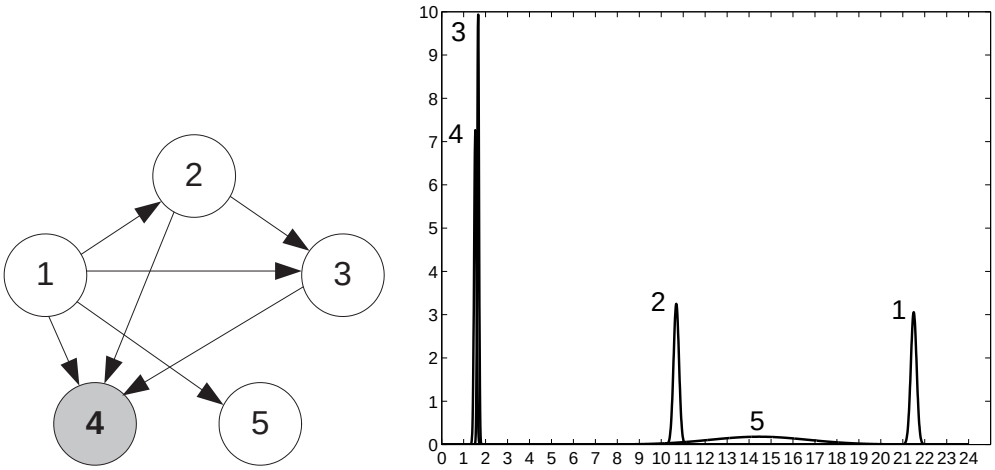


Figure 6.28. Results on *puma8fm*

These results on both classification and regression problems show that Anova test results can be converted to an ordering only if Anova test accepts; we order the algo-

rithms in terms of our prior preference. But this case occurs rarely, in our experiments, in only two datasets out of thirty. Converting Newman-Keuls results to an ordering is possible in some cases but not always. Note that we only compare five algorithms in this section. If there are more algorithms, we may expect to have more groups of algorithms having similar expected error and we may have more occurrences of this problematic second case for a range test like the Newman-Keuls where multiple underlines overlap. There are similar problems with a straightforward approach such as the heuristic TestFirst algorithm which is not always able to choose a single algorithm as the best. These results therefore show that a methodological approach as proposed by the MultiTest algorithm is necessary and that furthermore MultiTest is always able to find a best algorithm.

6.3. Decision Trees

In this section, we aim to compare our proposed MultiTest-based cross-validation technique with the other model selection techniques in the context of decision trees. First, we describe the experimental setup for comparisons, then we give our results.

6.3.1. Experimental Setup

Before giving the experimental setup, we first give the questions that motivate us. We make experiments to answer the following questions:

- Is model selection during decision tree induction justified, that is, does the omnivariate structure follows the best candidate model?
- Which model selection technique(s) is/are the best in terms of error, model complexity and learning time?
- Which type of pruning (prepruning or postpruning) is the best in terms of error and model complexity for these model selection techniques? Since postpruning uses validation for pruning, for model selection techniques other than cross-validation, such as AIC or BIC, we can have another pruning, where we prune according to the model selection technique used in tree construction, called

modelselection-pruning. For example, for AIC, if we want to prune a subtree, we calculate the generalization error of that subtree and the leaf which is replaced for that subtree and compare both generalization errors. If the generalization error of the leaf is smaller, we prune the subtree, otherwise we do not prune.

- What is the proportion of univariate, multivariate linear and multivariate quadratic nodes chosen with each model selection technique?
- Is the hypothesis "At each node of the tree, which corresponds to a different subproblem defined by the subset of the training data reaching that node, a different model is appropriate" true? Do the selection counts of candidate models change according to the level of the tree?
- Do the selection counts of candidate models change according to the size of the dataset?

The model selection techniques are compared on twenty-two datasets from the UCI machine learning repository (Table D.1) [94]. To see the effect of the dataset size, we created 25, 50, 75 percent of instances of *pendigits* and *segment* and named them *pendigits25*, *pendigits50*, etc. The error, model complexity and learning time figures contain boxplots, where the box has lines at the lower quartile, median, and upper quartile values. The whiskers are lines extending from each end of the box to show the extent of the rest of the data. Outliers, shown with '+', are data with values beyond the ends of the whiskers.

Since model selection comparisons contain more than two methods, we give two tables where in the first table the raw results are shown. We also give a second table which contains pairwise comparisons; the entry (i, j) in this second table gives the number of datasets (out of 22) on which method i is statistically significantly better than method j with at least 95% confidence. In the second table, row and column sums are also given. The row sum gives the number of datasets out of 22 where the algorithm on the row outperforms at least one of the other algorithms. The column sum gives the number of datasets where the algorithm on the column is outperformed by at least one of the other algorithms.

6.3.2. Results

Figures 6.29, 6.30, 6.31 show the expected error, model complexity and learning time plots for different percentage levels of *pendigits* where prepruning, postpruning and model-selection-pruning is used for pruning respectively. Figures 6.32, 6.33, 6.34 show the expected error, model complexity and learning time plots for different percentage levels of *segment* where prepruning, postpruning and model-selection-pruning is used for pruning respectively. Since model-selection-pruning is not defined for pure trees, only prepruning and postpruning is applied for those.

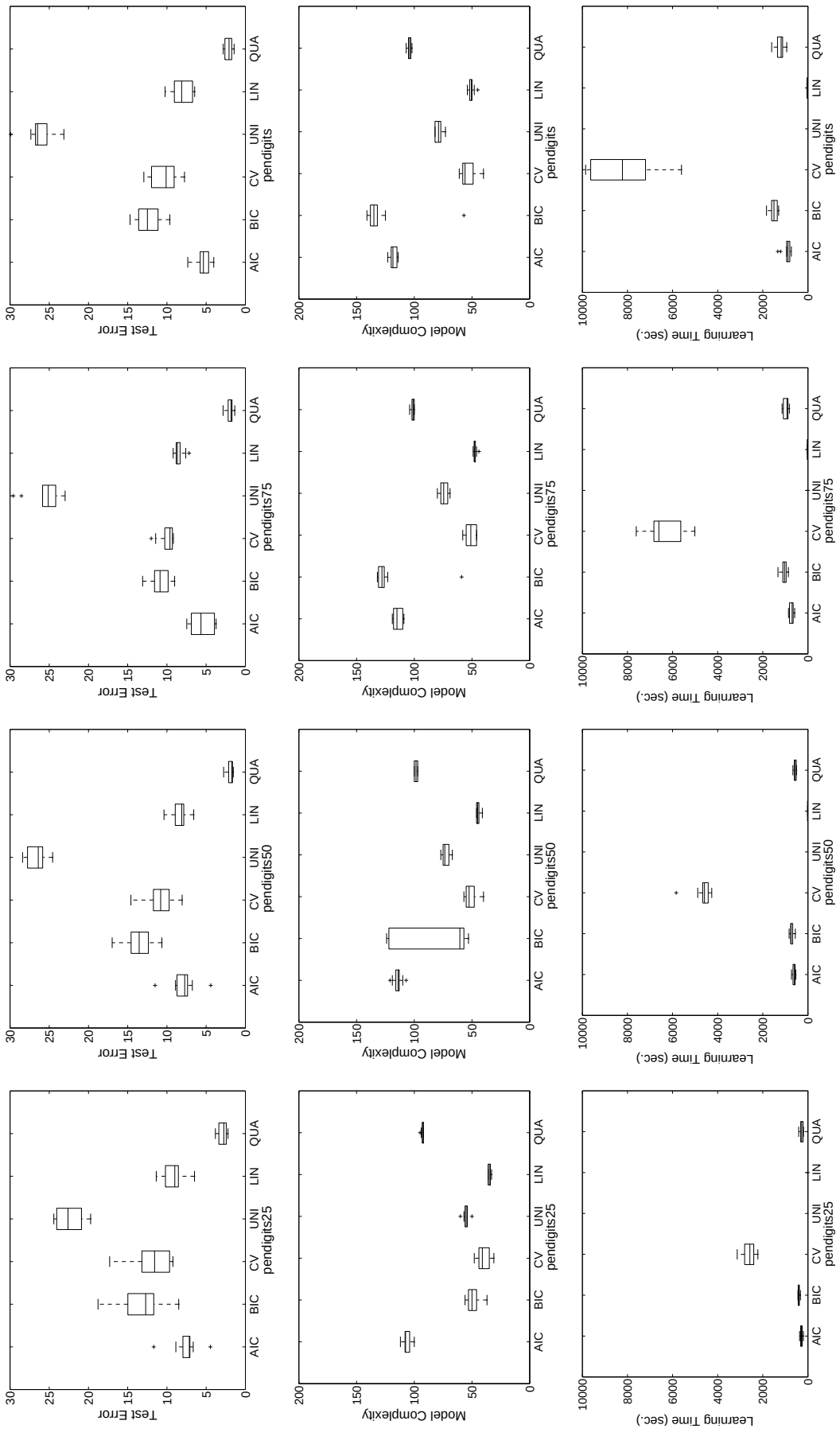


Figure 6.29. The error, model complexity and learning time plots for different percentage levels of *pendigits* with prepruning

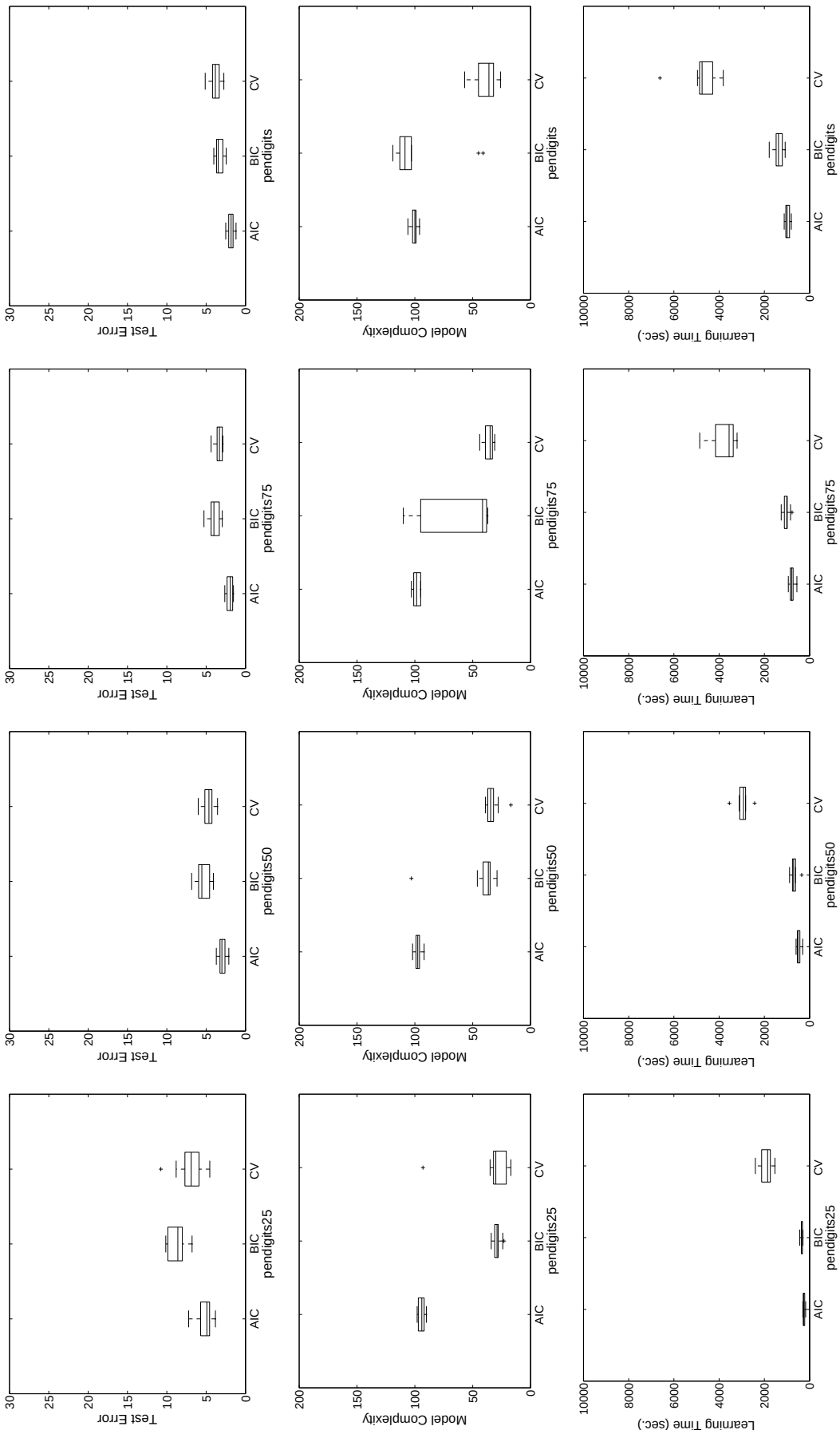


Figure 6.30. The error, model complexity and learning time plots for different percentage levels of *pendigits* with postpruning

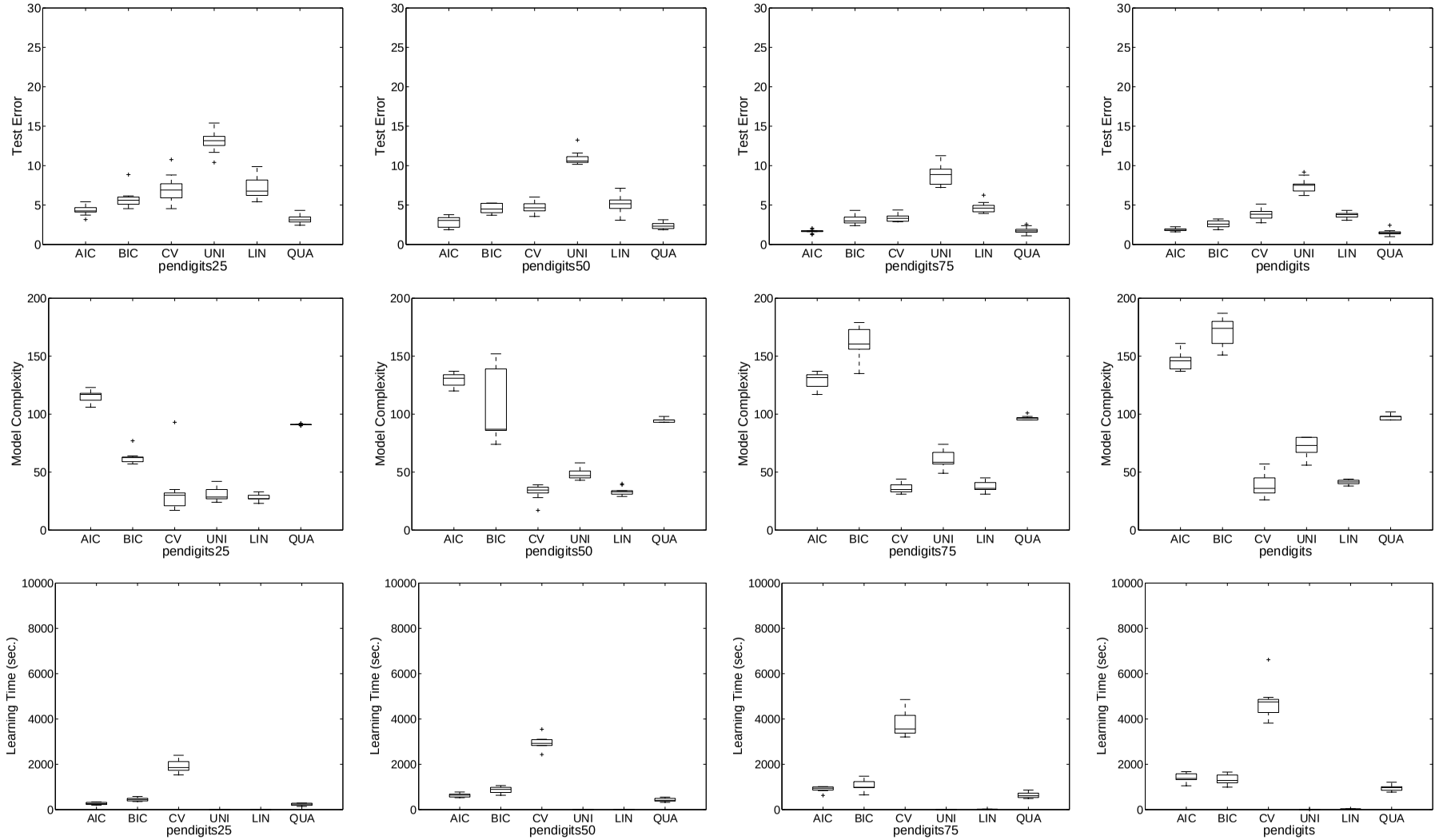


Figure 6.31. The error, model complexity and learning time plots for different percentage levels of *pendigits* with model-selection-pruning

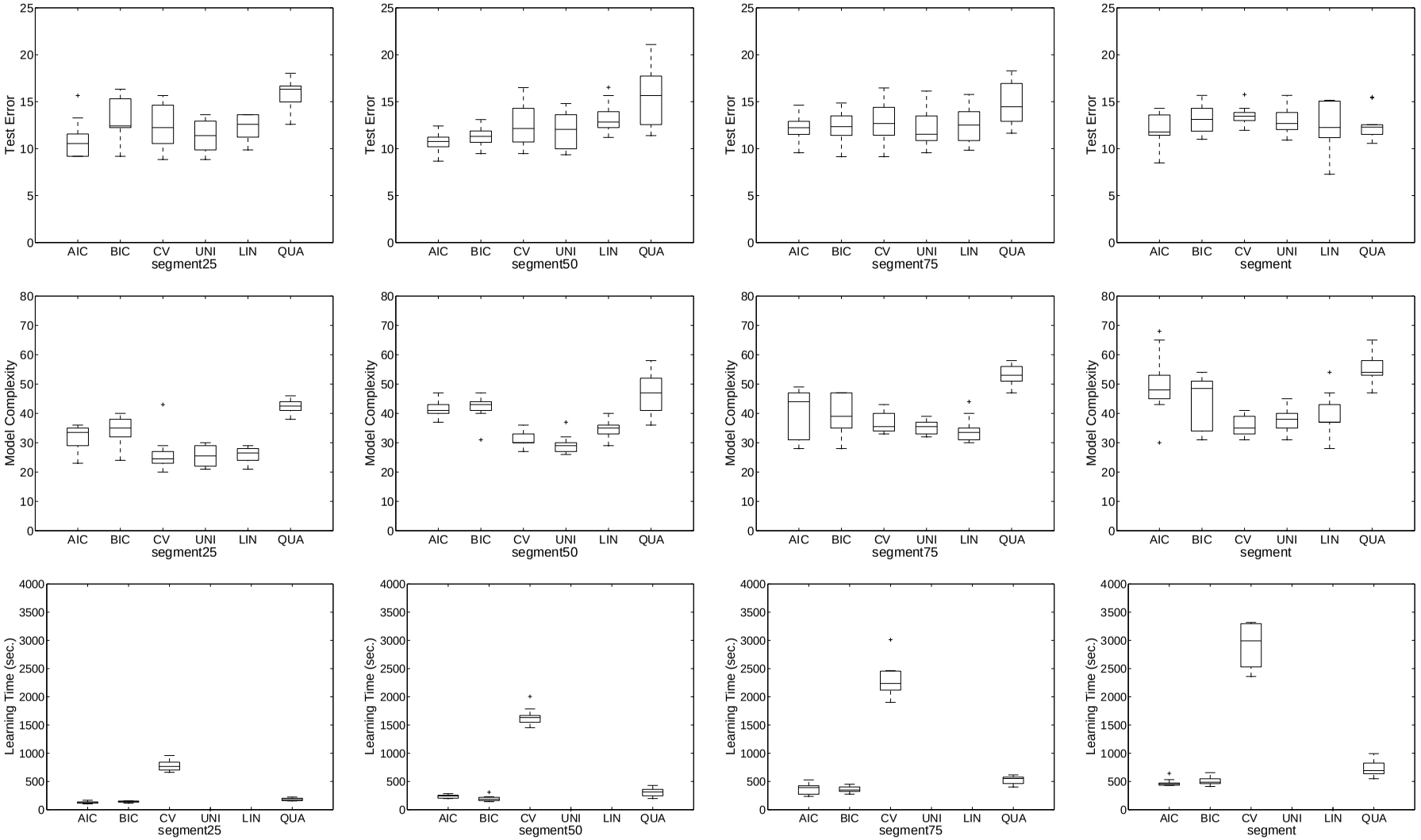


Figure 6.32. The error, model complexity and learning time plots for different percentage levels of *segment* with prepruning

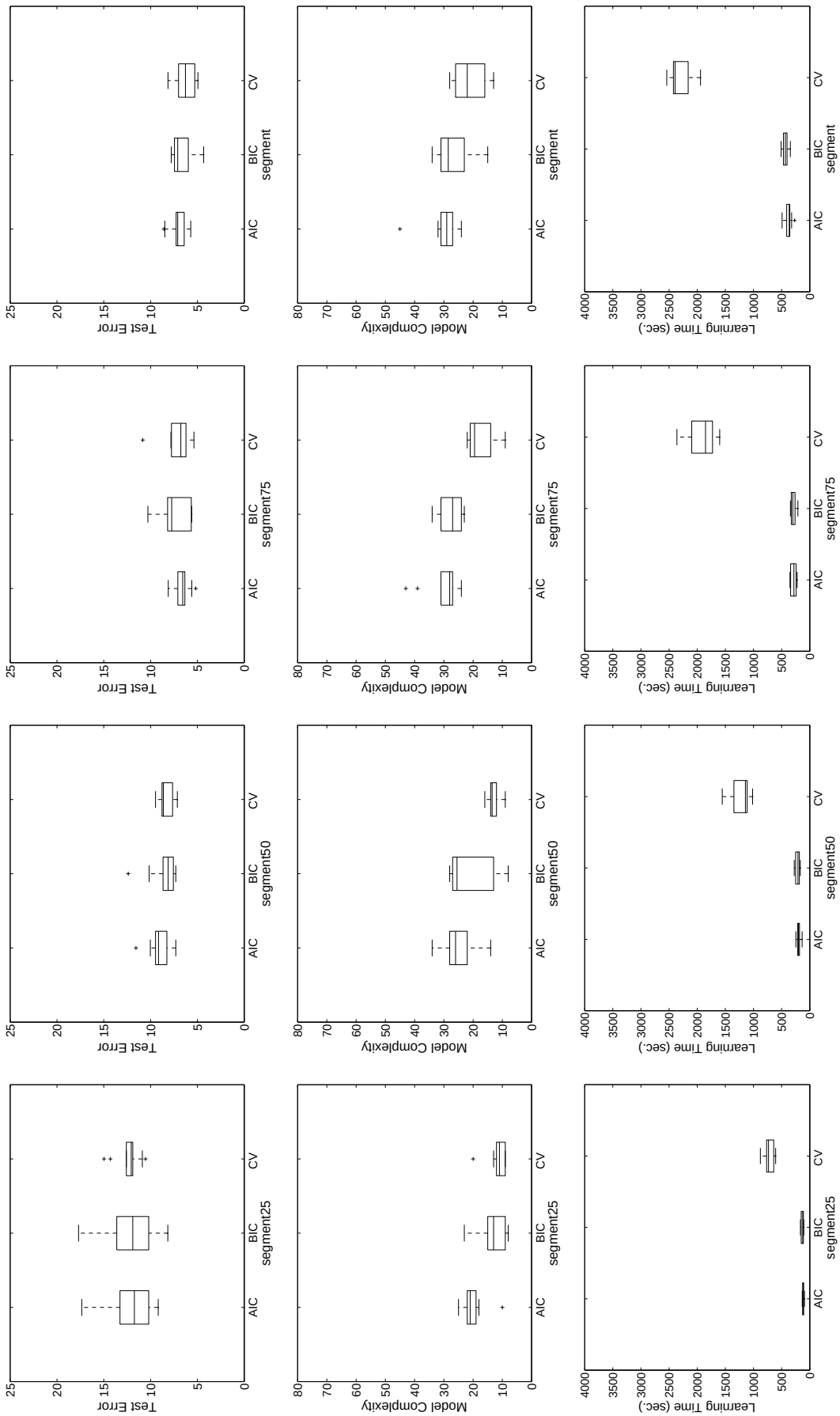


Figure 6.33. The error, model complexity and learning time plots for different percentage levels of *segment* with postpruning

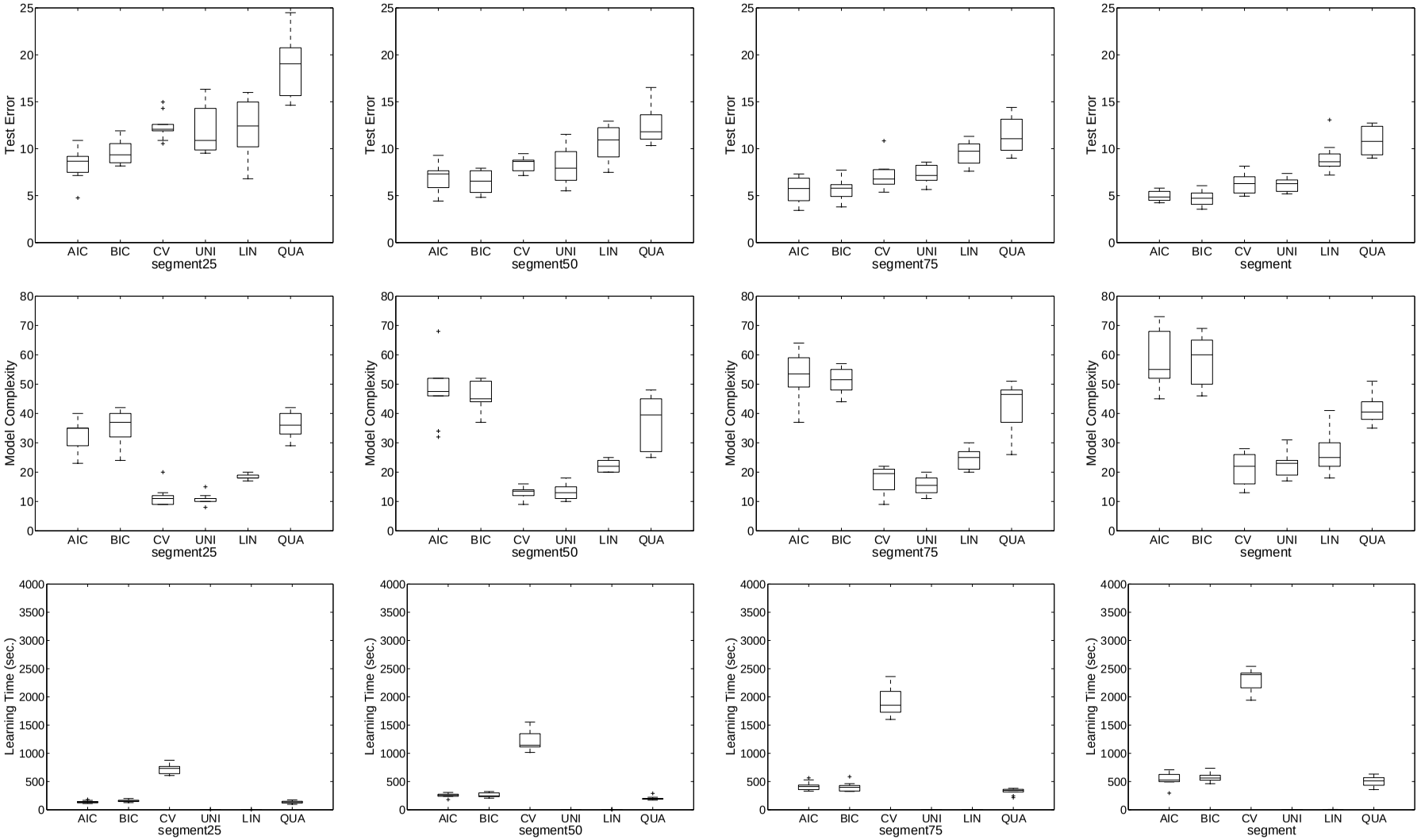


Figure 6.34. The error, model complexity and learning time plots for different percentage levels of *segment* with modelselection-pruning

We reach the following conclusions:

- Prepruning is worse than the other two pruning techniques in terms of validation error.
- Model-selection-pruning works, that is, it has smaller validation error than prepruning and similar error as postpruning.
- On *pendigits*, as the percentage of dataset increases, the average error decreases. On *segment*, there is no such decrease.
- On *pendigits*, quadratic model is better than the linear model which is better than the univariate model; on *segment*, univariate model is better than the linear model which is better than the quadratic model.
- Model selection in decision trees is justified. On *pendigits*, model selection techniques produce trees having expected error close to the quadratic model and on *segment*, is has expected error close to that of the univariate model.
- CV model selection technique produces trees with smaller complexity than AIC and BIC. AIC and BIC have similar performances with respect to model complexity where BIC penalizes the complex models more. Due to this reason, on *pendigits*, AIC selects quadratic model more, which makes it better in terms of expected error compared to BIC and CV.
- Since CV makes 5×2 cross-validation, its learning time is more than that of AIC and BIC.

Figures 6.35, 6.36, 6.37 show selection counts of univariate, multivariate linear and multivariate quadratic models at different levels of tree for different percentage levels of *pendigits* where prepruning, postpruning and model-selection-pruning is used for pruning respectively. Figures 6.38, 6.39, 6.40 show selection counts of univariate, multivariate linear and multivariate quadratic models at different levels of tree for different percentage levels of *segment* where prepruning, postpruning and model-selection-pruning is used for pruning respectively.

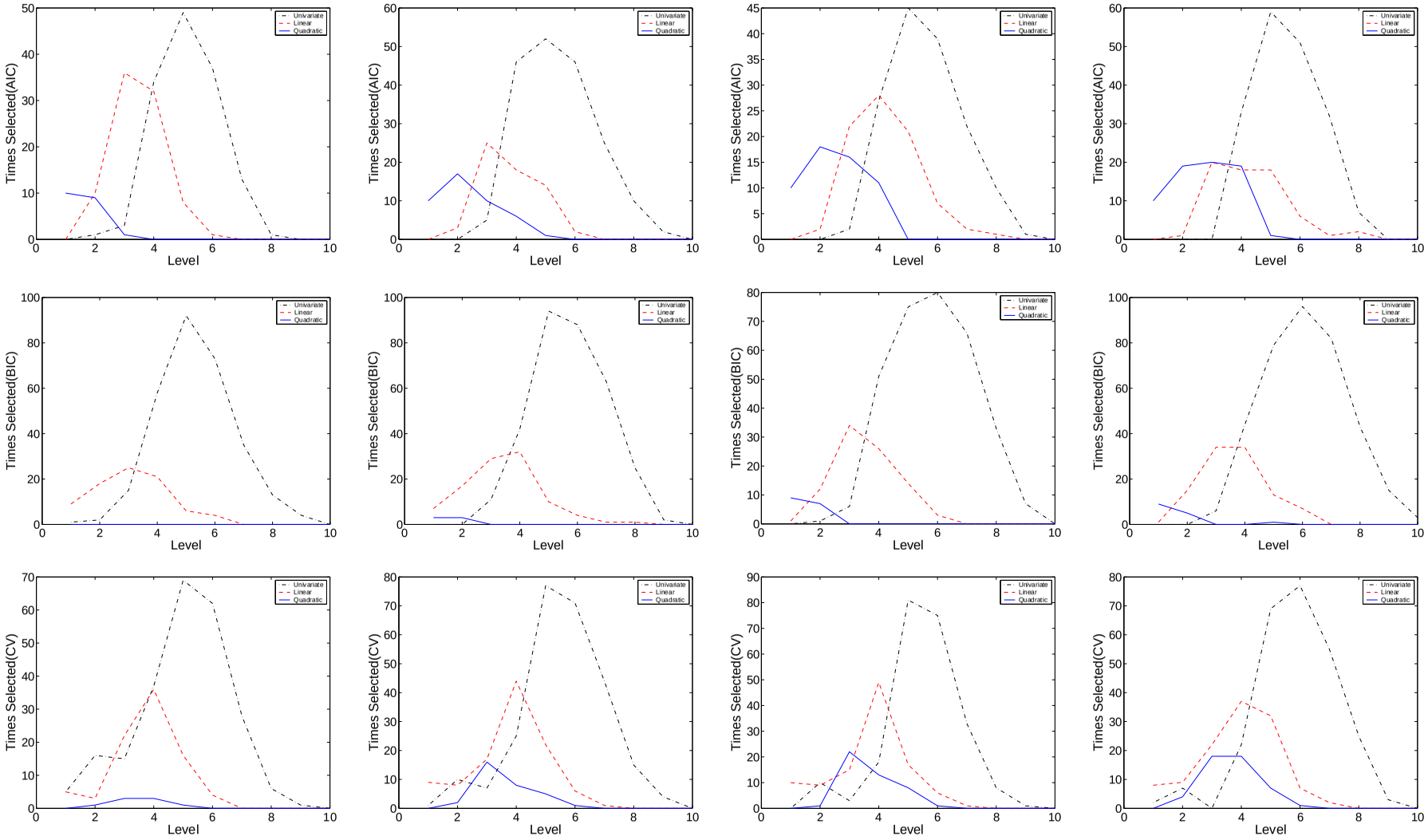


Figure 6.35. The number of times univariate, multivariate linear and multivariate quadratic models selected at different levels of tree for different percentage levels of *pendigits* with prepruning

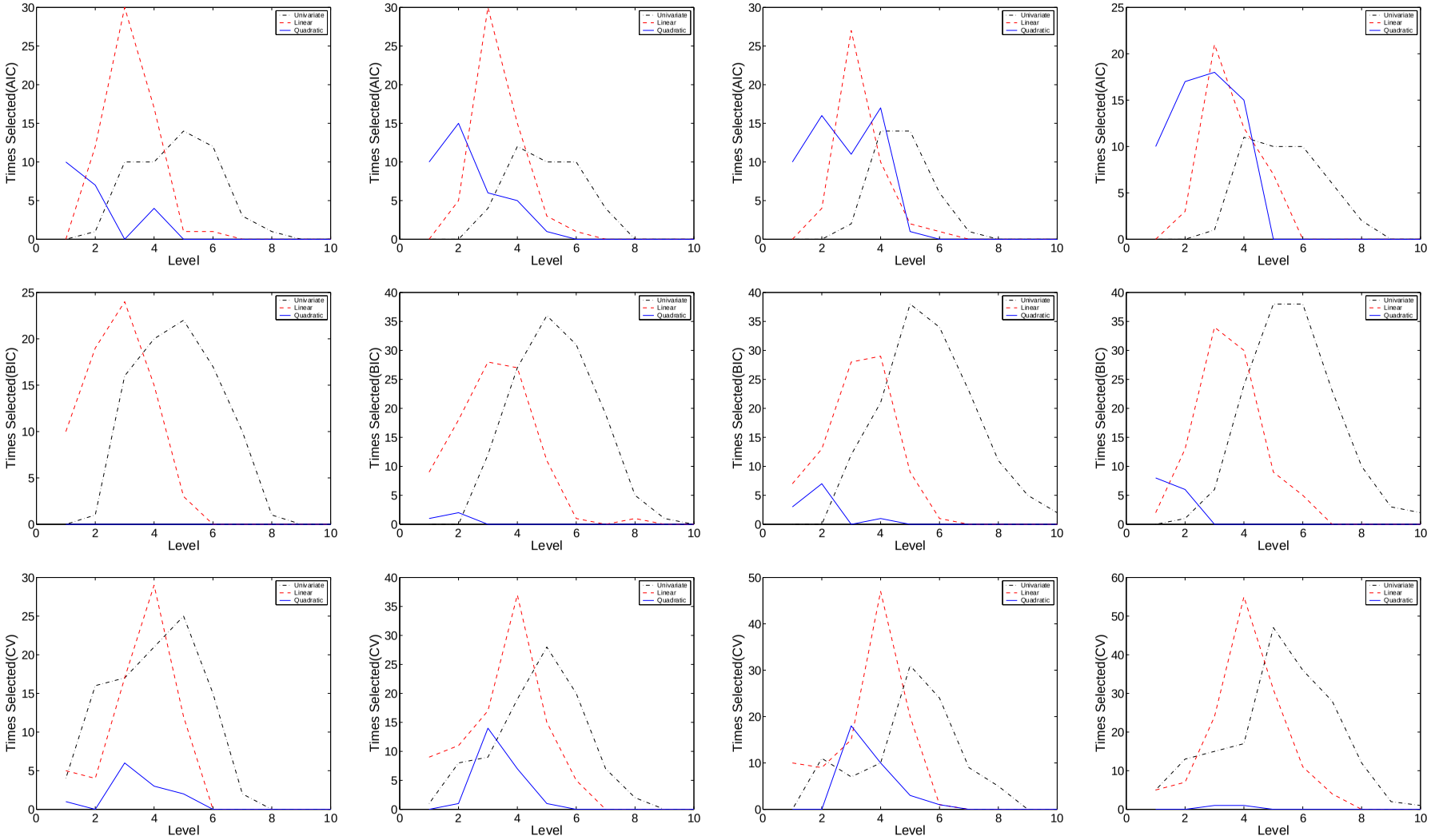


Figure 6.36. The number of times univariate, multivariate linear and multivariate quadratic models selected at different levels of tree for different percentage levels of *pendigits* with postpruning

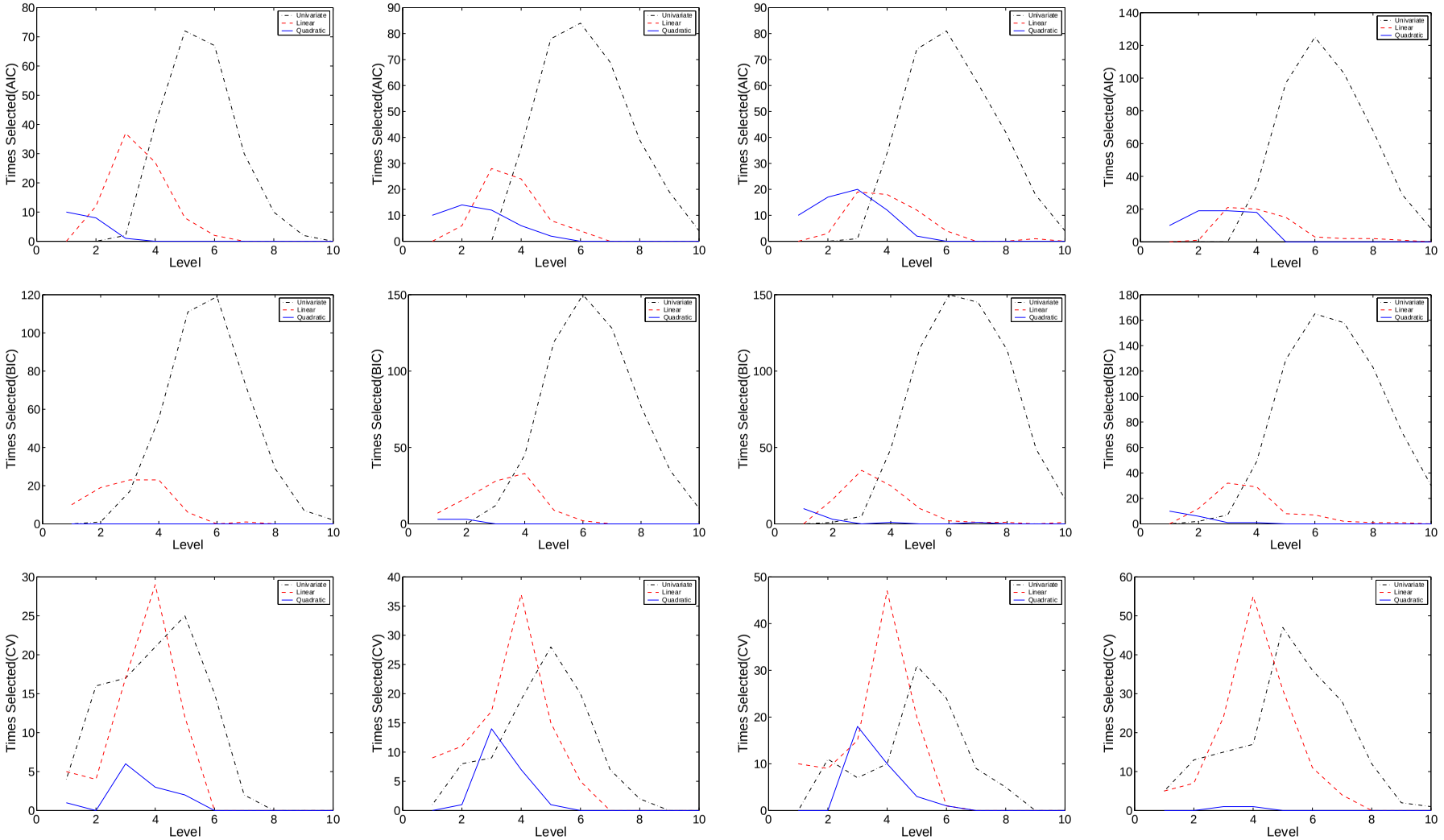


Figure 6.37. The number of times univariate, multivariate linear and multivariate quadratic models selected at different levels of tree for different percentage levels of *pendigits* with model-selection-pruning

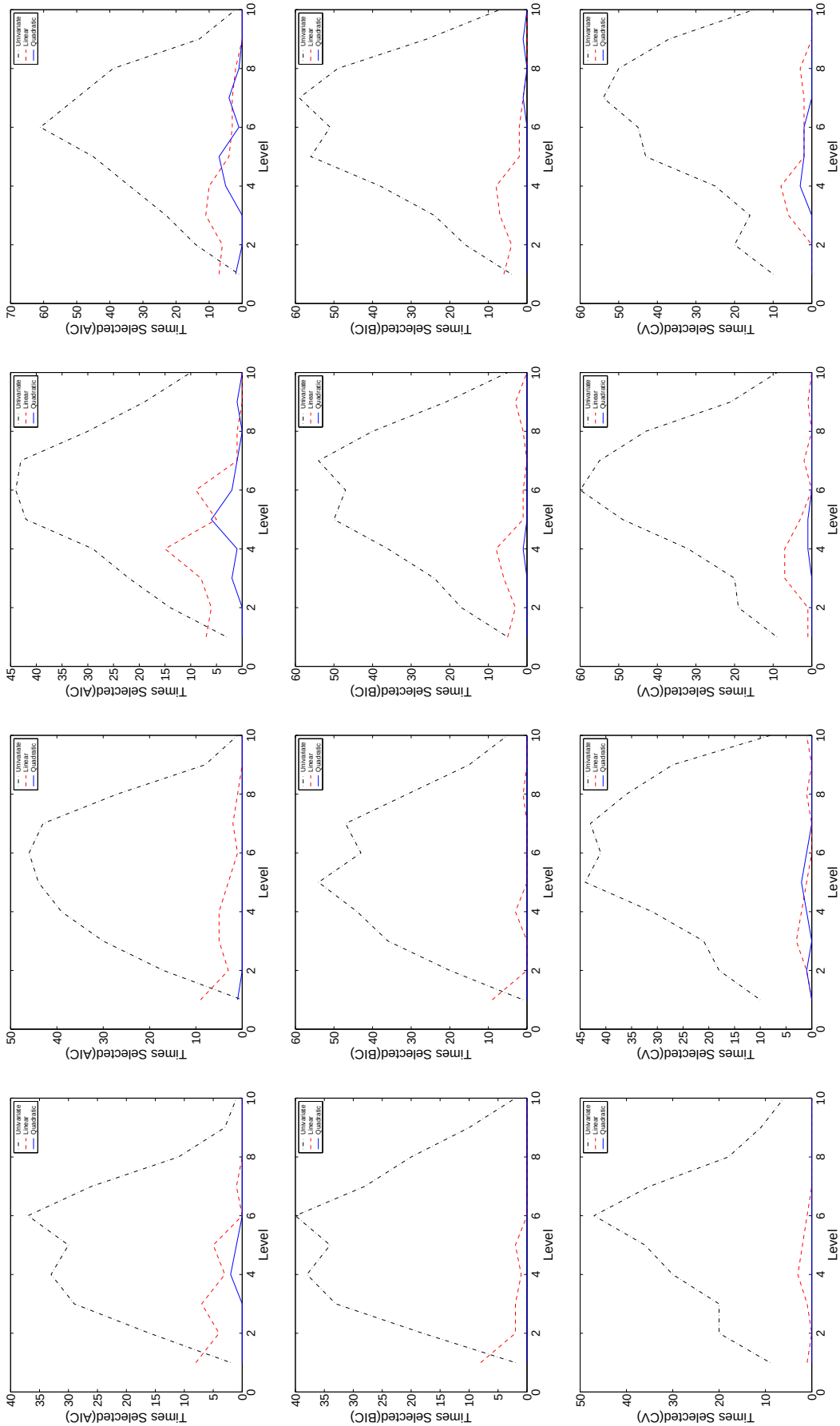


Figure 6.38. The number of times univariate, multivariate linear and multivariate quadratic models selected at different levels of tree for different percentage levels of *segment* with prepruning

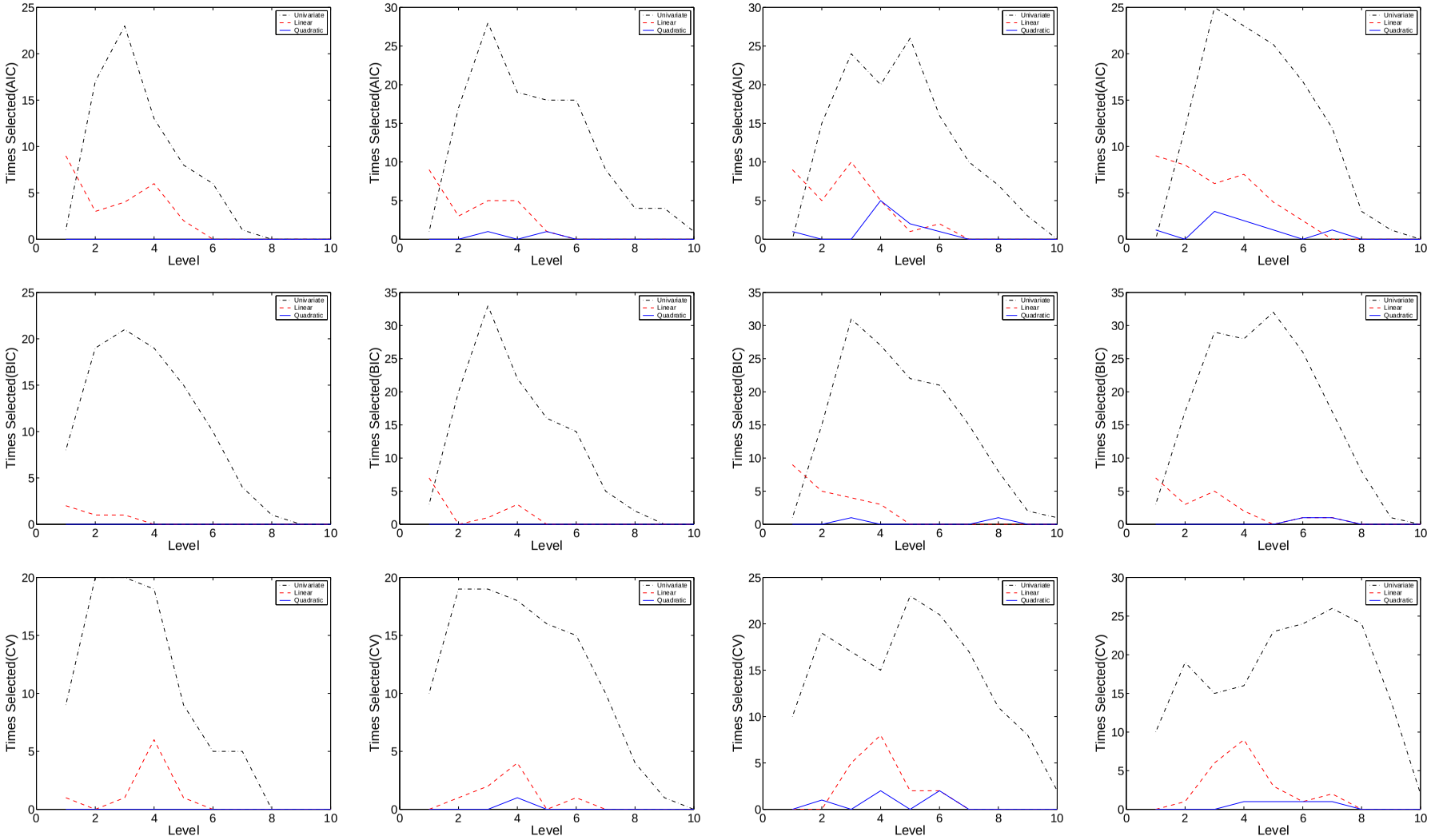


Figure 6.39. The number of times univariate, multivariate linear and multivariate quadratic models selected at different levels of tree for different percentage levels of *segment* with postpruning

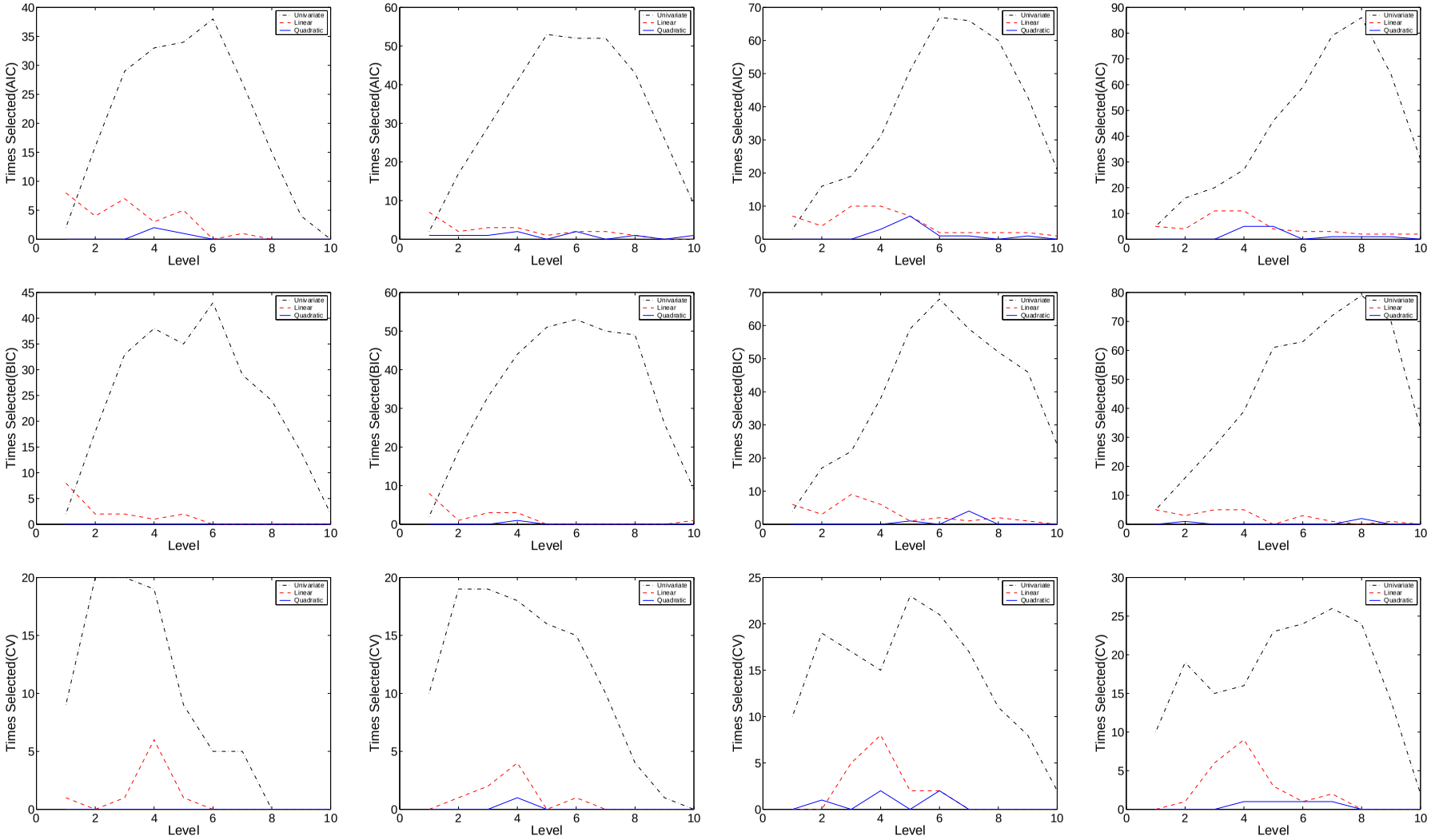


Figure 6.40. The number of times univariate, multivariate linear and multivariate quadratic models selected at different levels of tree for different percentage levels of *segment* with model-selection-pruning

We reach the following conclusions:

- On *pendigits*, as the dataset size increases, the selection count of quadratic model increases. On *segment* the reverse is true, that is, as the dataset size increases, the selection counts of linear and quadratic model decreases.
- Since quadratic model is the best on *pendigits* and univariate model is the best on *segment*, and since the omnivariate tree follows the best model, it selects the quadratic model on *pendigits* and the univariate model on *segment* more in the upper levels of the tree.
- Since the training set size decreases in the lower levels of the tree, the univariate model is selected most in those levels.
- AIC selects more complex models than BIC which selects more complex models than CV.
- There is not a significant difference between pruning techniques in terms of the percentages of different models (univariate, linear, quadratic).

The average and standard deviations of expected error, model complexity and learning time of decision trees produced by different model selection techniques and univariate, multivariate linear and multivariate quadratic decision trees for 22 datasets are given in Figures 6.8, 6.9, and 6.10 respectively. If we look at the expected errors, we see that AIC, BIC and CV win against the other techniques as many times as the best pure linear model. We also see that if there is a significant difference between one or two pure models, the omnivariate tree follows the better models. For example, on *balance*, pure linear model is the best and omnivariate trees have similar performance as the linear model. On *car*, pure univariate and linear models are the best and model selection techniques select those models. On *wave* and *wine*, pure linear and quadratic models are more accurate than the pure univariate model and the model selection techniques have expected error close to the expected error of those models. The model complexity table shows that CV prefers simpler models than AIC and BIC. It has more wins (22 against 10 and 15) and less losses (6 against 20 and 22). So CV chooses simpler but models as accurate as AIC and BIC. Since AIC and BIC techniques try all possible models, they have learning time close to the worst pure model (quadratic

model) learning time. CV tries all possible models 10 times (5×2 cross-validation), therefore it has the longest learning time.

The number of times univariate, multivariate linear and multivariate quadratic models are selected in decision trees produced by AIC, BIC and CV model selection techniques are given in Table 6.11. The selection counts of those models at different levels of tree for AIC, BIC and CV model selection techniques are given in Figures 6.41, 6.42, 6.43 respectively. We see that, as expected, quadratic models are selected least, then the linear models and the univariate models are selected most. Although CV has the smallest tree complexity, it also selects linear models the most (22 percent). AIC and BIC do not prune as much as the CV, therefore their node counts (tree complexity) are more than CV. We also see that AIC selects quadratic models more than BIC which selects more than CV in the early levels of the tree. For linear models, their percentages are close to each other.

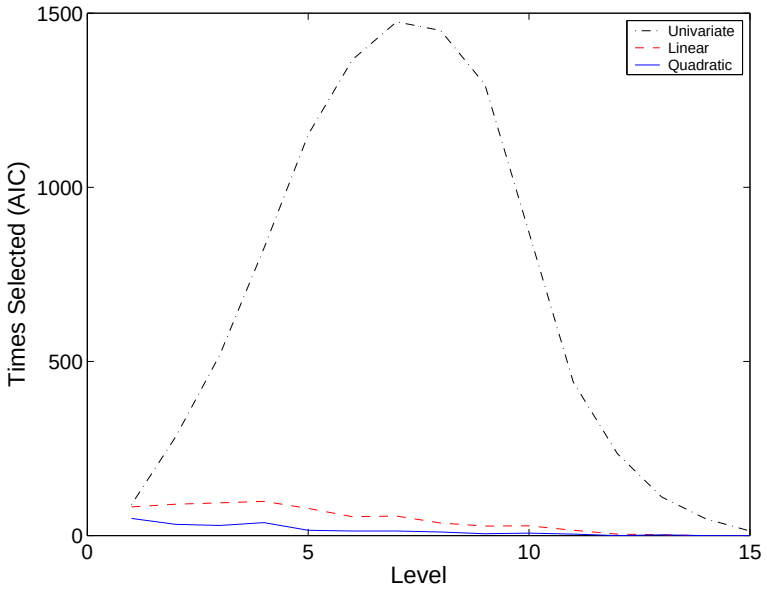


Figure 6.41. The number of times univariate, multivariate linear and multivariate quadratic models selected at different levels of tree for AIC model selection technique

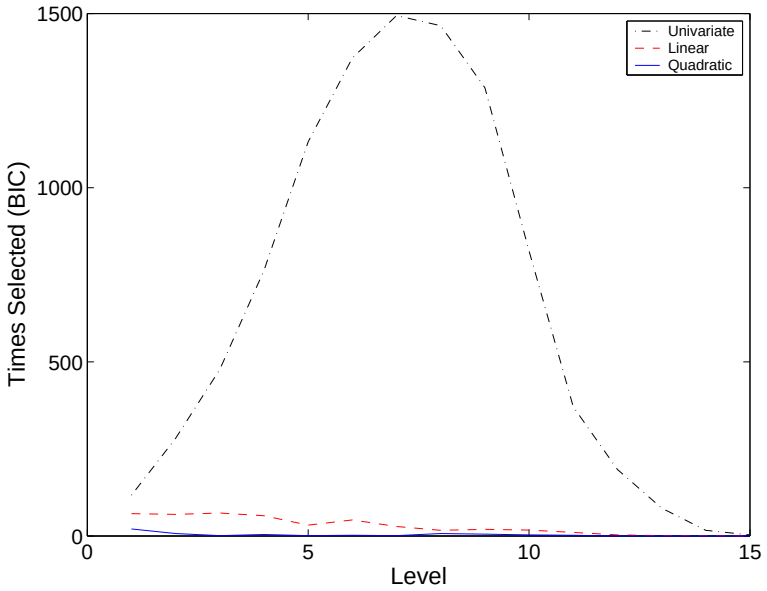


Figure 6.42. The number of times univariate, multivariate linear and multivariate quadratic models selected at different levels of tree for BIC model selection technique

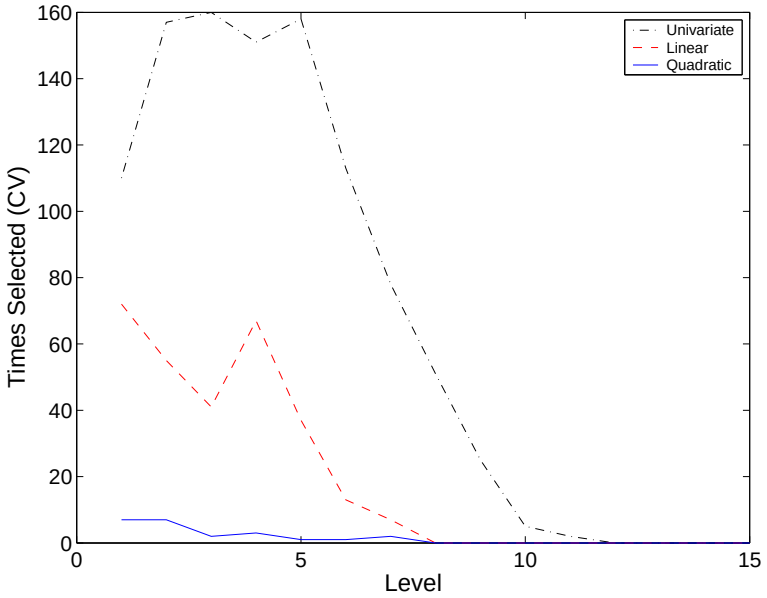


Figure 6.43. The number of times univariate, multivariate linear and multivariate quadratic models selected at different levels of tree for CV model selection technique

Table 6.8. The average and standard deviations of expected errors of omnivariate decision trees produced by AIC, BIC and CV and pure trees

	Omnivariate Trees			Pure Trees		
Set	AIC	BIC	CV	UNI	LIN	QUA
BAL	13.22, 1.56	12.70, 2.03	10.62, 1.13	25.89, 4.92	12.00, 1.92	24.10, 2.96
BRE	5.06, 1.12	4.92, 0.90	4.63, 1.33	5.69, 1.65	4.41, 0.54	3.72, 0.87
BUP	37.80, 4.06	37.44, 4.00	35.94, 4.80	40.17, 6.31	33.86, 4.17	41.22, 1.44
CAR	5.83, 1.07	6.08, 0.85	7.29, 1.77	7.38, 1.67	8.52, 1.64	21.32, 1.93
DER	5.74, 1.69	5.69, 1.48	7.70, 1.75	6.99, 1.88	3.33, 1.01	8.75, 1.99
ECO	21.41, 3.13	19.23, 2.30	19.74, 3.21	22.42, 3.78	18.17, 2.37	19.45, 2.62
FLA	14.79, 3.28	12.14, 2.49	11.21, 0.63	11.15, 0.54	11.39, 0.75	11.15, 0.54
GLA	36.80, 3.72	34.30, 3.12	38.40, 6.20	40.48, 9.06	42.71, 3.02	43.59, 6.21
HAB	33.58, 4.08	26.47, 0.16	26.60, 0.39	26.54, 0.21	26.47, 0.16	26.41, 0.97
HEP	22.57, 3.85	21.93, 4.26	21.93, 3.22	21.67, 1.95	20.39, 0.87	18.33, 2.88
IRI	4.14, 1.93	4.27, 2.07	5.73, 1.99	5.47, 4.10	3.07, 1.41	4.80, 3.51
IRO	11.51, 2.24	13.79, 1.65	12.71, 2.10	14.14, 2.80	11.56, 2.25	9.51, 2.62
MON	10.42, 3.58	9.21, 3.30	20.65, 6.61	20.79, 7.39	27.50,10.65	16.90, 2.21
PEN	1.86, 0.20	2.57, 0.44	3.83, 0.79	7.50, 0.93	3.70, 0.38	1.52, 0.37
PIM	30.70, 1.89	29.61, 2.90	30.78, 5.52	29.77, 3.90	23.10, 1.18	26.69, 3.04
SEG	4.96, 0.53	4.72, 0.77	6.27, 1.07	6.19, 0.75	9.05, 1.66	10.92, 1.47
TIC	18.89, 3.63	8.39, 1.67	16.41, 4.20	13.69, 4.47	29.42, 2.12	27.94, 4.83
VOT	5.98, 1.73	5.98, 1.75	4.32, 0.54	4.83, 1.32	5.34, 2.04	8.69, 1.99
WAV	20.47, 0.39	20.64, 1.04	16.25, 1.49	25.86, 1.09	14.77, 0.57	15.52, 0.46
WIN	5.40, 1.68	7.16, 4.59	6.15, 3.76	15.95, 4.16	3.15, 1.91	7.42, 4.00
YEA	51.07, 2.03	50.89, 1.63	49.96, 4.95	48.99, 4.04	45.09, 2.50	47.72, 3.64
ZOO	5.32, 2.73	5.32, 2.73	11.22, 4.69	10.70, 4.29	22.61, 5.07	21.50, 5.85

	AIC	BIC	CV	UNI	LIN	QUA	$\sum$
AIC	0	2	5	7	7	10	14
BIC	4	0	5	9	7	10	15
CV	5	3	0	5	4	8	15
UNI	4	1	1	0	3	6	8
LIN	9	6	5	10	0	9	17
QUA	7	6	3	6	3	0	11
$\sum$	14	10	12	17	8	13	

Table 6.9. The average and standard deviations of total node complexities of decision trees produced by AIC, BIC and CV and pure trees

	Omnivariate Trees			Pure Trees		
Set	AIC	BIC	CV	UNI	LIN	QUA
BAL	45.5, 5.9	43.2, 10.0	17.0, 0.0	11.7, 4.7	17.9, 1.3	106.9, 0.9
BRE	28.0, 10.0	22.4, 5.0	17.1, 12.0	6.0, 3.2	10.2, 0.4	37.1, 1.0
BUP	52.3, 5.3	45.1, 16.8	7.9, 4.6	5.3, 4.9	7.3, 2.8	6.1, 9.8
CAR	67.9, 9.8	63.0, 10.3	27.3, 6.5	28.6, 4.7	19.9, 2.5	110.1, 1.4
DER	25.7, 14.7	13.3, 2.1	6.4, 0.7	7.2, 1.2	33.7, 0.5	112.2, 1.5
ECO	30.5, 7.8	26.8, 2.8	4.8, 1.2	5.5, 2.9	10.7, 1.3	21.0, 1.4
FLA	17.8, 3.4	2.4, 5.1	0.4, 1.3	0.3, 0.9	3.9, 8.2	0.0, 0.0
GLA	29.7, 3.7	30.2, 3.9	7.6, 2.3	6.9, 3.7	11.3, 2.4	21.9, 2.6
HAB	36.1, 3.7	0.0, 0.0	0.7, 2.2	1.0, 3.2	0.0, 0.0	3.3, 5.3
HEP	12.8, 3.2	13.3, 2.2	2.6, 5.9	2.1, 3.2	5.8, 9.3	28.8, 24.9
IRI	4.5, 1.4	4.1, 1.7	3.0, 0.0	3.6, 1.0	5.7, 0.5	10.5, 0.5
IRO	36.8, 7.9	18.2, 2.5	17.2, 14.5	4.2, 1.9	29.6, 0.7	57.4, 2.9
MON	37.5, 5.4	29.8, 2.2	17.1, 11.8	9.1, 3.6	8.4, 3.6	26.2, 0.4
PEN	146.0, 7.7	171.5, 11.5	38.5, 10.0	72.0, 8.2	41.4, 2.0	97.4, 2.2
PIM	101.1, 15.6	92.2, 10.5	4.0, 5.3	7.4, 7.6	9.5, 0.8	36.0, 12.7
SEG	58.0, 9.8	57.9, 8.0	21.0, 5.4	22.8, 4.1	26.3, 6.7	41.7, 4.9
TIC	185.0, 58.1	52.8, 9.6	20.3, 9.2	21.4, 3.5	21.5, 3.0	96.6, 66.7
VOT	13.6, 3.0	10.0, 3.9	2.4, 1.3	2.9, 1.5	16.9, 0.7	82.8, 2.4
WAV	406.2, 96.4	331.2, 10.4	23.8, 0.9	35.9, 14.3	26.6, 1.7	221.0, 1.2
WIN	14.3, 0.7	11.4, 4.2	13.1, 3.6	4.0, 0.7	14.0, 0.0	48.0, 1.2
YEA	287.9, 13.8	281.3, 13.4	6.9, 3.6	20.0, 10.3	20.2, 4.7	37.6, 3.9
ZOO	8.1, 0.9	8.1, 0.9	6.2, 1.1	6.7, 0.8	15.8, 0.9	25.7, 1.6

	AIC	BIC	CV	UNI	LIN	QUA	$\sum$
AIC	0	1	0	0	3	9	10
BIC	10	0	0	0	5	11	15
CV	21	18	0	3	11	20	22
UNI	21	19	6	0	12	19	21
LIN	17	14	2	3	0	19	21
QUA	12	9	0	0	0	0	12
$\sum$	22	20	7	4	15	20	

Table 6.10. The average and standard deviations of learning times (secs.) for decision trees produced by AIC, BIC and CV and pure trees

	Omnivariate Trees			Pure Trees		
Set	AIC	BIC	CV	UNI	LIN	QUA
BAL	114, 32	139, 44	717, 293	0, 0	0, 0	42, 8
BRE	1, 1	1, 0	3, 1	0, 0	0, 0	0, 0
BUP	0, 0	0, 0	1, 0	0, 0	0, 0	0, 0
CAR	540, 83	657, 105	3904, 508	0, 0	1, 0	356, 59
DER	828, 265	1120, 411	9145, 2979	0, 0	1, 1	1312, 229
ECO	2, 1	2, 1	6, 1	0, 0	0, 0	1, 1
FLA	138, 44	163, 50	319, 80	0, 0	0, 0	62, 15
GLA	3, 1	3, 1	13, 1	0, 0	0, 0	2, 0
HAB	0, 0	0, 0	0, 0	0, 0	0, 0	0, 0
HEP	7, 4	8, 5	36, 37	0, 0	0, 0	1, 0
IRI	0, 0	0, 0	0, 0	0, 0	0, 0	0, 0
IRO	160, 64	282, 45	931, 243	0, 0	0, 0	106, 42
MON	0, 0	0, 0	1, 0	0, 0	0, 0	0, 0
PEN	1416, 193	1323, 226	4758, 744	4, 0	27, 2	969, 138
PIM	1, 0	1, 0	5, 0	0, 0	0, 0	1, 1
SEG	547, 116	568, 82	2288, 207	1, 0	4, 1	508, 91
TIC	589, 231	694, 214	4930, 2415	0, 0	0, 0	63, 7
VOT	187, 107	186, 96	960, 1125	0, 0	0, 0	54, 11
WAV	858, 107	890, 50	4596, 368	1, 0	4, 1	714, 58
WIN	1, 1	2, 1	7, 1	0, 0	0, 0	1, 0
YEA	44, 5	47, 3	204, 16	0, 0	1, 0	34, 5
ZOO	17, 3	21, 4	107, 14	0, 0	0, 0	11, 3

	AIC	BIC	CV	UNI	LIN	QUA	$\sum$
AIC	0	7	20	0	0	1	20
BIC	0	0	20	0	0	0	20
CV	0	0	0	0	0	0	0
UNI	16	17	20	0	6	17	20
LIN	17	17	20	0	0	17	20
QUA	13	15	20	0	0	0	20
$\sum$	17	18	20	0	6	17	

Table 6.11. The number of times univariate, multivariate linear and multivariate quadratic models selected in decision trees produced by AIC, BIC and CV

	AIC			BIC			CV		
	UNI	LIN	QUA	UNI	LIN	QUA	UNI	LIN	QUA
BAL	261	34	0	251	21	0	0	10	0
BRE	143	18	1	119	14	1	18	8	2
BUP	416	44	8	400	26	1	21	9	0
CAR	452	77	0	433	47	0	101	22	0
DER	64	14	0	123	0	0	51	3	0
ECO	232	30	4	230	18	0	32	6	0
FLA	164	3	1	22	0	0	3	0	0
GLA	273	9	5	283	7	2	65	1	0
HAB	309	21	13	0	0	0	6	0	0
HEP	116	2	0	123	0	0	4	1	0
IRI	26	8	1	22	9	0	20	0	0
IRO	93	11	6	172	0	0	20	11	0
MON	107	5	13	35	3	10	28	1	7
PEN	465	65	66	742	92	18	176	137	2
PIM	799	40	9	853	23	1	4	4	0
SEG	454	48	13	487	24	3	174	22	4
TIC	596	8	11	516	2	0	160	10	7
VOT	122	4	0	89	1	0	14	0	0
WAV	2792	59	5	3072	30	0	1	27	0
WIN	8	14	0	31	7	0	10	12	0
YEA	2401	157	60	2662	102	18	50	8	1
ZOO	71	0	0	71	0	0	52	0	0
$\sum$	10364	671	216	10736	426	54	1010	292	23
%	92.12	5.96	1.92	95.72	3.80	0.48	76.22	22.04	1.74

6.4. Rule Induction

In this section, we compare our proposed MultiTest-based cross-validation technique with other model selection techniques in the context of rule induction. We first list the questions that motivate us for the experiments. We design experiments to answer the following questions:

- Which model selection technique(s) is/are the best in terms of expected error, model complexity and learning time?
- What is the proportion of univariate, multivariate linear and multivariate quadratic conditions chosen with each model selection technique?
- Is the hypothesis "At each condition of the rule, which corresponds to a different subproblem defined by the subset of the training data reaching that condition, a different model is appropriate" true? Do the selection counts of candidate models change according to the order of condition in the rule?
- Do the selection counts of candidate models change according to the size of the dataset?

The model selection techniques are compared on twenty-two datasets from the UCI machine learning repository (Table D.1) [94]. To see the effect of the dataset size, we again used the 25, 50, 75 percent of instances of *pendigits* and *segment*. The error, model complexity and learning time figures are shown using boxplots.

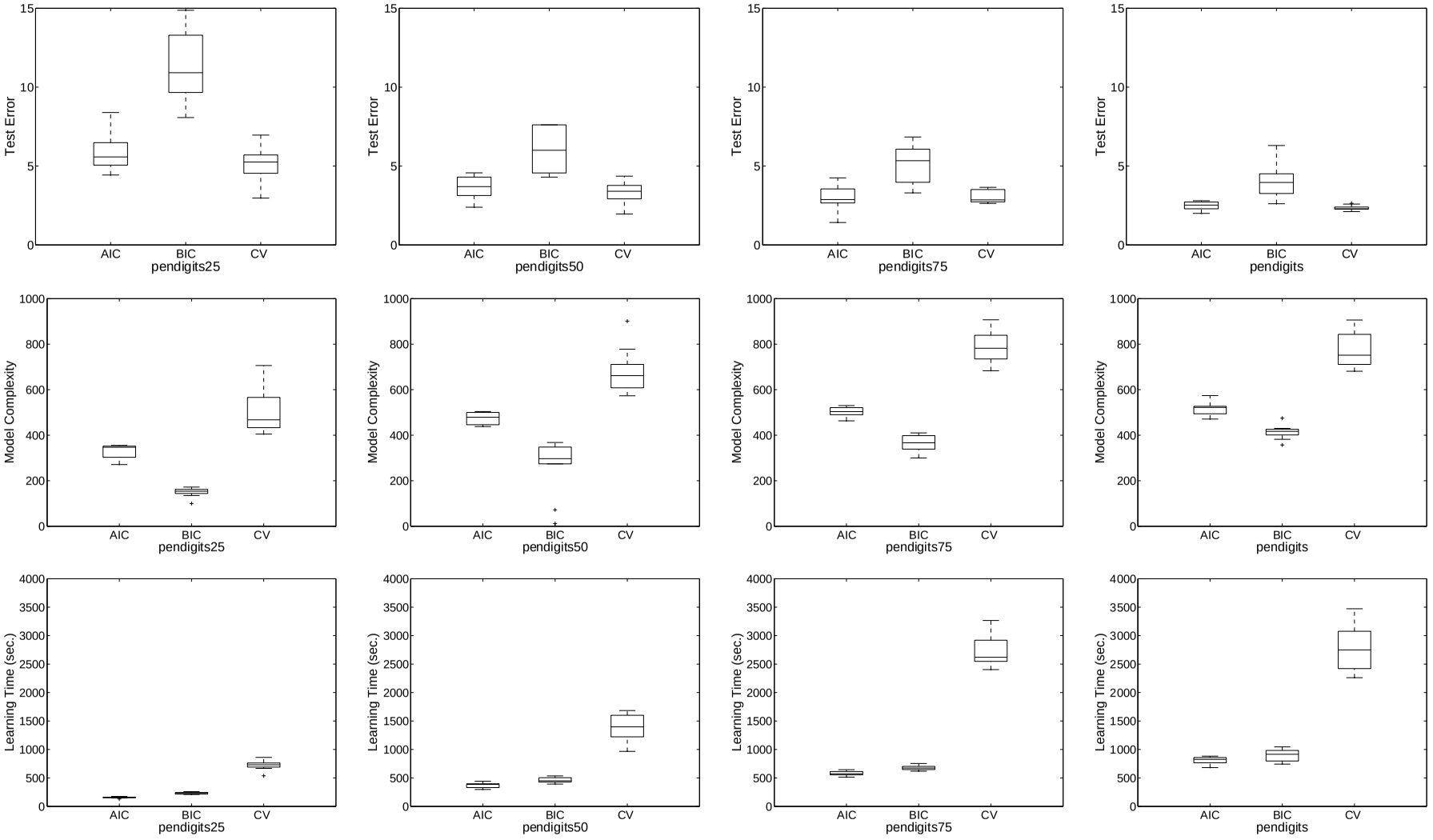


Figure 6.44. The expected error, model complexity and learning time plots of rulesets generated by AIC, BIC and CV for different percentage levels of *pendigits*

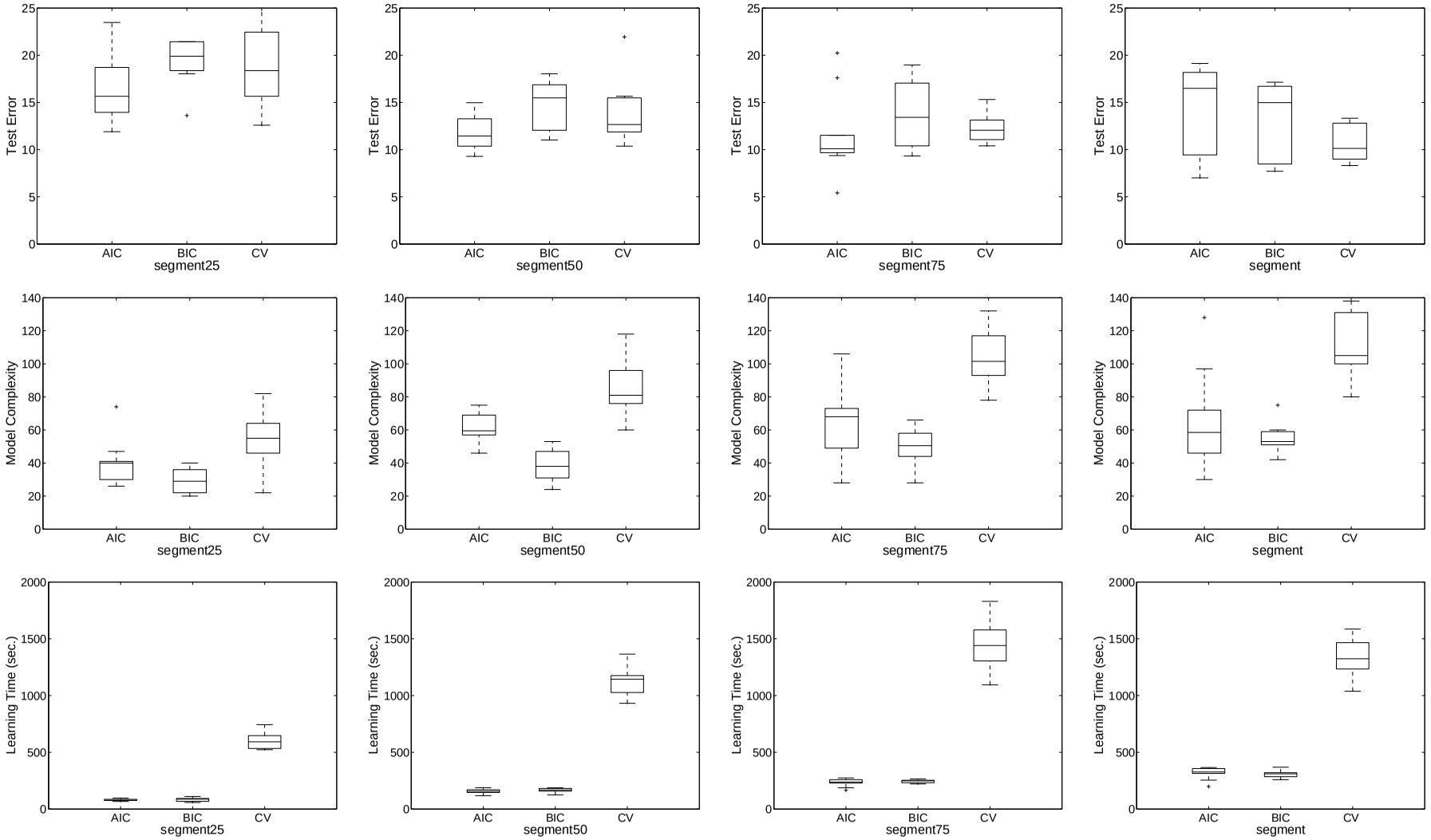


Figure 6.45. The expected error, model complexity and learning time plots of rulesets generated by AIC, BIC and CV for different percentage levels of *segment*

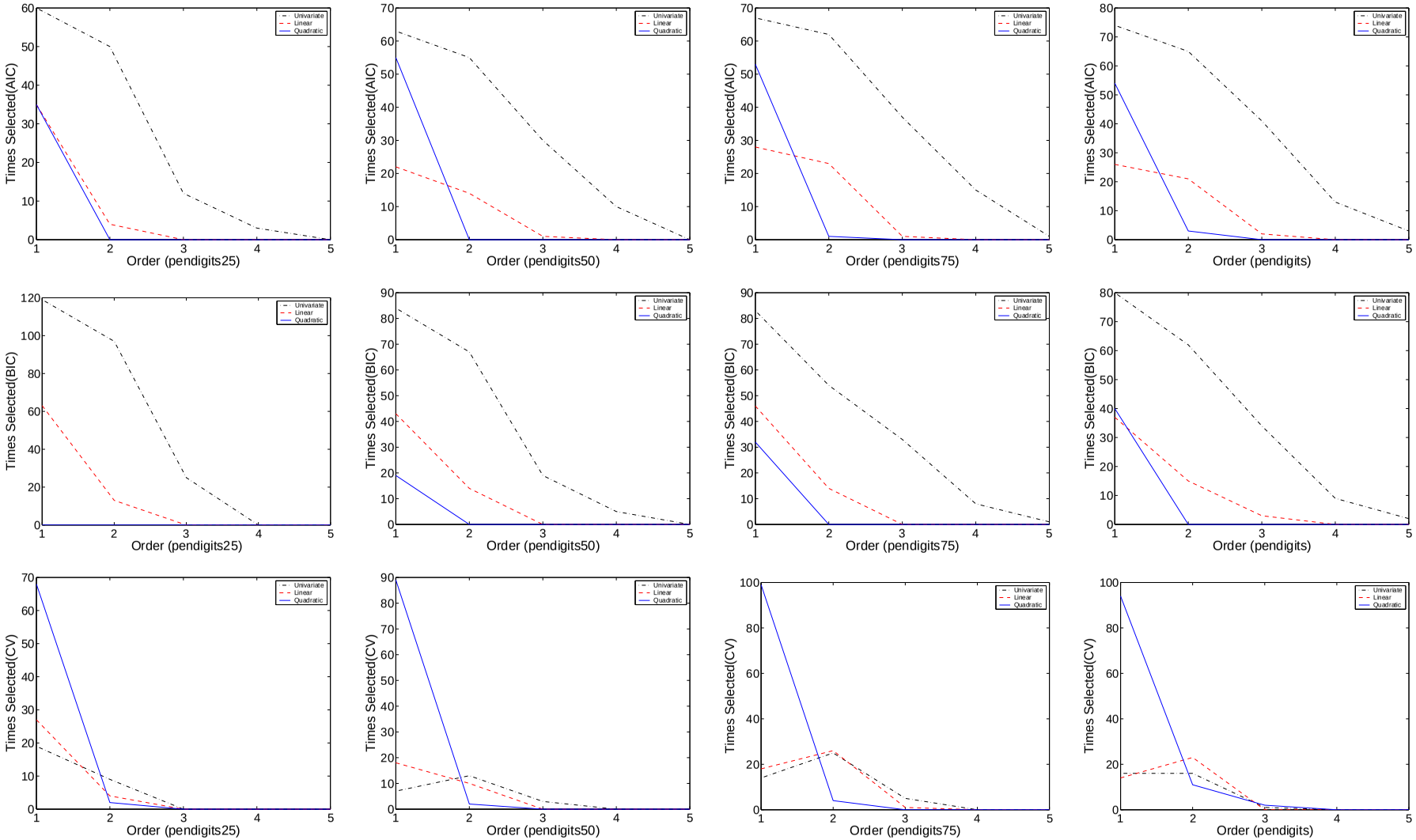


Figure 6.46. The number of times univariate, multivariate linear and multivariate quadratic models selected at different orders of condition in a rule for different percentage levels of *pendigits*

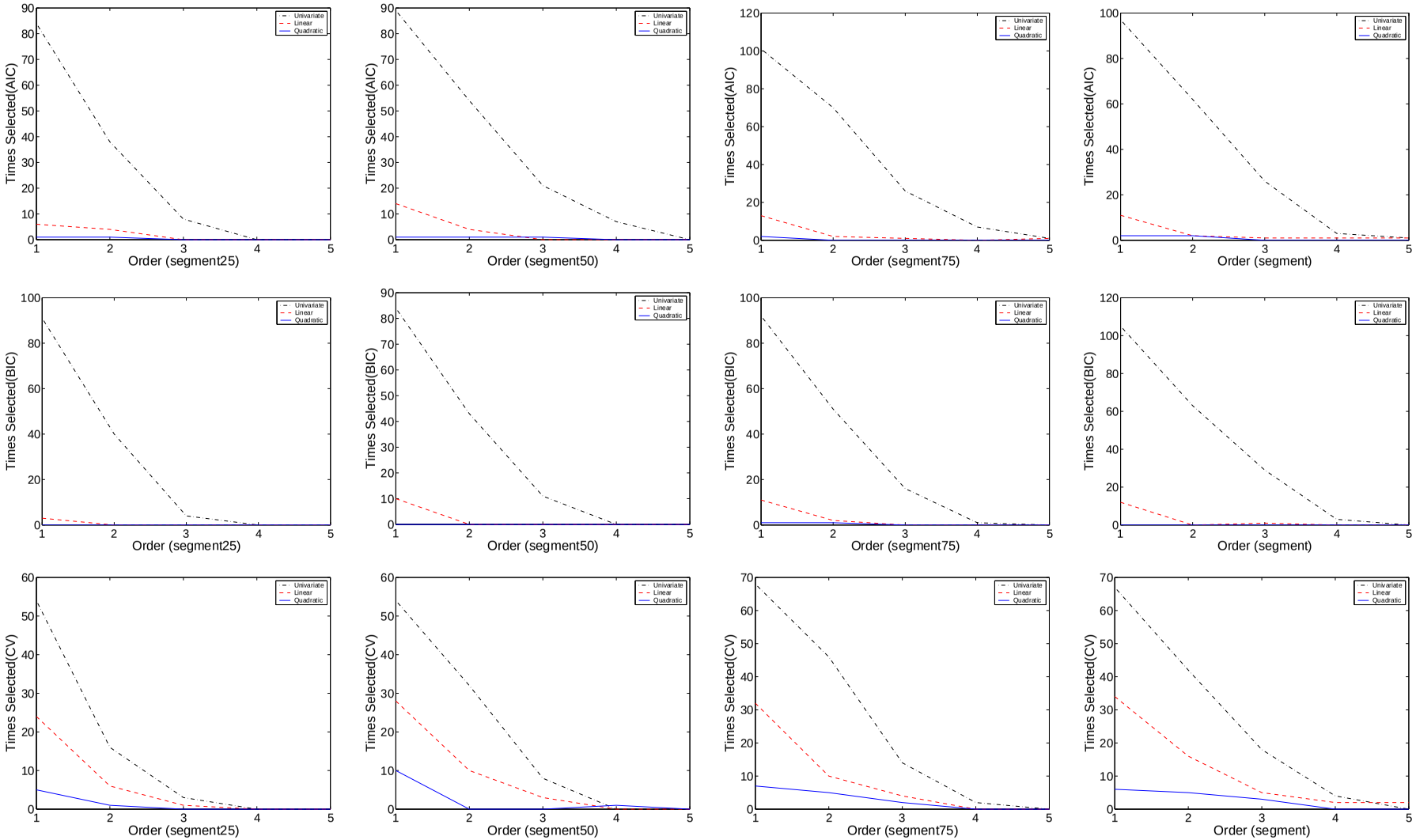


Figure 6.47. The number of times univariate, multivariate linear and multivariate quadratic models selected at different orders of condition in a rule for different percentage levels of *segment*

6.4.1. Results

Figures 6.44 and 6.45 show the expected error, model complexity and learning time plots of rulesets generated by AIC, BIC and CV model selection techniques for different percentage levels of *pendigits* and *segment* respectively. We see that, as the percentage level of the datasets increases, the average error decreases, the complexity and the learning time increases. On *pendigits*, AIC and CV has the smallest error, whereas on *segment*, the difference is not significant; they have nearly the same error. On *pendigits*, BIC generates simpler rulesets than AIC which generates simpler rulesets than CV. On *segment*, AIC and BIC have close ruleset complexity and they choose simpler models than CV. Since CV does 5×2 cross-validation, it has the longest training time for both datasets. AIC and BIC have similar learning time.

Figures 6.46 and 6.47 show the selection counts of univariate, multivariate linear and multivariate quadratic models at different orders of condition in a rule for different percentage levels of *pendigits* and *segment* respectively. We see that there is no rule having more than five conditions. The percentage level of the dataset does not change the selection counts of three model selection techniques. For both datasets, CV selects complex models the most, whereas BIC selects simple models the most. On *pendigits*, CV selects quadratic model more as the first condition in a rule, for other conditions, the univariate model is mostly selected. For AIC and BIC, for both datasets, the univariate model is considered as the best model in all orders.

The average and standard deviations of expected error, model complexity and learning time of rulesets produced by AIC, BIC and CV model selection techniques and the univariate RIPPER algorithm for twenty-two datasets are given in Tables 6.12, 6.13, and 6.14 respectively. With respect to the expected error, CV seems to be the best. It has 8 wins (the most) and 3 losses. RIPPER performs well with 6 wins and 6 losses. Since RIPPER is the univariate rule induction algorithm, it has the smallest ruleset complexity (18 wins, 3 losses). Like on *pendigits* and *segment*, CV selects the most complex models and BIC is better than AIC in terms of losses due to the $\log N$ term in the complexity term calculations (6 against 16). Since the

Table 6.12. The average and standard deviations of expected error of rulesets produced by AIC, BIC, CV, and RIPPER algorithm

	Omnivariate Rules			Pure Rules
Dataset	AIC	BIC	CV	RIPPER
BAL	10.63 ± 1.16	11.23 ± 1.38	10.69 ± 1.19	29.76 ± 1.61
BRE	5.18 ± 1.60	5.66 ± 2.25	4.49 ± 0.78	5.44 ± 1.08
BUP	38.43 ± 6.25	39.71 ± 6.10	34.96 ± 5.05	38.37 ± 4.90
CAR	12.16 ± 4.27	17.34 ± 4.39	12.67 ± 1.90	22.80 ± 1.42
DER	6.01 ± 1.93	15.38 ± 9.40	6.72 ± 1.04	9.47 ± 3.16
ECO	19.33 ± 3.86	21.62 ± 3.98	18.62 ± 4.43	21.13 ± 2.11
FLA	11.89 ± 2.06	12.95 ± 4.03	11.15 ± 0.54	11.88 ± 1.97
GLA	40.67 ± 3.41	41.12 ± 4.44	40.94 ± 2.03	47.85 ± 6.41
HAB	29.46 ± 3.89	28.37 ± 3.29	26.80 ± 1.76	27.05 ± 1.77
HEP	25.16 ± 5.55	23.25 ± 6.32	18.98 ± 4.43	20.40 ± 3.15
IRI	6.13 ± 2.68	6.40 ± 2.42	5.33 ± 1.78	6.53 ± 1.60
IRO	10.54 ± 2.16	11.34 ± 2.88	11.97 ± 2.16	11.57 ± 2.35
MON	6.99 ± 2.92	8.38 ± 10.36	14.26 ± 4.13	1.90 ± 3.44
PEN	2.49 ± 0.27	4.07 ± 1.14	2.35 ± 0.16	7.28 ± 0.43
PIM	27.53 ± 5.28	27.08 ± 2.91	24.95 ± 2.91	25.29 ± 1.99
SEG	14.80 ± 4.60	13.33 ± 3.99	10.69 ± 1.89	8.63 ± 1.15
TIC	4.82 ± 4.54	16.28 ± 0.52	25.09 ± 4.46	1.77 ± 0.40
VOT	4.51 ± 0.75	4.37 ± 1.25	5.01 ± 1.31	4.51 ± 0.67
WAV	20.29 ± 4.05	21.31 ± 3.68	15.96 ± 0.95	23.05 ± 1.15
WIN	11.68 ± 4.46	12.47 ± 2.91	9.64 ± 4.50	13.48 ± 2.48
YEA	46.41 ± 3.41	45.77 ± 1.93	44.58 ± 1.41	44.77 ± 1.52
ZOO	13.10 ± 3.59	15.21 ± 5.09	16.10 ± 7.09	12.64 ± 2.06

	AIC	BIC	CV	RIPPER	$\sum$
AIC	0	5	2	5	8
BIC	0	0	1	4	5
CV	3	4	0	6	8
RIPPER	4	4	3	0	6
$\sum$	6	9	3	6	

three model selection techniques try all three candidate models at each condition, they have significantly longer training time compared to the univariate RIPPER algorithm.

Table 6.13. The average and standard deviations of complexity of rulesets produced by AIC, BIC, CV, and RIPPER algorithm

	Omnivariate Rules			Pure Rules
Dataset	AIC	BIC	CV	RIPPER
BAL	17.0 ± 0.0	17.0 ± 0.0	29.3 ± 14.4	12.8 ± 5.1
BRE	13.5 ± 8.4	8.2 ± 2.7	15.3 ± 8.3	8.2 ± 2.4
BUP	10.6 ± 7.2	2.8 ± 1.9	12.6 ± 8.8	4.0 ± 2.1
CAR	88.7 ± 27.7	42.3 ± 26.0	149.8 ± 56.2	56.0 ± 23.1
DER	48.6 ± 10.0	21.6 ± 6.9	73.8 ± 12.1	16.6 ± 1.3
ECO	24.4 ± 6.1	35.9 ± 6.4	30.9 ± 5.5	12.8 ± 2.3
FLA	1.4 ± 1.9	3.0 ± 5.8	4.7 ± 14.9	0.6 ± 1.3
GLA	25.4 ± 7.7	25.0 ± 7.7	17.8 ± 8.9	4.6 ± 3.0
HAB	2.8 ± 4.2	2.4 ± 1.3	3.6 ± 3.4	1.8 ± 1.5
HEP	4.0 ± 3.8	2.6 ± 4.1	22.2 ± 22.3	1.2 ± 1.0
IRI	5.5 ± 1.6	4.7 ± 1.2	6.3 ± 3.2	4.2 ± 0.6
IRO	13.4 ± 7.3	9.8 ± 2.4	32.3 ± 15.9	4.8 ± 1.0
MON	36.5 ± 3.4	14.6 ± 6.7	19.9 ± 9.1	20.4 ± 4.0
PEN	518.2± 33.3	413.4 ± 31.1	776.9 ± 73.9	253.8 ± 20.6
PIM	7.7 ± 3.8	3.0 ± 1.4	27.9 ± 15.4	3.8 ± 2.4
SEG	64.5 ± 29.0	55.0 ± 9.1	111.0 ± 18.6	54.4 ± 6.6
TIC	72.4 ± 41.7	26.0 ± 0.0	73.3 ± 54.1	48.6 ± 1.9
VOT	3.0 ± 3.2	2.0 ± 0.0	4.7 ± 5.3	2.8 ± 2.5
WAV	47.8 ± 6.8	47.8 ± 5.5	404.6 ± 175.9	80.2 ± 15.3
WIN	6.6 ± 1.0	6.6 ± 1.9	17.9 ± 10.5	5.0 ± 1.4
YEA	38.2 ± 11.0	30.8 ± 11.6	85.5 ± 16.3	33.2 ± 6.7
ZOO	16.2 ± 3.6	29.9 ± 6.4	17.8 ± 11.8	8.4 ± 0.8

	AIC	BIC	CV	RIPPER	$\sum$
AIC	0	2	12	1	13
BIC	9	0	14	2	15
CV	2	1	0	0	3
RIPPER	15	6	16	0	18
$\sum$	16	6	17	2	

Again CV due to its cross-validation nature has the longest training time.

Table 6.14. The average and standard deviations of learning time (secs.) of rulesets produced by AIC, BIC, CV, and RIPPER algorithm

	Omnivariate Rules			Pure Rules
Dataset	AIC	BIC	CV	RIPPER
BAL	110 ± 35	113 ± 31	477 ± 107	0 ± 0
BRE	2 ± 0	2 ± 1	13 ± 3	0 ± 0
BUP	0 ± 1	0 ± 0	5 ± 1	0 ± 0
CAR	1070 ± 161	863 ± 151	4491 ± 1118	0 ± 0
DER	1148 ± 261	1688 ± 371	5352 ± 930	0 ± 0
ECO	1 ± 1	1 ± 0	9 ± 1	0 ± 0
FLA	83 ± 46	70 ± 47	260 ± 118	0 ± 0
GLA	2 ± 1	3 ± 1	13 ± 2	0 ± 0
HAB	0 ± 0	0 ± 0	1 ± 0	0 ± 0
HEP	20 ± 16	16 ± 10	116 ± 45	0 ± 0
IRI	0 ± 0	0 ± 0	1 ± 1	0 ± 0
IRO	933 ± 316	1091 ± 206	4310 ± 2057	0 ± 0
MON	0 ± 0	1 ± 0	3 ± 1	0 ± 0
PEN	805 ± 66	897 ± 103	2769 ± 383	48 ± 4
PIM	2 ± 1	2 ± 1	23 ± 4	0 ± 0
SEG	316 ± 52	306 ± 32	1337 ± 176	4 ± 0
TIC	1157 ± 408	1057 ± 201	3945 ± 1257	0 ± 0
VOT	533 ± 284	590 ± 261	2540 ± 646	0 ± 0
WAV	826 ± 76	837 ± 93	6194 ± 1701	5 ± 1
WIN	2 ± 1	3 ± 1	16 ± 1	0 ± 0
YEA	28 ± 2	27 ± 2	122 ± 8	3 ± 0
ZOO	12 ± 3	9 ± 3	32 ± 9	0 ± 0

	AIC	BIC	CV	RIPPER	$\sum$
AIC	0	3	22	0	22
BIC	2	0	21	0	21
CV	0	0	0	0	0
RIPPER	18	19	21	0	21
$\sum$	18	19	22	0	

The number of times univariate, multivariate linear and multivariate quadratic models selected in rulesets produced by AIC, BIC and CV model selection techniques

Table 6.15. The number of times univariate, multivariate linear and multivariate quadratic models selected in rulesets produced by AIC, BIC and CV

	AIC			BIC			CV		
	UNI	LIN	QUA	UNI	LIN	QUA	UNI	LIN	QUA
BAL	0	10	0	0	10	0	2	17	0
BRE	17	7	1	21	4	0	2	12	1
BUP	40	4	0	14	0	0	14	13	1
CAR	88	51	0	49	24	0	3	87	1
DER	58	13	0	108	0	0	30	24	2
ECO	83	12	0	148	10	0	44	21	6
FLA	7	0	0	15	0	0	1	0	1
GLA	123	1	0	121	1	0	38	14	0
HAB	14	0	0	12	0	0	10	4	0
HEP	20	0	0	13	0	0	6	3	5
IRI	20	3	0	19	2	0	16	3	2
IRO	52	1	0	49	0	0	15	10	3
MON	42	3	10	52	6	0	18	2	6
PEN	196	49	57	187	55	40	33	37	107
PIM	16	5	0	15	0	0	18	23	3
SEG	189	16	4	200	13	0	131	59	14
TIC	210	2	2	130	0	0	43	15	8
VOT	15	0	0	10	0	0	16	1	0
WAV	19	20	0	19	20	0	10	38	15
WIN	33	0	0	33	0	0	11	10	1
YEA	172	5	0	154	0	0	103	58	10
ZOO	81	0	0	86	10	0	57	1	3
$\sum$	1495	202	74	1455	155	40	621	452	189
%	84.41	11.40	4.17	88.18	9.39	2.42	49.21	35.81	14.98

are given in Table 6.15. Also the selection counts of those models as a function of the position of a condition in a rule for AIC, BIC and CV model selection techniques

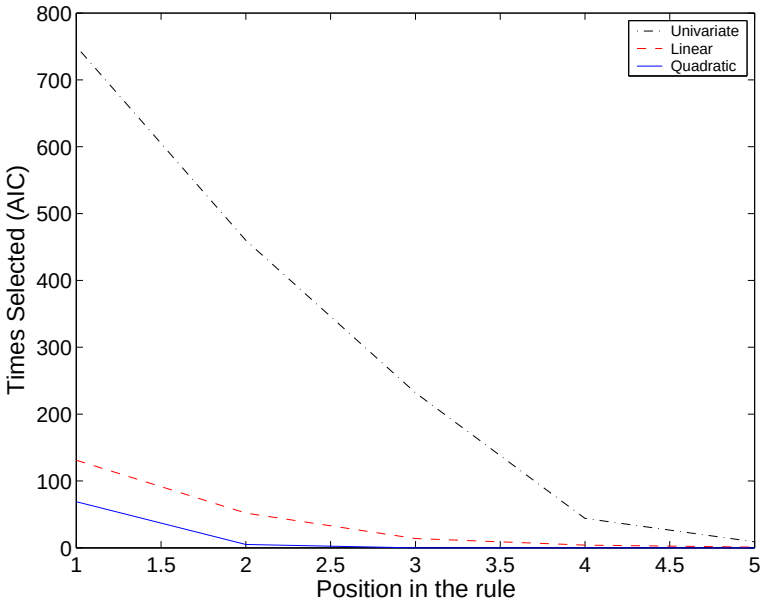


Figure 6.48. Number of times univariate, multivariate linear and multivariate quadratic models selected at different positions of condition in a rule for AIC model selection technique

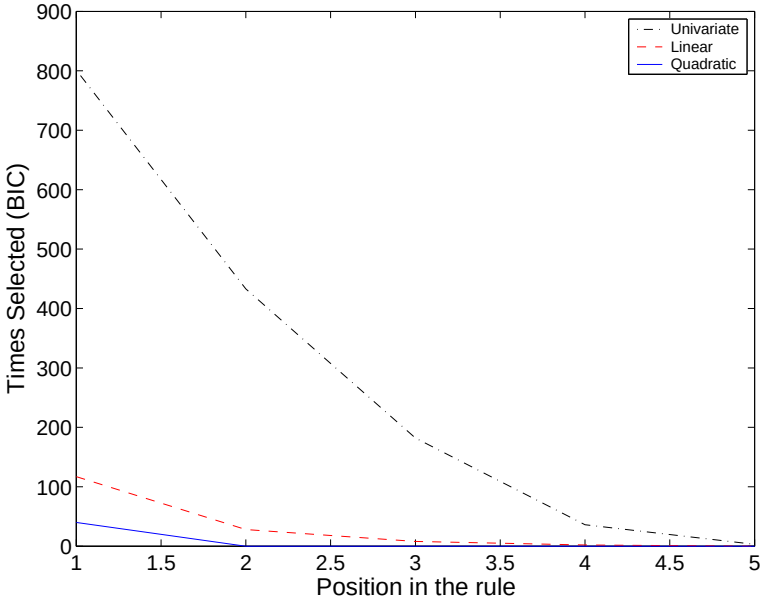


Figure 6.49. The bumver of times univariate, multivariate linear and multivariate quadratic models selected at different positions of condition in a rule for BIC model selection technique

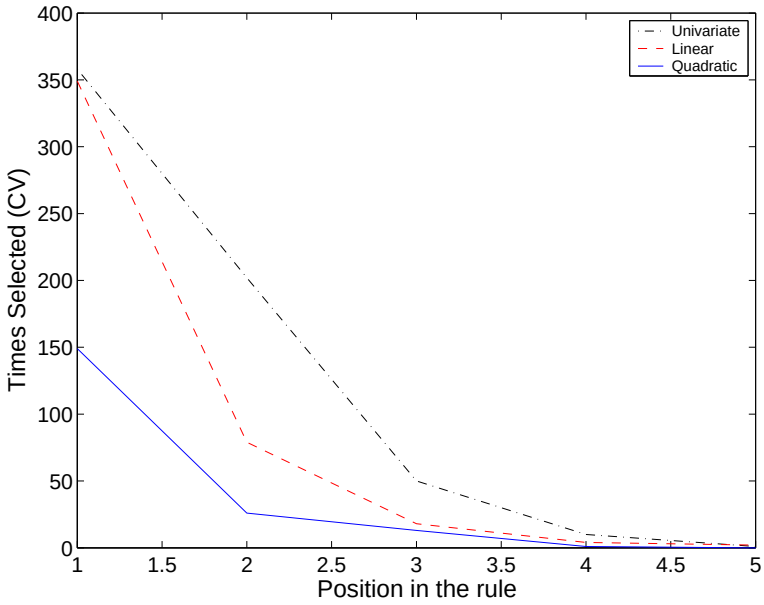


Figure 6.50. The number of times univariate, multivariate linear and multivariate quadratic models selected at different positions of condition in a rule for CV model selection technique

are given in Figures 6.48, 6.49, 6.50 respectively. We see that, as expected, quadratic models are selected least, then the linear models and the univariate models are selected most. CV has the smallest number of conditions but it selects both quadratic and linear models with the largest percentage (15 and 36 percents respectively). BIC selects simpler models slightly more frequently than AIC. For example, BIC selects quadratic models only in *pendigits*, whereas AIC selects the quadratic model in *monks*, *pendigits*, *segment* and *tictactoe*. If we look to the figures, we see that CV selects linear models as much as univariate models in the first position, it also selects quadratic models significantly (20 percent). As the position increases, the univariate models are selected as the best model (as expected). AIC and BIC are similar in that perspective; they select the univariate model as the best model more frequently compared to the linear and quadratic models.

6.5. Comparison of Tree Induction with Rule Induction

In this section, we want to compare the performances of model selection techniques on tree induction and rule induction. Figures 6.51 and 6.52 show expected error, model complexity and learning time plots of rules and trees generated by AIC, BIC and CV model selection techniques for different percentage levels of *pendigits* and *segment* respectively. AIC and BIC model selection techniques have smaller validation errors for tree induction compared to rule induction in both datasets. For CV, trees have larger validation errors compared to rules in *pendigits*, whereas trees have smaller validation errors compared to rules in *segment*. If we look at the complexity of the models, on *pendigits*, all three model selection methods produce trees with smaller complexity than their corresponding rules. On *segment*, there is not any significant difference between AIC and BIC's rules and trees, whereas with CV, trees again are simpler than rules. For all model selection techniques and on both datasets, trees have longer training time compared to rules.

The number of wins of each model selection technique in comparing their omnivariate rules and trees in twenty-two datasets is given in Table 6.16. The omnivariate rules of AIC model selection technique is better than the omnivariate trees of AIC in terms of expected error (9 wins, 5 losses). On the other hand, the reverse is true for BIC. Its trees are better than its rules (9 wins, 5 losses). The trees of CV have similar performance as the rules of CV. If we look to the total complexity, since AIC and BIC's model-selection pruning does not prune much, their trees are significantly more complex than their corresponding rules. For CV the reverse is true. Since CV selects many quadratic models in its rules, in 16 datasets out of 22, CV's trees are simpler than its rules and in none of the datasets, CV's rules are simpler than its trees. For all three techniques, rules have longer training time compared to trees.

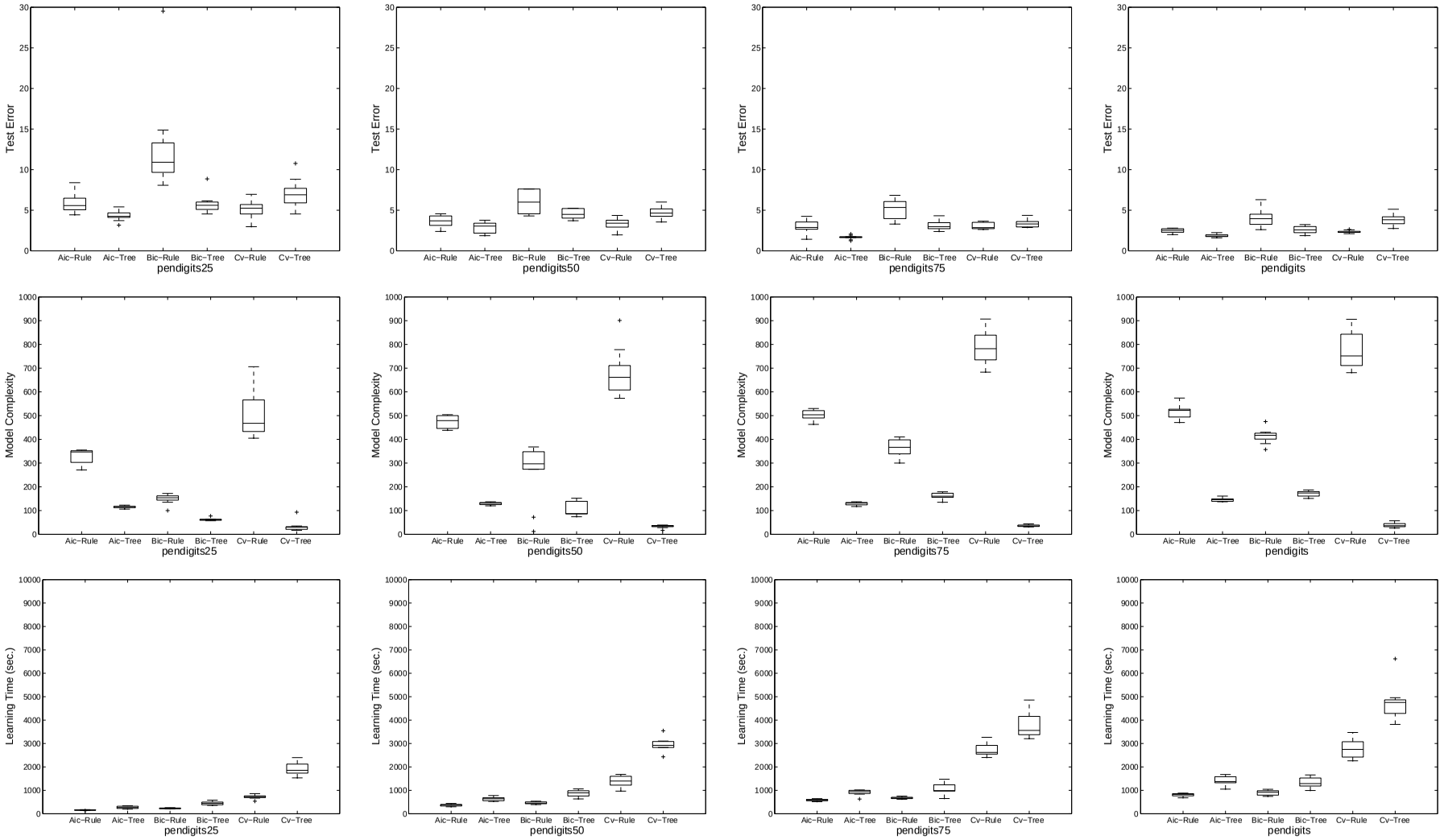


Figure 6.51. The expected error, model complexity and learning time plots of rules and trees generated by AIC, BIC and CV for different percentage levels of *pendigits*

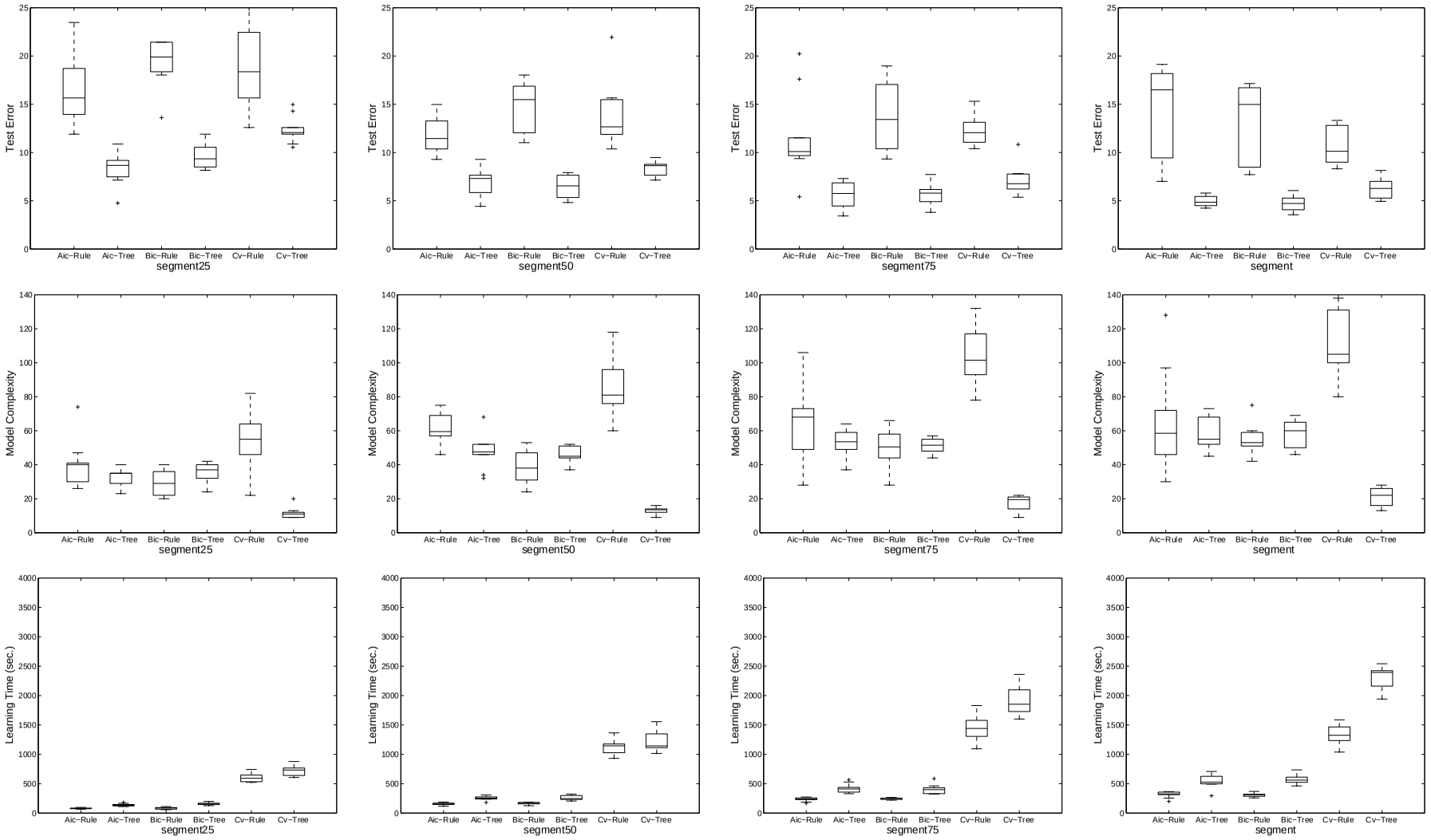


Figure 6.52. The expected error, model complexity and learning time plots of rules and trees generated by AIC, BIC and CV for different percentage levels of *segment*

Table 6.16. The number of wins of each model selection technique in comparing omnivariate rules and trees

	Error		Complexity		Learning Time	
Technique	Tree	Rule	Tree	Rule	Tree	Rule
AIC	5	9	3	14	9	5
BIC	9	5	5	12	10	7
CV	4	3	16	0	10	6

7. CONCLUSIONS AND FUTURE WORK

7.1. MultiTest Algorithm

In this thesis, we first introduce the MultiTest method which solves the problem of choosing the best of an arbitrary number of candidate supervised learning algorithms. Existing pairwise tests allow a comparison of two algorithms only; range tests and Anova check whether multiple methods have the same expected error and cannot be used for finding the smallest. We propose a methodology, the MultiTest algorithm, whereby we order supervised learning algorithms (for classification and regression) taking into account (i) the result of pairwise statistical tests on expected error (what the data tells us), and (ii) our prior preferences, e.g., due to complexity. We define the problem in graph-theoretic terms and propose an algorithm to find the "best" algorithm in terms of these two criteria, or in the more general case, order the algorithms in terms of their "goodness". Our proposed method can be used for 0/1 classification and mean-square error regression and can be generalized to other loss functions by using a suitable pairwise test.

We compared our MultiTest methodology with Anova, Newman-Keuls and a straightforward approach TestFirst on both classification and regression problems. We show that Anova test results can be converted to an ordering only if Anova test accepts; we order the algorithms in terms of our prior preference. But this case occurs rarely, in our experiments, in only two classification datasets out of thirty. Converting Newman-Keuls results to an ordering is possible in some cases but not always. For example, if there are groups of algorithms having similar expected error, there is a high possibility for Newman-Keuls having multiple overlapping underlines for which no ordering can be defined. There are similar problems with a straightforward approach such as the heuristic TestFirst algorithm which is not always able to choose a single algorithm as the best. These results show that a methodological approach as proposed by the MultiTest algorithm is necessary and that furthermore MultiTest is always able to find a best algorithm.

The MultiTest method we propose does not significantly increase the computational and/or space complexity because the real cost is the training part of the learning algorithms. Once the learning algorithms are trained and validated and these validation errors are recorded, applying the calculation necessary for MultiTest is simple in comparison. These validation errors need to be calculated for us to be able to do *any* statistical comparison, e.g., Anova, range test, etc., and is not a particular requirement of our proposed method.

7.2. The Combined 5×2 cv t test

We also propose a novel test, the combined 5×2 t test, to use as the one-sided test in MultiTest. Although there are statistical tests in the literature such as the 5×2 t test [37], or the 5×2 F test [43], we do not use them. The 5×2 t test uses only the difference of the first fold error values of the algorithms, and this is not robust due to the variance in folds. We replace this with a statistic that uses all ten differences of error values and is therefore more robust. The 5×2 cv F test uses square of the differences of error values and cannot be used for a one-sided test.

To validate our novel test, we compare it with the 5×2 cv t test on classification and regresssion datasets. We see that the combined 5×2 t test has higher power (lower type II error); that is, it rejects when there is a large difference between the error values of two algorithms. Similarly when there is not any significant difference between the error values of algorithm, the combined test accepts; that is, it has a lower probability of reject indicating that it has lower type I error.

7.3. Comparison of Model Selection Methods: MultiTest-based CV, AIC, BIC, SRM and MDL

By using the MultiTest method, we try to solve the optimal model complexity problem. For doing this, either we compare all possible models using MultiTest and select the best model or if the model space is very large, we make an effective search on the model space via MultiTest. If all possible models can be searched, MultiTest-based

model selection always selects the simplest model with expected error not significantly worse than any other model.

We first apply our MultiTest-based cross-validation model selection technique on a toy problem of polynomial regression. We also compare our technique with other model selection techniques such as AIC, BIC, MDL, and SRM on different sample sizes. Our proposed MultiTest-based cross-validation model selection technique usually selects simpler models than the other model selection methodologies for small and medium sample sizes. Our technique has larger expected error than the other model selection techniques but the difference is not significant. Therefore we can conclude that the Multitest-based cross-validation technique selects simpler models without sacrificing from expected error. For large sample sizes, as expected, the techniques come closer. Given enough training data, we do not need to rely on any a priori assumptions and therefore the assumptions we do make do not matter.

7.4. Omnivariate Architectures Using Model Selection

We propose a novel decision tree architecture, the omnivariate decision tree, which is a hybrid tree that contains both univariate, multivariate linear, and multivariate quadratic nodes. We use LDA to determine the weights of multivariate nodes. Our approach is not limited to having multivariate linear or multivariate quadratic models in the decision nodes, we can also use neural networks [93] and other architectures. The ideal node type is determined via model selection using MultiTest by taking into account our preference due to simplicity and the results of statistical tests comparing expected error. Such a tree, instead of assuming the same bias at each node, matches the complexity of a node with the data reaching that node. Our simulation results indicate that such an omnivariate architecture generalizes better than trees with the same types of nodes everywhere, and it also follows the best candidate model, resulting in smaller expected error. As expected, the quadratic model, the most complex model, is selected least, followed by the linear model and the univariate model. The quadratic model is mostly selected in the early levels of the tree where there is enough data. The only handicap is the longer training time but the test runs can be parallelized

very easily on a parallel processor system. We believe that with processors getting faster and cheaper, omnivariate architectures with built-in automatic model selection will become more popular as they free the designer from choosing the appropriate structure.

Rule induction is another application area of MultiTest-based cross-validation model selection technique. Rule induction is different from tree induction in the sense that the univariate rules are generated via separate-and-conquer compared to divide-and-conquer technique used in tree induction. To make a model selection in rule induction, we first create candidate models. We propose a novel multivariate rule induction algorithm where the rules consists of multivariate conditions. The multivariate model may be linear or quadratic and the weights are determined using LDA. Although multivariate conditions are more complex, rules can be generated by using a smaller number of them. Combining these candidate models, we propose a novel ruleset architecture, the omnivariate ruleset, which is a hybrid ruleset that contains both univariate, multivariate linear, and multivariate quadratic conditions in its rules. The three candidate models are compared using model selection with Multitest and the best model is selected. Our simulation results with omnivariate rules show that the omnivariate ruleset performs as well as the univariate ruleset produced by RIPPER on different data sets. Again the quadratic model is selected the least and the univariate model is selected the most. The quadratic conditions takes early positions in a rule where the complexity (and the size) of the data reaching the condition is high.

For both tree induction and rule induction, we also introduce omnivariate trees and rules where the model selection is done by AIC, BIC, MDL and SRM. Hence, we compare our proposed MultiTest-based cross-validation model selection technique with them not only on polynomial regression and classification but also in the context of tree induction and rule induction. AIC selects the most complex models in all the techniques which may result in overfitting. BIC punishes complex models more than AIC, especially when the sample size is large. This is explained easily with their generalization error estimation formulae. BIC has a $\log N$ term corresponding to 2 in AIC, therefore when $\log N > 2$, that is, when $N > 7.34$, BIC prefers simpler

models. There are two problems with AIC and BIC; estimating the noise variance σ^2 and finding the number of free parameters of a model. Their performance highly depend on the noise estimate in the context of regression. SRM performs well in terms of model complexity and it chooses and works fine for small sample sizes, but the parameters of SRM, a_1, a_2, c must be correctly tuned to get the best performance. Although SRM uses the VC-dimension to measure the complexity of an estimator, unfortunately, it is not possible to obtain an accurate estimate of the VC-dimension for some estimators. In the MDL technique, the λ parameter controls the amount of penalization for complexity, and it must be tuned well according to the encoding of the model. AIC, BIC and MDL try to estimate the generalization error, SRM tries to estimate the upper bounds of generalization error, and they have one or more parameters to tune. On the other hand, the MultiTest-based cross-validation model selection technique does not any have parameters to fine-tune and it finds the best model considering the expected error and the complexity.

7.5. Future Work

The MultiTest-based model selection technique presented in this thesis may also be applied to the model selection in other machine learning algorithms such as regression trees, neural networks, hidden Markov models.

In regression trees, we can apply our model selection technique by creating an omnivariate regression tree where at each node there are three candidate models, namely, univariate model, linear multivariate model and linear quadratic model. Then the candidate models at each node try to minimize the sum of mean square errors of child nodes. Our proposed combined 5×2 t test can be used for comparing the mean square errors of the candidate models.

The model selection in neural networks is defined as finding the structure (number of hidden layers, number of hidden nodes at each layer) of the neural network. We can effectively search the best neural network model in the model space via MultiTest method. We start searching from an initial neural network model and create candidate

models by applying operators such as adding or removing a hidden node, adding or removing a hidden layer etc. Then the initial model and the candidate models will be compared by MultiTest method considering the model complexities and expected errors of the models. MultiTest selects the best model and we continue search from that model. Since MultiTest method is independent from the search technique, we can also apply heuristic search techniques such as A^*.

APPENDIX A: PROBABILITY DISTRIBUTIONS

We first review probability distributions because,

- In interval estimation, the confidence interval of a parameter is calculated based on an assumption of the underlying probability distribution of this parameter.
- Hypothesis tests are accepted or rejected based on critical regions. According to the test statistic's position according to the critical region (test statistic is in the critical region or not), an hypothesis test will be accepted or rejected. These critical regions are calculated according to the underlying probability distribution of the test statistic.
- Statistical intervals and hypothesis tests are often based on specific assumptions about the distribution of the data.

Probability distributions can be categorized into two as discrete and continuous probability distributions. Each probability distribution is founded on a probability density function, which is the probability that the variable X coming from that probability distribution has the value of x. For discrete distributions, this value can be calculated as single point probability but for continuous distributions it has to be expressed in terms of an integral between two points.

Well-known distributions are

A.1. Uniform Distribution

Probability density function for the uniform distribution defined in the interval (a, b) is given as:

$$f(x) = \frac{1}{b-a} \tag{A.1}$$

If $b = 1$ and $a = 0$, it is called *standard uniform distribution.* The parameters, mean and variance are

$$E[X] = \frac{a+b}{2}$$
$$Var[X] = \sqrt{\frac{(b-a)^2}{12}} \tag{A.2}$$

The uniform distribution defines equal probability over a given range for a continuous distribution. For this reason, it is important as a reference distribution. One application of the uniform distribution is generating random numbers between 0 and 1.

A.2. Normal Distribution

The probability density function for normal distribution is denoted as N(μ, σ^2),

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \tag{A.3}$$

If $\mu = 0$ and $\sigma = 1$, it is called *standard normal distribution* denoted as Z. To convert normal distribution to standard normal distribution, we subtract the mean and divide to the square root of the variance. The parameters, mean and variance are,

$$E[X] = \mu$$
$$Var[X] = \sigma^2 \tag{A.4}$$

Many statistical tests are based on the assumption that the data comes from a normal distribution. Central limit theorem states that the sum of independent and identically distributed random variables approaches to normal distribution as the sample size becomes large (usually larger than 30). The sum of normally distributed random variables is also normally distributed. There is also a connection between random numbers generated using uniform distribution and normal distribution. By summing 12 randomly generated numbers and subtracting 6, we get a standard normally distributed

random variable.

A.3. Chi Square Distribution

The probability density function for chi square distribution is,

$$f(x) = \frac{e^{\frac{-x}{2}} x^{\frac{v}{2}-1}}{2^{\frac{v}{2}}\Gamma(\frac{v}{2})} \tag{A.5}$$

where v is the degree of freedom of the chi square distribution and Γ is the Gamma function. The parameters, mean and variance are,

$$\begin{aligned} E[X] &= v \\ Var[X] &= \sqrt{2v} \end{aligned} \tag{A.6}$$

The sum of squares of N normally distributed random variables is chi-square distributed with N degrees of freedom.

$$\chi_n = \mathcal{N}_1^2 + \mathcal{N}_2^2 + \ldots + \mathcal{N}_n^2 \tag{A.7}$$

The sum of N chi-square distributed random variables is also chi-square distributed. The sum of the degrees of freedoms of the N random variables is the degree of freedom of the resulting random variable.

$$\chi_{v_1+v_2+\ldots+v_n} = \chi_{v_1} + \chi_{v_2} + \ldots + \chi_{v_n} \tag{A.8}$$

A.4. t Distribution

If $Z \sim \mathrm{Z}$ and $X \sim \chi_v^2$ are independent, then

$$T = \frac{Z}{\sqrt{X/v}} \tag{A.9}$$

is t distributed with v degree of freedom. The parameters, mean and variance are,

$$\begin{aligned} E[X] &= 0 \\ Var[X] &= \sqrt{\frac{v}{v-2}} \end{aligned} \tag{A.10}$$

t distribution approaches to normal distribution as v gets larger.

A.5. F Distribution

If $X_1 \sim \chi^2_{v_1}$ and $X_2 \sim \chi^2_{v_2}$ are independent chi-square random variables with v_1 and v_2 degrees of freedom respectively,

$$F = \frac{X_1/v_1}{X_2/v_2} \tag{A.11}$$

is F distributed with v_1 and v_2 are degrees of freedom. The parameters, mean and variance are,

$$\begin{aligned} E[X] &= \frac{v_2}{v_2-2} \\ Var[X] &= \sqrt{\frac{2v_2^2(v_1+v_2-2)}{v_1(v_2-2)^2(v_2-4)}} \end{aligned} \tag{A.12}$$

If $v_2 = 1$, then F distribution value is equal to the square of the t distribution's value with v_1 degrees of freedom.

$$F_{v_1,1} = t^2_{v_1} \tag{A.13}$$

APPENDIX B: INTERVAL ESTIMATION AND STATISTICAL TESTS

B.1. Intervals

B.1.1. Interval Estimation

Point estimators find a single value for a parameter θ. In case of interval estimation, an interval is specified for parameter θ, where θ lies with a certain degree of confidence. In interval estimation, we use the assumed underlying probability distribution of the point estimator.

As an example, suppose that we want to estimate the mean μ of data x^t coming from normal distribution $\mathcal{N}(\mu, \sigma^2)$. From the property of the normal distribution, the sum of the examples in the sample $\sum_t x^t$ is also normally distributed. So the sample mean $m = \frac{\sum_t x^t}{N}$, is normally distributed with mean μ and a known variance of $\frac{\sigma^2}{N}$. Using the conversion from normal distribution to standard normal distribution, we get $\frac{\sqrt{N}(m-\mu)}{\sigma} \sim \mathcal{Z}$. Since $\mathcal{Z}$ lies with 95% confidence in the interval $(-1.96, 1.96)$,

$$P\{-1.96 < \frac{\sqrt{N}(m-\mu)}{\sigma} < 1.96\} = 0.95 \tag{B.1}$$

or equivalently

$$P\{m - 1.96\frac{\sigma}{\sqrt{N}} < \mu < m + 1.96\frac{\sigma}{\sqrt{N}}\} = 0.95 \tag{B.2}$$

Here m is a point estimator and $(m - 1.96\frac{\sigma}{\sqrt{N}}, m + 1.96\frac{\sigma}{\sqrt{N}})$ is an interval estimator for the unknown mean μ. By using interval estimation we can say that with 95 degrees of confidence, μ lies in the interval $(m - 1.96\frac{\sigma}{\sqrt{N}}, m + 1.96\frac{\sigma}{\sqrt{N}})$.

B.1.2. Hypothesis Testing

In hypothesis testing, we first set up a hypothesis on a certain parameter, then we check using a test statistic which, if the hypothesis (called the null hypothesis) H_0 holds, should be in a given interval according to a given confidence level. If the sample is consistent with the given hypothesis, then we say that the hypothesis is accepted, otherwise it is rejected. For example, if our hypothesis is that the sample mean is equal to a given value of μ_0, then the null hypothesis is

$$H_0 : \mu = \mu_0$$

and the alternative hypothesis is

$$H_1 : \mu \neq \mu_0$$

If we reject when the hypothesis is correct, this is called *type I error*. Similarly, if we accept a hypothesis when it is not correct, this is called *type II error*. The *power* of a test is defined as the probability of rejecting the null hypothesis when it is not correct. We want hypothesis tests to have small type I and type II errors and large power.

If we continue from this example, to test H_0, we say that if our point estimator m is not too far from μ_0, we can accept H_0, otherwise we reject it. For scientific purposes, we must give a mathematical definition for being not too far. We know from section B.1.1 that $\frac{\sqrt{N}(m-\mu)}{\sigma}$ lies in $(-z_{\alpha/2}, z_{\alpha/2})$ with a probability of α. To test the hypothesis, we first set the confidence level α then check if the calculated μ_0 is in that interval. If μ_0 is in the interval we accept the null hypothesis, i. e., we conclude that $\mu = \mu_0$, otherwise we reject the null hypothesis and conclude that they are not equal.

If the hypothesis tests check for equality such as $\mu = \mu_0$, these tests are called *two-sided tests*. If they test for inequalities such as $\mu \leq \mu_0$, then they are called *one-sided tests*. The name comes from the bound of the confidence intervals. In two-sided tests, confidence intervals are bounded from $-\infty$ to $+\infty$, whereas in one-sided tests it

is bounded either from $-\infty$ to a number or from a number to $+\infty$. Because of this difference, to get a total confidence level of α, in two sided tests, we set the confidence level to $\alpha/2$ whereas in one sided tests, we set the confidence level to α.

B.2. Confidence Level

B.2.1. Bonferroni Correction

We want to get α level of confidence for hypotheses $H_1, H_2, \ldots, H_n$ simultaneously. If we set the significance level of each hypothesis to α, according to the law of independent trials, the probability that all hypothesis are true is $(1-\alpha)^n$, which is not significant. For example, if $\alpha = 0.05$ and if we have 5 hypotheses, then the probability is $0.95^5 = 0.77$. To get an overall α level of confidence, Bonferroni correction is applied. In Bonferroni correction, we set the significance levels of each hypothesis to $\frac{\alpha}{n}$. In our example, we will set the significance levels of each hypotheses to $\frac{0.05}{5} = 0.01$ to get an overall probability of $0.99^5 = 0.95$.

B.2.2. Holm's Correction

Although Bonferroni correction is well suited for our purposes of getting α level of confidence for hypotheses $H_1, H_2, \ldots, H_n$ simultaneously, we must set very high significance levels to each hypothesis. In that case, we may reject hypotheses which have high significance levels individually. In Holm's correction, all hypotheses are ordered in increasing order of significance level: the most significant hypothesis is $H_{(1)}$, the least significant hypothesis is $H_{(n)}$. Holm's correction iterates through the hypotheses, where at step $i, i = 1, \ldots, n$ the significance level α_i of hypothesis $H_{(i)}$ is compared with $\alpha/(n-i+1)$, and

- if $\alpha_i \leq \alpha/(n-i+1)$ then the hypothesis $H_{(i)}$ is rejected and we go to the next step,
- if $\alpha_i > \alpha/(n-i+1)$ then the hypotheses $H_{(i)}, H_{(i+1)}, \ldots, H_{(n)}$ are accepted and iteration stops.

B.3. Statistical Tests

Statistical tests in the literature can be divided into two groups as parametric and nonparametric. In parametric methods, there is an assumption on the underlying distribution of the data. Most parametric methods are based on a normality assumption. In contrast to parametric methods, nonparametric methods do not assume a particular population probability distribution which makes them applicable in any population. When the data is discrete, we can only use nonparametric methods.

The second type of grouping of statistical tests divides tests into three parts. Pairwise tests, range tests and multiple tests. These types of tests differ from each other on the number of populations they compare. Pairwise comparison methods compare two populations at a time, multiple comparison methods can compare $L > 2$ populations at once and range tests can compare any number of populations.

For each of the tests we say that, we have L populations each having N instances. So we have $L \times N$ instances and let Y_{ij} represents the instance i of population j. Each population has a mean of $m_i, i = 1, 2, \ldots, L$. All of the populations have a general mean of m. For parametric tests we assume that, all of the L samples are independent and normally distributed. There is no such an assumption for nonparametric tests.

In the following sections we will give the methods, their hypotheses, definitions, assumptions, derivations of the tests stated above. We will also give an application example for each test. When we compare populations by using statistical tests, we will use the data in Table 2. This data contains four populations and there are 10 instances for each population.

B.3.1. One-Way Anova Test

Hypothesis. The null hypothesis of this test is

$$H_0 : m_1 = m_2 = \ldots = m_L$$

Table B.1. Example instances of four populations

Population1	Population2	Population3	Population4
9	15	18	21
11	13	16	19
10	17	15	17
13	14	17	23
8	15	13	18
12	16	15	20
10	15	14	16
11	17	17	21
7	14	15	21
10	13	16	19

against the alternative

H_1: At least one of the samples has a larger or smaller mean than at least one of the other samples

Method. The method starts by calculating MST as

$$MST = N\frac{\sum_{i=1}^{L}(m_i - m)^2}{L-1} \tag{B.3}$$

This is the total standard error between the means of the populations. We calculate MSE as

$$MSE = \frac{\sum_{i=1}^{L}\sum_{j=1}^{N}(Y_{ij} - m_i)^2}{LN - L} \tag{B.4}$$

which is total mean square error. If $\frac{MST}{MSE}$ ratio is smaller than the F distribution value with $L-1$ and $LN-L$ degrees of freedom, then H_0 is accepted. Otherwise it is rejected.

Assumptions.

- All populations are independent and composed of normal random variables with unknown mean μ_i and unknown variance σ^2.

Derivation. This test compares two estimators of σ^2 which are calculated using between-group sum of squares and within-group sum of squares.

If the hypothesis is true, $Y_{ij} \sim N(\mu, \sigma^2)$, then the average of the sample i is $m_i = \frac{\sum_{j=1}^{N} Y_{ij}}{N} \sim N(\mu, \frac{\sigma^2}{N})$.

The first estimator is $MST = \frac{\sum_{i=1}^{L}(m_i - m)^2}{L-1}$. Since m_i and m are normal random variables, their difference is a normal random variable. The square of the difference then will be a chi-square variable with one degree of freedom. Sum of such L chi-square distributed variable will be chi-square distributed with L degrees of freedom. But since we can calculate m from m_i's, we must subtract one degree of freedom. So $MST \sim \chi^2_{L-1}$.

Second estimator is MSE. Since Y_{ij} and m_i's are normal random variables, their difference is normal and their square is chi-square distributed. Sum of N such chi-square distributed variables is also chi-sqaure distributed with $N-1$ degrees of freedom. Like in MST, m_i could be calculated using Y_{ij}, therefore we must subtract 1 degree of freedom $(N-1)$. Since sum of chi-squares is also a chi-square, if we sum up ${Y_{ij} - m_i}^2$ for all i, MSE is chi-distributed with $L \times (N-1)$ degrees of freedom.

We take the ratio of these two estimators. The ratio of two independent chi-square random variables divided by their respective degrees of freedom is a random variable that is F distributed with degrees of freedoms of those chi-square random variables' degrees of freedoms. So the ratio is F distributed with $L-1$ and $LN-L$ degrees of freedoms.

Example. The means of four populations are; m_1=10.1, m_2=14.9, m_3=15.6 and m_4=19.5. The overall mean is, m=15.05. MST is $10\frac{(10.1-15.05)^2+(14.9-15.05)^2+(15.6-15.05)^2+(19.5-15.05)^2}{4-1} = 148.76$. MSE is $\frac{(9-10.1)^2+...+(15-14.9)^2+...+(18-15.6)^2+...+(21-19.5)^2+...}{40-4}$=3.02. So we compare the F distribution value with 3 and 36 degrees of freedom (8.06) with $\frac{148.76}{3.02} = 49.26$. We see that F distribution value is much smaller. With 99.99 percent of confidence, we reject the null hypothesis.

B.3.2. K-Fold Crossvalidated Paired t Test

The name of this test comes from the fact that first it uses K-fold cross-validation (there are K instances for each population $N = K$), second it pairs instances of the populations and third it uses t distribution.

Hypothesis. The null hypothesis of this test is

$$H_0 : m_1 = m_2$$

against the alternative

H_1: One of the populations has larger or smaller mean than the other.

Method. p_i is the difference between the instances of the two populations as $p_i = Y_{1i} - Y_{2i}$.

The estimator for mean

$$m = \frac{\sum_{i=1}^{N} p_i}{N} \tag{B.5}$$

and variance

$$S^2 = \frac{\sum_{i=1}^{N}(p_i - m)^2}{N-1} \tag{B.6}$$

We calculate the test statistic

$$T = \frac{\sqrt{N}m}{S} \tag{B.7}$$

The null hypothesis is accepted with α level of confidence, if this value is between $-t_{\alpha/2,N-1}$ and $t_{\alpha/2,N-1}$, otherwise it is rejected.

Assumptions.

- The difference between the instances of the two populations is approximately normally distributed with zero mean and unknown variance σ^2.

Derivation. If a sample with N examples is normally distributed, then

$$\frac{\sqrt{N}(m-\mu)}{S} \tag{B.8}$$

is t distributed with $N-1$ degrees of freedom. Since there are N normally distributed p_i's (assumption) and we test $\mu = 0$

$$\frac{\sqrt{N}m}{S} \tag{B.9}$$

is t distributed with $N-1$ degrees of freedom.

Example. We will compare the first population with the second population. First we calculate p_i's. $p_1 = 9 - 15 = -6, p_2 = 11 - 13 = -2, \ldots, p_{10} = 10 - 13 = -3$.

Second we calculate the means of p_i's as $m = \frac{\sum_{i=1}^{N} p_i}{N} = \frac{-6-2+...-3}{10} = -4.8$. Third we calculate the variance S as $S = \frac{\sum_{i=1}^{N}(p_i-m)^2}{N-1} = \frac{(-6+4.8)^2+...+(-3+4.8)^2}{9} = 4.31$. Finally we calculate the test statistic as $T = \frac{\sqrt{N}m}{S} = \frac{\sqrt{10}-4.8}{4.31} = -3.52$. We compare this number with $-t_{0.025,9}$ and $t_{0.025,9}$, which are -2.26 and 2.26. Since the number -3.52 is outside this inverval, we conclude that the first and the second populations do not have the same mean.

B.3.3. 5×2 cv t Test

This test is proposed by Dietterich [37]. In this test five replications of 2–fold cross–validation are performed (N = 10).

Hypothesis. The null hypothesis of this test is

$$H_0 : m_1 = m_2$$

against the alternative

H_1: One of the populations has larger or smaller mean than the other.

Method. $p_i^{(j)}$ is the difference between the instances of the two populations as $p_i^1 = Y_{1(2i-1)} - Y_{2(2i-1)}$ and $p_i^2 = Y_{1(2i)} - Y_{2(2i)}$. s_i^2 is the estimated variance of the i'th replication as $s_i^2 = (p_i^{(1)} - \bar{p_i})^2 + (p_i^{(2)} - \bar{p_i})^2$.

Total variance is then

$$VAR = \sum_{j=1}^{5} s_i^2 \tag{B.10}$$

Now the null hypothesis H_0 is accepted with α level of confidence if the value $\frac{p_i^1}{\sqrt{\frac{VAR}{5}}}$

is between $-t_{\alpha/2,5}$ and $t_{\alpha/2,5}$. Otherwise it is rejected.

Assumptions.

- The difference of the instances of the two populations is treated as approximately normally distributed with zero mean and unknown variance σ^2.
- $p_i^{(1)}$ and $p_i^{(2)}$ are independent normals.
- Each of the estimated variances s_i^2 are assumed to be independent.
- $p_i^{(1)}$ and the sum of the variances of the differences are independent of each other.

Derivation. According to the assumption, the difference of the instancs of two populations is unit normal, their square is chi-square distributed with 1 degree of freedom. From our second assumption, we can say that s_i^2/σ^2 is chi-square distributed with one degree of freedom. When we sum up 5 s_i's, which are 5 chi-square distributions with one degree of freedom, we will get a chi-square distribution with 5 degrees of freedom.

If Z is normally distributed and X is chi-square distributed with n degrees of freedom and if Z and X are independent, then $T_n = \frac{Z}{\sqrt{\frac{X}{n}}}$ is t distributed with n degrees of freedom. Since p_i^1 is normally distributed, sum of s_i's (VAR) is chi-square distributed and according to the fourth assumption both are independent, then $\frac{p_i^1}{\sqrt{\frac{VAR}{5}}}$ is t-distributed with 5 degrees of freedom.

Example. We will compare the first population with the second population. First we calculate $p_i^{(j)}$'s. $p_1^{(1)} = 9 - 15 = -6, p_1^{(2)} = 11 - 13 = -2, \ldots, p_5^{(2)} = 10 - 13 = -3$. $\bar{p_1} = \frac{p_1^{(1)} + p_1^{(2)}}{2} = -4, \ldots, \bar{p_5} = \frac{p_5^{(1)} + p_5^{(2)}}{2} = -5$. From $\bar{p_i}$'s and $p_i^{(j)}$'s we calculate VAR = 78.0 and the test statistic $\frac{6}{\sqrt{\frac{78}{5}}} = 1.52$. The confidence interval $(-t_{\alpha/2,0.025}, t_{\alpha/2,0.025})$, which is (-2.571, 2.571) contains 1.52. Therefore the null hypothesis is accepted.

B.3.4. 5×2 cv F Test

This test is proposed by Alpaydın [43]. It is a variant of the previous version 5×2 cv t test. This test has lower Type I error and higher power.

Hypothesis. The null hypothesis of this test is

$$H_0 : m_1 = m_2$$

against the alternative

H_1: One of the populations has larger or smaller mean than the other.

Method. $p_i^{(j)}$ is the difference between the instances of the two populations as $p_i^1 = Y_{1(2i-1)} - Y_{2(2i-1)}$ and $p_i^2 = Y_{1(2i)} - Y_{2(2i)}$. s_i^2 is the estimated variance for each of the five replication as $s_i^2 = (p_i^{(1)} - \bar{p}_i)^2 + (p_i^{(2)} - \bar{p}_i)^2$.

To test this, first we calculate sum of squares of $p_i^{(j)}$'s:

$$DIFFS = \sum_{i=1}^{5} \sum_{j=1}^{2} (p_i^{(j)})^2 \tag{B.11}$$

Second we calculate sum of variances of five replications

$$VAR = 2 \sum_{j=1}^{5} s_i^2 \tag{B.12}$$

Now the null hypothesis H_0 is accepted with α level of confidence, if $\frac{DIFFS}{VAR}$ is smaller than the probability value of F distribution with degrees of freedoms 10, 5 and α level of confidence.

Assumptions.

- The difference of the instances of two populations, is treated as approximately normally distributed with zero mean and unknown variance σ^2.
- $p_i^{(1)}$ and $p_i^{(2)}$ are independent normals.
- Each of the estimated variances s_i^2 are assumed to be independent.
- The sum of squares of the differences and sum of the variances of the differences are independent of each other.

Derivation. Since we assume that the difference of the instances of two populations are unit normal, their square is chi-square distributed with one degree of freedom. Therefore the sum of the squares, DIFFS is chi-square with 10 degrees of freedom. From our second assumption, we can say that s_i^2/σ^2 is chi-square distributed with one degree of freedom. When we sum up 5 s_i's, 5 chi-square distributions with one degree of freedom, we get a chi-square distribution with 5 degrees of freedom.

The ratio of two independent chi-square random variables divided by their respective degrees of freedom is a random variable that is F distributed with degrees of freedoms of those chi-square random variables freedoms. So the ratio of DIFFS to VAR is F distributed with 10 and 5 degrees of freedoms, assuming that DIFFS and VAR are independent.

Example. We will compare first and second populations. First we calculate $p_i^{(j)}$'s. $p_1^{(1)} = 9 - 15 = -6, p_1^{(2)} = 11 - 13 = -2, \ldots, p_5^{(2)} = 10 - 13 = -3$. So DIFFS, $DIFFS = {p_1^{(1)}}^2 + {p_1^{(2)}}^2 + \ldots + {p_5^{(2)}}^2 = 274.0$. $\bar{p_1} = \frac{p_1^{(1)} + p_1^{(2)}}{2} = -4, \ldots, \bar{p_5} = \frac{p_5^{(1)} + p_5^{(2)}}{2} = -5$. From $\bar{p_i}$'s and $p_i^{(j)}$'s we calculate VAR = 78.0 and the test statistic $\frac{DIFFS}{VAR} = 3.51$. When this number is compared with F distribution value with 10 and 5 degrees of freedom and 0.95 level of confidence, the null hypothesis is accepted.

B.3.5. Newman-Keuls Test

Hypothesis. The null hypothesis of this test is that all means in a subset of P means are equal

$$H_0 : m_1 = m_2 = \ldots = m_P$$

against the alternative

H_1: At least one of the samples has a larger or smaller mean than at least one of the other samples in a subset P samples of L samples of data.

If H_0 is rejected, we make comparisons on all the means and find which means are significantly different from each other.

Method. The basic idea in multiple range tests is comparing the best and the worst of the subset of populations selected. If they are not significantly different from each other, then all of the populations between this two methods are not significantly different.

In Newman-Keuls test, we first put the population means in increasing order. We then test for equality of L means $(m_1, \ldots, m_L)$. If they are equal we stop. Otherwise we check for equality of $L-1$ means $((m_1, \ldots, m_{L-1}), (m_2, \ldots, m_L))$, ..., and 2 means $((m_1, m_2),(m_2, m_3),\ldots,(m_{L-1}, m_L))$. If P means are equal to each other, then we do not need to make any further comparisons between these means. So we do not have to control any of the subsets of the means between m_i and m_j, when m_i and m_j are not significantly different from each other.

The comparisons of m_i and m_j is done by subtracting the two means and comparing the difference with $q^{0.95}_{L(N-1),P}\sqrt{\frac{MSE}{L}}$, where P is the number of means in the

Table B.2. 5 percent critical points for p means

	2	3	4
critical points	2.87	3.46	3.82
multiplied	2.49	3.01	3.32

subset compared, $q^{\alpha}_{f_1,f_2}$ is the studentized range distribution with α confidence and f_1, f_2 degrees of freedom. If the difference is smaller than the calculated number we accept the null hypothesis, otherwise we reject it.

Assumptions.

- Y_{1i} and Y_{2i} are normally distributed random variables with means μ_i and a common variance σ^2.

Derivation. This test wholely depends on Tukey's test. From L populations a subset of P populations are selected. We can say that the P population have different means only if in the prior stages of the algorithm they are declared different.

Example. We want to order populations 1 through 4. MSE is calculated in the same way as in one-way anova, so it is 3.02. We get the multiplier $\sqrt{\frac{MSE}{L}} = \sqrt{3.02/4} = 0.869$. The 5 percent critical points for measuring a group of P means, $P = 2, 3, 4$ and multiplication of it by 0.869 are given in Table 2. The means of populations are, m_1=10.1, m_2=14.9, m_3=15.6 and m_4=19.5.

The method is applied step by step in Table B.3. We order the populations in increasing order of error rate. First we compare four populations at a step. There is only one four population comparison namely first population with fourth population. Their mean difference is larger than the critical point for four populations (3.32). Second we compare three populations. There are two possible comparisons; first and

Table B.3. Newman-Keuls method applied for example

P	Current comparison
4	$m_4 - m_1 = 19.5 - 10.1 = 9.4 > 3.32$; draw no line, they are not the same
3	$m_3 - m_1 = 15.6 - 10.1 = 5.5 > 3.01$; draw no line, they are not the same
3	$m_4 - m_2 = 19.5 - 14.9 = 4.6 > 3.01$; draw no line, they are not the same
2	$m_2 - m_1 = 14.9 - 10.1 = 4.8 > 2.49$; draw no line, they are not the same
2	$m_3 - m_2 = 15.6 - 14.9 = 0.7 < 2.49$; underline, they are the same
2	$m_4 - m_3 = 19.5 - 15.6 = 3.9 > 2.49$; draw no line, they are not the same

third populations; second and fourth populations. In both cases, their mean difference is larger than the critical point for three populations (3.01). Third we compare two populations. There are three possible comparisons; first and second populations, second and third populations and third and fourth populations. Only in one of them, the difference between second and third populations is smaller than the critical point for two populations (2.49). Therefore we underline both populations.

So we conclude that the first population is significantly worse than the second population and the third population (they are the same), both of them are significantly worse than the fourth population.

B.3.6. Sign Test

The sign test is one of the nonparametric oldest tests. It was first used in 18th century to compare the number of males born with the number of females born in 82 years in England. The aim of the sign test is to check for equality of the number of positive examples with the number of negative examples. If one can separate the data into two groups of examples, sign test could be applied.

Hypothesis. The hypothesis of this test is that the number of positive examples are equal to the number of negative examples.

$$H_0 : P(+) = P(-)$$

against the alternative

$$H_1 \colon P(+) \neq P(-).$$

Method. The data consists of observations of a bivariate random sample (X_1, Y_1), (X_2, Y_2), ..., (X_n, Y_n), where there are N examples. Within each pair (X_i, Y_i) a comparison is done and each pair is classified as '+' if $X_i > Y_i$, '–' if $X_i < Y_i$ and 0 if $X_i = Y_i$. Number of positive examples are represented as P(+), number of negative examples are represented as $P(-)$ and total number of examples are represented as $P(+) + P(-) = N$. The test statistic uses standard normal distribution with $\alpha/2$ level of confidence. Test statistic t is given as:

$$t = \frac{1}{2}(n + z_{\alpha/2}\sqrt{n}) \tag{B.13}$$

If $P(+)$ is between t and $N - t$, then the null hypothesis is accepted, otherwise the null hypothesis is rejected.

Assumptions.

- The bivariate random variables (X_i, Y_i) are mutually independent.
- Each pair (X_i, Y_i) may be represented as positive, negative or 0.

Derivation. Since $P(+) + P(-) = N$, saying in the null hypothesis P(+) = P(-) is equal to saying that the probability of the positive class is 1/2. If the bivariate random variables are independent, the probability of having K positive examples in a sample with N examples is equal to the $\frac{\binom{N}{K}}{2^N}$. To be able to reject the null hypothesis

with α level of confidence, we must find such a number t that, the probability of having 0 to t positive examples will be $\alpha/2$. So the test statistic t is calculated as:

$$\frac{\sum_{i=0}^{t} \binom{n}{i}}{2^n} = \alpha/2 \tag{B.14}$$

For $N > 20$, using normal distribution we can approximate the t value by $\frac{1}{2}(n+z_{\alpha/2}\sqrt{n})$.

Example. Let's say we have compared two algorithms with each other in 30 datasets. According to the results of the pairwise comparisons, the first algorithm has better score in 13 datasets out of 30 datasets, whereas the second algorithm has better score in 17 datasets out of 30. Can we say that the two algorithms have similar performance?

To answer this question, we can use the sign test. $N = 30$ and let P(+) denotes the number of times the first algorithm won and P(-) denotes the number of times the second algorithm won. We may use the normal approximation of the t value (30 is larger than 20) and calculate the test statistic $t = \frac{1}{2}(30+z_{\alpha/2}\sqrt{30}) = \frac{1}{2}(n+-1.96\sqrt{30}) = 9.63$. Since P(+) = 13 is between $t = 9.63$ and $n-t = 20.37$, we can accept the null hypothesis and say that both algorithms have similar performance.

APPENDIX C: METRICS USED IN RULE INDUCTION

C.1. Information Gain

This measure of information gained from a particular split is popularized in [56]. Quinlan takes the famous entropy formula of Information Theory, which is the minimum number of bits to encode the classification of an arbitrary member of a collection S. So if we transform this idea into our problem, the entropy of node t is

$$Entropy(t) = \sum_{i=1}^{c} -p_i log_2 p_i \tag{C.1}$$

where p_i is the occuring probability of class i in the node t.

C.2. Rule Value Metric

We will mention two different rule value metrics proposed in the literature. Fürnkranz and Widmer [70] proposed that

$$v = \frac{p + (N - n)}{P + N} \tag{C.2}$$

Cohen [73] argues that this rule metric does not favor predictive conditions, and proposes

$$v = \frac{p - n}{p + n} \tag{C.3}$$

In both of these metrics, P (respectively N) represents the total number of positive (negative) examples in the whole data, p (respectively n) represents the total number of positive (negative) examples covered by the rule.

C.3. Minimum Description Length

Minimum Description Length is used to encode a set of examples given a theory. The description length of the examples is the sum of two components. First we find the number of bits to encode the theory, second we find the number of bits to encode the exceptions, which can not be handled using the theory.

The number of bits to encode the theory (number of bits required to encode a rule)

$$S = ||k|| + k\ log_2\frac{n}{k} + (n-k)\ log_2\frac{k}{n-k} \tag{C.4}$$

where k is number of conditions in the rule, n is the number of possible conditions for that data, $||k||$ is the number of bits required to send the integer k.

The number of bits to encode the exceptions is

$$\begin{aligned} & log_2(|D|+1) \\ & +fp \times (-log_2(\frac{e}{2C})) \\ & +(C-fp) \times (-log_2(1-\frac{e}{2C})) \\ & +fn \times (-log_2(\frac{fn}{2U})) \\ & +(U-fn) \times (-log_2(1-\frac{fn}{U})) \end{aligned} \tag{C.5}$$

where C (respectively U) is the number of covered (uncovered) examples, fp (respectively fn) is the number of false positives (negatives), and e is the number of exceptions ($e = fp + fn$) in the data.

APPENDIX D: DATASETS

Table D.1. Description of the classification datasets. K is the number of classes, N is the dataset size, and d is the number of inputs

Set	K	N	d	Missing	Attr Type
balance	3	625	5	No	symbolic
breast	2	699	10	Yes	numeric
bupa	2	345	7	No	numeric
car	4	1728	7	No	symbolic
cmc	3	1473	10	No	mixed
credit	2	690	16	No	mixed
cylinder	2	541	36	Yes	mixed
dermatology	6	366	35	Yes	numeric
ecoli	8	336	8	No	numeric
flare	3	323	11	No	mixed
glass	7	214	10	No	numeric
haberman	2	306	4	No	numeric
hepatitis	2	155	20	Yes	numeric
horse	2	368	27	Yes	mixed
iris	3	150	5	No	numeric
ironosphere	2	351	35	No	numeric
monks	2	432	7	No	numeric
mushroom	2	8124	23	Yes	symbolic
nursery	5	12960	9	No	symbolic
optdigits	10	3823	64	No	numeric
pendigits	10	7494	16	No	numeric
pima	2	768	9	No	numeric

Table D.1. continued

Set	K	N	d	Missing	Attr Type
segment	7	2310	19	No	numeric
spambase	2	4601	58	No	numeric
tictactoe	2	958	10	No	symbolic
vote	2	435	17	Yes	symbolic
wave	3	5000	22	No	numeric
wine	3	178	14	No	numeric
yeast	10	1484	9	No	numeric
zoo	7	101	17	No	numeric

Table D.2. Description of the regression datasets. N is the dataset size, and d is the number of inputs

Set	N	d	Missing	Attr Type
abalone	4177	8	No	Numeric
add10	9792	11	No	Numeric
bank8fm	8192	9	No	Numeric
bank8nm	8192	9	No	Numeric
boston	506	14	No	Numeric
comp	8192	22	No	Numeric
kin8fm	8192	9	No	Numeric
kin8nm	8192	9	No	Numeric
puma8fm	8192	9	No	Numeric
puma8nm	8192	9	No	Numeric
sine	8000	2	No	Numeric

REFERENCES

1. Hastie, T., R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, Springer Verlag, New York, 2001.

2. Craven, P. and G. Wahba, "Smoothing Noisy Data with Spline Functions", *Numerical Mathematics*, Vol. 31, pp. 377–403, 1979.

3. Moody, J., "The Effective Number of Parameters: An Analysis of Generalization and Regularization in Nonlinear Learning Systems", *Proceedings of Neural Information Processing Systems*, 1992.

4. Cherkassky, V. and Y. Ma, "Comparison of Model Selection for Regression", *Neural Computation*, Vol. 15, pp. 1691–1714, 2003.

5. Akaike, H., "Information Theory and an Extension of the Maximum Likelihood Principle", *Second International Symposium on Information Theory*, pp. 267–281, 1973, `citeseer.ist.psu.edu/31739.html`.

6. Schwarz, G., "Estimating the Dimension of a Model", *Annals of Statistics*, Vol. 6, pp. 461–464, 1978.

7. Madigan, D. and A. Raftery, "Model Selection and Accounting for Model Uncertainty using Occam's Window", *Journal of American Statistical Association*, Vol. 89, pp. 1535–1546, 1994.

8. Madigan, D., "Bayesian Interpolation", *Neural Computation*, Vol. 4, pp. 415–447, 1992.

9. Friedman, N. and D. Koller, "Being Bayesian About Network Structure. A Bayesian Approach to Structure Discovery in Bayesian Networks", *Machine Learning*, Vol. 50, pp. 95–125, 2003.

10. Clarke, B., "Comparing Bayes Model Averaging and Stacking When Model Approximation Error Cannot be Ignored", *Journal of Machine Learning Research*, Vol. 4, pp. 683–712, 2003.

11. Chickering, D. M., "Learning Equivalence Classes of Bayesian-Network Structures", *Journal of Machine Learning Research*, Vol. 2, pp. 445–498, 2002.

12. Andrieu, C., N. de Freitas, and A. Doucet, "Robust Full Bayesian Learning for Radial Basis Networks", *Neural Computation*, Vol. 13, pp. 2539–2407, 2001.

13. Vila, J., V. Wagner, and P. Neveu, "Bayesian Nonlinear Model Selection and Neural Networks: A Conjugate Prior Approach", *IEEE Transactions on Neural Networks*, Vol. 11, pp. 265–278, 2000.

14. Seeger, M., "Bayesian Model Selection for Support Vector Machines, Gaussian Processes, and Other Kernel Classifiers", *Proceedings of the Neural Informations Processing Systems*, 2000.

15. Rissanen, J., "Stochastic Complexity and Modeling", *Annals of Statistics*, Vol. 14, pp. 1080–1100, 1986.

16. Luxburg, U., O. Bousquet, and B. Schölkopf, "A Compression Approach to Support Vector Model Selection", *Journal of Machine Learning Research*, Vol. 5, pp. 293–323, 2004.

17. Schuurmans, D. and F. Southey, "Metric-Based Methods for Adaptive Model Selection and Regularization", *Machine Learning*, Vol. 48, pp. 51–84, 2002.

18. Schuurmans, D. and F. Southey, "An Adaptive Regularization Criterion for Supervised Learning", *Proceedings of the 17th International Conference on Machine Learning*, pp. 847–854, 2000.

19. Hsu, W. H., S. R. Ray, and D. C. Wilkins, "A Multistrategy Approach to Classifier Learning from Time Series", *Machine Learning*, Vol. 38, pp. 213–236, 2000.

20. Bengio, J. and N. Chapados, "Extensions to Metric-Based Model Selection", *Journal of Machine Learning Research*, Vol. 3, pp. 1209–1227, 2003.

21. Bach, R. B. and M. I. Jordan, "Learning Graphical Models with Mercer Kernels", *Proceedings of the Neural Informations Processing Systems*, 2002.

22. Chapelle, O., V. Vapnik, and Y. Bengio, "Model Selection for Small Sample Regression", *Machine Learning*, Vol. 48, pp. 9–23, 2002.

23. Sugiyama, M. and H. Ogawa, "Subspace Information Criterion for Model Selection", *Neural Computation*, Vol. 13, pp. 1863–1889, 2001.

24. Wahba, H., *Spline Model for Observational Data*, Society for Industrial and Applied Mathematics, 1990.

25. Sugiyama, M. and H. Ogawa, "Theoretical and Experimental Evaluation of the Subspace Information Criterion", *Machine Learning*, Vol. 48, pp. 25–50, 2002.

26. Friedman, J. H., "An Overview of Predictive Learning and Function Approximation", Cherkassky, V., J. H. Friedman, and H. Wechsler (editors), *From Statistics to Neural Networks*, Springer Verlag, New York, 1994.

27. Vapnik, V., *The Nature of Statistical Learning Theory*, Springer Verlag, New York, 1995.

28. Cherkassky, V. and F. Mulier, *Learning From Data*, John Wiley and Sons, 1998.

29. Cherkassky, V., X. Shao, F. M. Mulier, and V. Vapnik, "Model Complexity Control for Regression Using VC Generalization Bounds", *IEEE Transactions on Neural Networks*, Vol. 10, pp. 1075–1089, 1999.

30. Anthony, M. and P. L. Bartlett, *Neural Network Learning: Theoretical Foundations*, Cambridge University Press, 1999.

31. Antos, A., B. Kegl, T. Linder, and G. Lugosi, "Data-dependent Margin-based Generalization Bounds for Classification", *Journal of Machine Learning Research*, Vol. 3, pp. 73–98, 2002.

32. Shao, X., V. Cherkassky, and W. Li, "Measuring the VC-Dimension Using Optimized Experimental Design", *Neural Computation*, Vol. 12, pp. 1969–1986, 2000.

33. Vapnik, V., E. Levin, and Y. L. Cun, "Measuring the VC-dimension of a Learning Machine", *Neural Computation*, Vol. 6, pp. 851–876, 1994.

34. Kearns, M., Y. Mansour, A. Y. Ng, and D. Ron, "An Experimental and Theoretical Comparison of Model Selection Methods", *Machine Learning*, Vol. 27, pp. 7–50, 1997.

35. Bartlett, P. L., S. Boucheron, and G. Lugosi, "Model Selection and Error Estimation", *Machine Learning*, Vol. 48, pp. 85–113, 2002.

36. Breiman, L. and P. Spector, "Submodel Selection and Evaluation in Regression-the X-random Case", *International Review Statistics*, Vol. 3, pp. 291–319, 1992.

37. Dietterich, T. G., "Approximate Statistical Tests for Comparing Supervised Classification Learning Classifiers", *Neural Computation*, Vol. 10, pp. 1895–1923, 1998.

38. Efron, B., "Computers and the Theory of Statistics", *SIAM Review*, Vol. 21, pp. 460–480, 1979.

39. Efron, B., "Estimating the Error Rate of a Prediction Rule: Some Improvements on Cross-validation", *Journal of American Statistical Association*, Vol. 78, pp. 316–331, 1983.

40. Weiss, S. M., *Computer Systems that Learn*, Morgan Kaufmann, 1991.

41. Rivals, I. and L. Personnaz, "On Cross Validation for Model Selection", *Neural Computation*, Vol. 11, pp. 863–870, 1999.

42. Moore, A. W. and M. S. Lee, "Efficient Algorithms for Minimizing Cross Validation Error", *International Conference on Machine Learning*, 1994.

43. Alpaydın, E., "Combined 5×2 cv F Test for Comparing Supervised Classification Learning Classifiers", *Neural Computation*, Vol. 11, pp. 1975–1982, 1999.

44. Alpaydın, E., *Introduction to Machine Learning*, The MIT Press, 2004.

45. Miller, R. G., *Simultaneous Statistical Inference*, Springer Verlag, New York, 1981.

46. Dean, A. and D. Voss, *Design and Analysis of Experiments*, Springer Verlag, New York, 1999.

47. Hochberg, Y. and A. C. Tamhane, *Multiple Comparison Procedures*, John Wiley and Sons, 1987.

48. Conover, W. J., *Practical Nonparametric Statistics*, John Wiley and Sons, New York, 1999.

49. Turney, P. D., "Types of Cost in Inductive Concept Learning", *Workshop on Cost-Sensitive Learning in 17th International Conference on Machine Learning*, pp. 15–21, Stanford University, CA, 2000.

50. Holm, S., "A Simple Sequentially Rejective Multiple Test Procedure", *Scandinavian Journal of Statistics*, Vol. 6, pp. 65–70, 1979.

51. Rosen, K. H., *Discrete Mathematics and its Applications*, Mc Graw-Hill, New York, 1995.

52. Murthy, S. K., S. Kasif, and S. Salzberg, "A System for Induction of Oblique Decision Trees", *Journal of Artificial Intelligence Research*, Vol. 2, pp. 1–32, 1994.

53. Breslow, L. A. and D. W. Aha, "Simplifying Decision Trees: A Survey", Technical Report AIC-96-014, Navy Center for Applied Research in AI, Naval Research

Laboratory, Washington DC, USA, 1997.

54. Murthy, S. K., "Automatic Construction of Decision Trees from Data: A Multi-Disciplinary Survey", *Data Mininig and Knowledge Discovery*, Vol. 2, No. 4, pp. 345–389, 1998.

55. Lim, T. S., W. Y. Loh, and Y. S. Shih, "A Comparison of Prediction Accuracy, Complexity, and Training Time of Thirty-three Old and New Classification Algorithms", *Machine Learning*, Vol. 40, pp. 203–228, 2000.

56. Quinlan, J. R., "Induction of Decision Trees", *Machine Learning*, Vol. 1, pp. 81–106, 1986.

57. Breiman, L., J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*, John Wiley and Sons, 1984.

58. Quinlan, J. R., *C4.5: Programs for Machine Learning*, Morgan Kaufmann, San Meteo, CA, 1993.

59. Friedman, J. H., "A Recursive Partitioning Decision Rule for Non-parametric Classification", *IEEE Transactions on Computers*, pp. 404–408, 1977.

60. Loh, W. Y. and N. Vanichsetakul, "Tree-Structured Classification Via Generalized Discriminant Analysis", *Journal of the American Statistical Association*, Vol. 83, pp. 715–725, 1988.

61. Guo, H. and S. B. Gelfand, "Classification Trees with Neural Network Feature Extraction", *IEEE Transactions on Neural Networks*, Vol. 3, pp. 923–933, 1992.

62. Brodley, C. E. and P. E. Utgoff, "Multivariate Decision Trees", *Machine Learning*, Vol. 19, pp. 45–77, 1995.

63. Loh, W. Y. and Y. S. Shih, "Split Selection Methods for Classification Trees", *Statistica Sinica*, Vol. 7, pp. 815–840, 1997.

64. Gama, J., "Discriminant Trees", *16th International Conference on Machine Learning*, pp. 134–142, Morgan Kaufmann, New Brunswick, New Jersey, 1999.

65. Kim, H. and W. Loh, "Classification Trees with Unbiased Multiway Splits", *Journal of the American Statistical Association*, pp. 589–604, 2001.

66. Fürnkranz, J., "Separate-and-Conquer Learning", *Artificial Intelligence Review*, Vol. 13, pp. 3–54, 1999.

67. Cendrowska, J., "PRISM: An Algorithm for Inducing Modular Rules", *International Journal of Man-Machine Studies*, Vol. 27, pp. 349–370, 1987.

68. Webb, G. I. and N. Brkič, "Learning Decision Lists by Prepending Inferred Rules", *Proceedings of the AI'93 Workshop on Machine Learning and Hybrid Systems*, Melbourne, Australia, 1993.

69. Brunk, C. A. and M. J. Pazzani, "An Investigation of Noise-Tolerant Relational Concept Learning Algorithms", *Proceedings of the 8th International Workshop on Machine Learning*, pp. 389–393, 1991.

70. Fürnkranz, J. and G. Widmer, "Incremental Reduced Error Pruning", *11th International Conference on Machine Learning*, pp. 378–383, Morgan Kaufmann, New Brunswick, New Jersey, 1994.

71. Cohen, W. W., "Efficient Pruning Methods for Separate–and–Conquer Rule Learning Systems", *Proceedings of the 13th International Joint Conference on Artificial Intelligence*, pp. 988–994, 1993.

72. Weiss, S. M. and N. Indurkhaya, "Reduced Complexity Rule Induction", *Proceedings of the 12th International Joint Conference on Artificial Intelligence*, pp. 678–684, 1991.

73. Cohen, W. W., "Fast Effective Rule Induction", *The Twelfth International Conference on Machine Learning*, pp. 115–123, 1995.

74. Joshi, M. V., R. C. Agarwal, and V. Kumar, "Mining Needles in a Haystack: Classifying Rare Classes via Two-Phase Rule Induction", *Proceedings of ACM SIGMOD Conference*, pp. 91–102, Santa Barbara, CA, 2001.

75. Fensel, D. and M. Wiese, "Refinement of Rule Sets with JOJO", *Proceedings of the 6th European Conference on Machine Learning*, pp. 378–383, 1993.

76. Michalski, R. S., "On the Quasi-Minimal Solution of the Covering Problem", *Proceedings of the 5th International Symposium on Information Processing*, Vol. A3, pp. 125–128, Bled, Yugoslavia, 1969.

77. Michalski, R. S., I. Mozetič, J. Hong, and N. Lavrač, "The Multi-Purpose Incremental Learning System AQ15 and Its Testing Application to Three Medical Domains", *Proceedings of the 5th National Conference on Artificial Intelligence*, pp. 1041–1045, Philadelphia, PA, 1986.

78. Bloedorn, E. and R. S. Michalski, "Constructive Induction From Data in AQ17-DCI: Further Experiments", Technical Report MLI91-12, Artificial Intelligence Center, George Mason University, Fairfax, VA, 1991.

79. Clark, P. and T. Niblett, "The CN2 Induction Algorithm", *Machine Learning*, Vol. 3, pp. 261–283, 1989.

80. Bergadano, F., S. Matwin, R. S. Michalski, and J. Zhang, "Learning Two-Tiered Descriptions of Flexible Concepts: The POSEIDON System", *Machine Learning*, Vol. 8, pp. 5–43, 1992.

81. Theron, H. and I. Cloete, "BEXA: A Covering Algorithm for Learning Propositional Concept Descriptions", *Machine Learning*, Vol. 24, pp. 5–40, 1996.

82. Webb, G. I., "Learning Disjunctive Class Descriptions by Least Generalization", Technical Report TR C92/9, Deakin University, School of Computing & Mathematics, Geelong, Australia, 1992.

83. Chisholm, M. and P. Tadepalli, "Learning Decision Rules by Randomized Iterative Local Search", *Proceedings of the 19th International Conference on Machine Learning*, pp. 75–82, 2002.

84. Hart, P. E., N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths", *IEEE Transactions on Systems Science and Cybernetics*, Vol. 4, pp. 100–107, 1968.

85. Bergadano, F., A. Giordana, and L. Saitta, "Automated Concept Acquisition in Noisy Environments", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Vol. 10, pp. 555–578, 1988.

86. Muggleton, S. H., "Inverse Entailment and Progol", *New Generation Computing*, Vol. 13, pp. 245–286, 1995.

87. Pompe, U., M. Kocačič, and I. Kohonenko, "SFOIL: Stochastic Approach to Inductive Logic Programming", *Proceedings of the 2nd Slovenian Conference on Electrical Engineering and Computer Science*, Vol. B, pp. 189–192, Potorož, Slovenia, 1993.

88. Kovačič, M., *Stochastic Inductive Logic Programming*, Ph.D. thesis, Department of Computer and Information Science, University of Ljubljana, 1994.

89. Venturini, G., "SIA: A Supervised Inductive Algorithm with Genetic Search for Learning Attributes Based Concepts", *Proceedings of the 6th European Conference on Machine Learning*, pp. 280–296, Vienna, Austria, 1993.

90. Kohonenko, I. and M. Kovačič, "Learning as Optimization: Stochastic Generation of Multiple Knowledge", *Proceedings of the 9th International Workshop on Machine Learning*, pp. 257–262, 1992.

91. Quinlan, J. R. and R. L. Rivest, "Inferring Decision Trees using the Minimum Description Length Principle", *Information and Computation*, Vol. 80, pp. 227–

248, 1989.

92. Duda, R. O. and P. E. Hart, *Pattern Classification and Scene Analysis*, John Wiley and Sons, 1973.

93. Yıldız, O. T. and E. Alpaydın, "Omnivariate Decision Trees", *IEEE Transactions on Neural Networks*, Vol. 12, No. 6, pp. 1539–1546, 2001.

94. Blake, C. and C. Merz, "UCI Repository of Machine Learning Databases", 2000, `http://www.ics.uci.edu/~mlearn/MLRepository.html`.

95. Hinton, G. H., "Delve Project, Data for Evaluating Learning in Valid Experiments", 1996, `http://www.cs.utoronto.ca/~delve/`.